Implementation and Evaluation of two Automated Vehicle Stacks in a Simulation Environment based on Open Source Software

Master Thesis

written by

Stefan Teschl, BSc

at the Master's Degree Program
Electronics and Computer Engineering
of FH JOANNEUM – University of Applied Sciences, Austria

supervised by

FH-Prof. DI Dr. Christian Netzberger

external supervisor

DI Peter Priller

Graz, 26.11.2021

Obligatory signed declaration

I hereby declare that the present Master's thesis was composed by myself and that the work contained herein is my own and that I have only used the specified resources. I also confirm that I have prepared this thesis in compliance with the FH JOANNEUM Standards for Good Scientific Practice and Prevention of Research Misconduct. I declare in particular that I have cited all formulations and concepts taken verbatim or in substance from printed or unprinted material or from the Internet according to the rules of good scientific practice and that I have indicated them by footnotes or other exact references to the original source.

I understand that the provision of incorrect information may have legal consequences.

Graz, 26.11.2021

Abstract

Nowadays, advanced driving assistance systems have become a matter of course in road traffic. And in the future, they will automate driving more and more. There are several open-source automated driving stacks available, probably the best known being the two stacks from the Autoware Foundation.

In this thesis the two Robot Operating System (ROS) based automated driving stacks of the Autoware Foundation are explained and tested. First, the LIDAR perception and detection layer from Autoware.Auto will be examined and tested in combination with the environment simulation CARLA. Additionally, the included perception layer will be extended by a V2X sensor and compared with the LIDAR detection. And second, the full stack from Autoware.AI is explained and a self-driving demo in combination with CARLA will be set up.

The LIDAR perception of Autoware.Auto achieves good results in object detect, although the bounding boxes are not always set perfectly. The V2X sensor implementation receives the position, orientation, and dimensions of the vehicles from the CARLA simulation. The Autoware.AI stack achieved good results in the self-driving demo. The stack navigates and controls the vehicle in CARLA from point A to point B while preventing collisions against obstacles on the road.

Kurzfassung

Fahrerassistenzsysteme sind heute zu einer Selbstverständlichkeit im Straßenverkehr geworden. In Zukunft werden diese Systeme das Fahren mehr und mehr automatisieren. Es gibt verschiedene Open-Source-Stacks für automatisiertes Fahren, die wohl bekanntesten sind die beiden der Autoware Foundation.

In dieser Arbeit werden die beiden auf dem Robot Operating System (ROS) basierten Stacks der Autoware Foundation beschrieben und getestet. Zunächst wird die LIDAR Perception und Detection von Autoware.Auto in Kombination mit der Umweltsimulation CARLA erklärt und anschließend getestet. Zusätzlich wird der bereits inkludierte Perception Layer um einen V2X-Sensor erweitert und mit der LIDAR Detection verglichen. Anschließend wird der komplette Stack von Autoware.AI erklärt und eine selbstfahrende Demo im Kombination mit CARLA erstellt.

Die LIDAR Perception von Autoware.Auto erzielt gute Ergebnisse bei der Objekterkennung, obwohl die erstellten Boundingboxen nicht immer akkurat gesetzt werden. Die V2X Sensor Implementierung empfängt die Position, Orientierung und Abmessungen der Fahrzeuge von der CARLA Simulation. Der Autoware.AI-Stack erzielte gute Ergebnisse in der selbstfahrenden Demo. Es ist möglich, das Fahrzeug mithilfe von Autoware.AI von Punkt A nach Punkt B zu befördern. Zusätzlich verhindert der Stack Kollisionen mit anderen Objekten auf der Straße.

Table of contents

Table of figures

List of abbreviations

ROS	Robot Operating System
ADAS	Advanced Driver Assistance Systems
AD	Automated Driving
AV	Autonomous Vehicle
ACC	Automatic Cruise Control
LDW	Lane Departure Warning
PD	Pedestrian Detection
V2V	Vehicle to Vehicle
V2I	Vehicle to Infrastructure
V2X	Vehicle to Everything
API	Application Programming Interface
DDS	Data Distribution System
VM	Virtual Machine
OS	Operating System
DRSC	Direct Shortrange Communication
IEEE	Institute of Electrical and Electronics Engineers
ETSI	European Telecommunications Standards Institute
ITS	Intelligent Transportation Systems
ADE	Agile Development Environment
NPC	Non-Player Character
AI	Artificial Intelligence
GUI	Graphical User Interface
AVP	Automated Valet Parking
AABB	Axis-Aligned Bounding Box

OBB	Oriented Bounding Box
PCA	Principal Component Analysis
NDT	Normal Distribution Transform

1 Introduction

Nowadays, driving assistance systems have become a matter of course in road traffic. And in the future, they will automate driving more and more. There are several open-source automated driving stacks available, probably the best known being the two stacks from the Autoware Foundation. These two stacks are based on the Robot Operating System (ROS). Both will be independently implemented and tested on an Ubuntu system in combination with the environment simulator CARLA. Autoware.Auto was developed for a specific valet parking use case and uses the LIDAR sensor for detection. First, the functional basics of LIDAR perception and detection of Autoware.Auto are explained, then implemented and tested. In addition, the perception stack of Autoware.Auto will be extended with an ideal V2X sensor and the results will be compared with the LIDAR sensor detection. Second, the entire driving stack of Autoware.AI is explained, implemented and tested since Autoware.AI claims to be the first full self-driving open-source automated driving stack.

2 Fundamentals

2.1 Advanced Driver Assistance Systems (ADAS)

Advanced Driver Assistance Systems (ADAS) are designed to assist the driver in various driving situations. The aim of these systems is to minimize human errors on the road by giving feedback or even intervene in critical situations like an emergency brake. The system relies on the inputs of the sensors mounted on the vehicle such as, LIDAR, radar and cameras. [1]

2.1.1 Example Systems

Some example ADAS systems are listed below.

- Adaptive cruise control
- Adaptive light control
- Anti-lock braking system
- Automatic parking
- Collision avoidance
- Lane change assistance
- Lane keeping assistance

2.2 Automated Driving

Automated Driving (AD) or an Autonomous Vehicle (AV) is capable of sensing its environment and moving safely from point A to point B with no or just little input from a human being. The world's various official authorities are trying to gradually smooth the way for automated vehicles. Therefore, they are trying to introduce plans and laws which standardize testing and in-vehicle technology. Automated Driving is designed to improve the following points. [2]

- Reducing human driving errors
- Optimization of traffic flow
- Optimization of fuel consumption and CO_2 emissions
- Enhancing of the mobility for unexperienced drivers or elderly people

The reason behind the significant increase in traffic safety is that the algorithms which are powering the automated driving functions are in some cases much more effective than a human being could ever be [3] [1]. The advanced driving assistance system of today will form the foundation of the automated mobility of tomorrow. Some of the ADAS systems are listed below. [4]

- Automatic Cruise Control (ACC)
- Lane Departure Warning (LDW)
- Pedestrian Detection (PD)

Vehicles will additionally communicate with each other, and exchange relevant information like their position, speed, lane position, traffic data and much more. This is called Vehicle-to-Vehicle (V2V) communication. In addition, infrastructure like traffic lights or busy intersections can then communicate with the vehicles to control the traffic flow and decrease the risk for a traffic jam, also known as Vehicle-to-Infrastructure (V2I). In general, this technology is known as Vehicle-to-Everything (V2X). [4]

2.2.1 Levels of Automation

The following figure shows the levels of automation categorized by SAE International and published in the SAE J3016 standard. It spans from no automation to full automation, and it defines the levels as well as the responsibilities of the underlaying driving assistance functions. Up until level 3 the driver must constantly supervise the support of the ADAS features like lane centering or adaptive cruise control. In level 3 the vehicle driver does not need to drive the car actively anymore, if the automated driving features are enabled. However, if the feature requests the human driver, he must take control over the vehicle. Level 4 and level 5 will drive totally autonomous without any human interaction needed. [5]

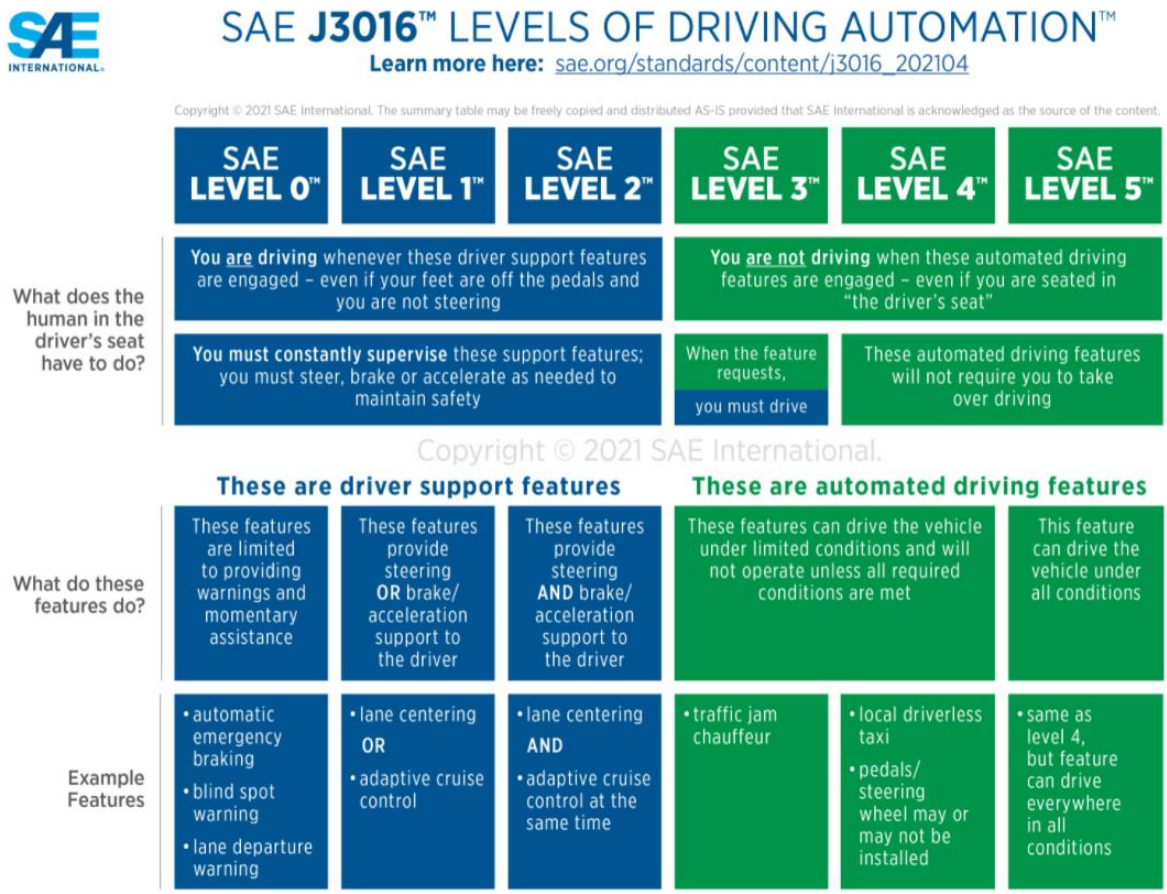

Figure 1: SAE J3016 levels of driving automation [6]

2.2.2 Layers of an Automated Driving Stack

The next figure shows the layers of the stack. The foundation is the vehicle control layer where braking, steering and the general control of the actuators of the car takes place. The sensing layer is directly above and includes sensors like radar, camera, LIDAR, GPS and maps are placed, additionally the V2X sensors are in this layer. Above these layers is the processing and decision-making layer, where the actual intelligence of the car is placed. Here reasoning and decision-making happens. [3]

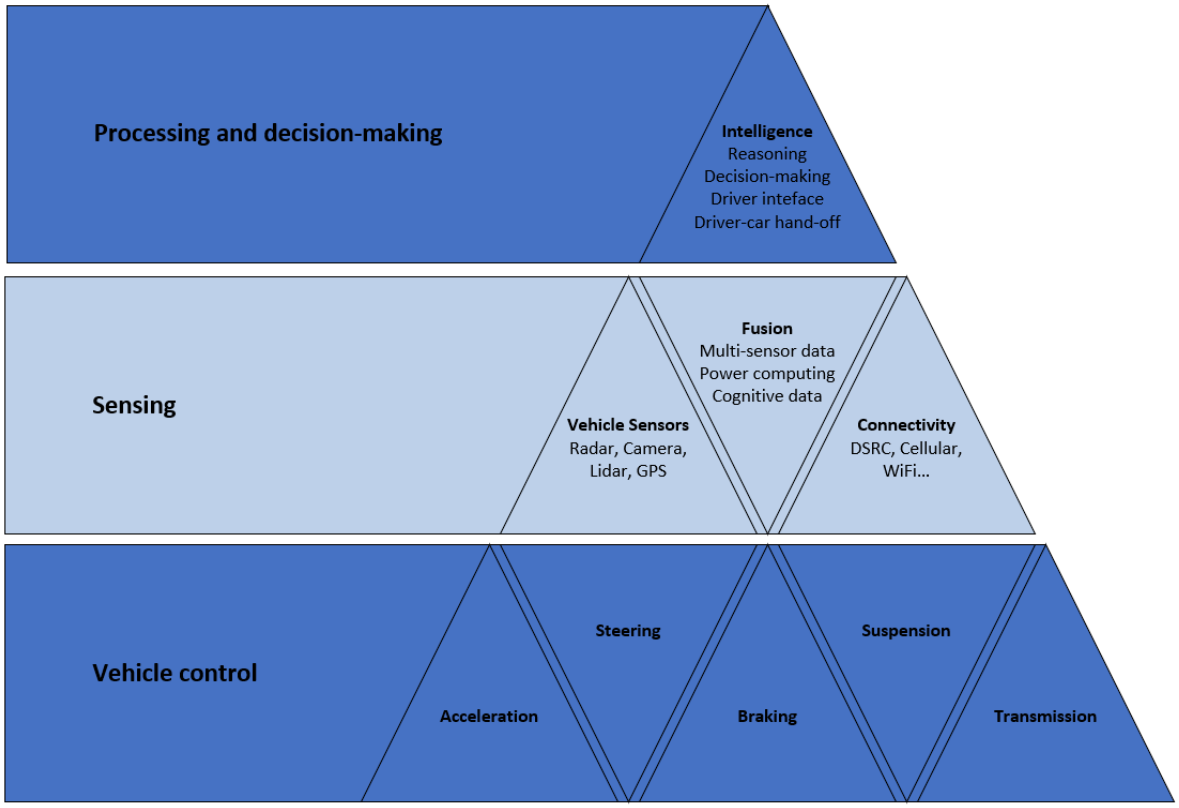

Figure 2: Layers of an Automated Driving Stack own portrayal after [3]

2.3 Robot Operating System (ROS)

Robot Operating System (ROS) is an open-source based robotics middleware developed by the Open Source Robotics Foundation. It provides services like package management and message passing between processes and can be used to connect different computers or sensors through a defined network interface. The principle is that nodes can either publish or subscribe to a certain topic. Python or C++ can be used to create custom code which can then be executed on a node in ROS. When running a set of processes with ROS they are represented in a graph architecture where each node will process its own task. It is not an operating system as the name implies, it is more of a software framework for robot development. [7]

2.3.1 ROS Versions

There are two ROS versions in existence, ROS and ROS2 whereby both versions got developed with different requirements in mind.

ROS

The Robot Operating System was developed for specific use cases in mind. The goal was to develop software tools which support research and development projects with the Willow Garage PR2 robot. In parallel they knew that the PR2 will not stay the only robot where the operating system will be deployed in the future. Therefore, they developed ROS with a lot of abstraction by defining message interfaces which allow the software to be reused on different robots and machines. ROS was developed guided by the following use cases. [7]

- A single robot
- No real-time requirements
- Excellent network connectivity
- Applications in research
- Maximum flexibility

ROS satisfied all requirements for the PR2, but today it is not used only by the PR2, it is used beyond the academic research community which was the initial focus for it. ROS based products which are commonly in use are manufacturing robots, agriculture robots, commercial cleaning robots, and others. Since there are so many new use cases for ROS, the platform was stretched in so many unexpected ways while still holding up well. Due to the fact that the requirements became higher and higher and new technologies arrived, ROS2 was created. [7]

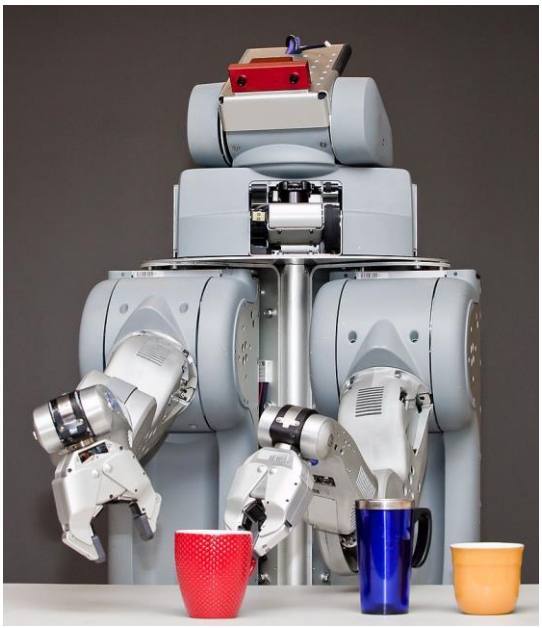

Figure 3: Willow Garage PR2 Robot [8]

ROS2

ROS2 was developed for new use cases in mind, additionally new technology was available at this point. First the development team thought of expanding the existing ROS Core for all the new use cases but came to the decision to open a totally new branch with different packages for ROS2. The problem was that too many people would get involved when expanding the original ROS Core and too many dependencies would have been created. Below are some of the new use cases listed. [7]

- Teams of multiple robots
- Small embedded platforms
- Real-time systems
- Non-ideal networks
- Production environments

2.3.2 Nodes

A ROS node is a process which performs computations. Usually, a robot consists of many nodes whereby each node is responsible for a certain task, like controlling an electric- or a hydraulic engine. The different nodes can communicate which each other through different services. [9]

2.3.3 Topics

Topics are named busses over which nodes exchange messages. The topics will be published and subscribed anonymously. The node will not know with which node it is communicating with, it will only know the topic which it has subscribed to. When subscribed to a topic, the node will get an update of the topic periodically. Below is an illustration of how the nodes will communicate with each other. [10]

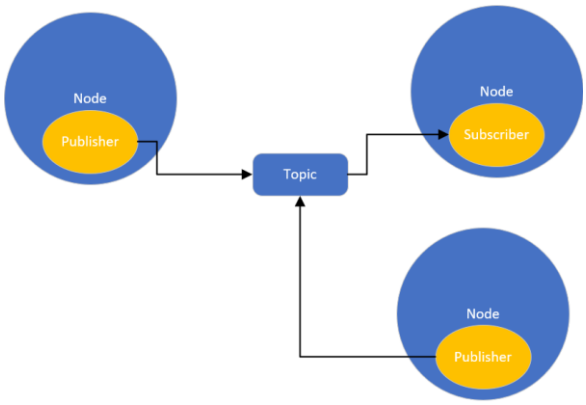

Figure 4: ROS node communication own portrayal after [11]

2.3.4 Services

The difference between services and topics is that a service is built on a request and reply base. The service client only requests data when needed and will get the response afterwards whereby the topic gets published periodically. The service will lower the network traffic and therefore increase the overall performance of the system. Below the illustration with the service client. [12]

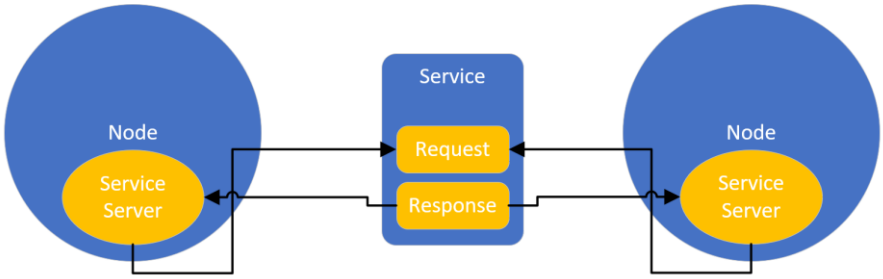

Figure 5: ROS node service own portrayal after [13]

2.3.5 Parameters

In each node parameters can be created and set. A node can be configured with parameters, for instance a parameter for speed or the direction of a moving object. The parameters can be controlled via get/set commands, either in the command line or with one of the ROS Application Programming Interfaces (APIs). One can see the parameters like global variables. [14]

2.3.6 Actions

When publishing a topic, the communication is linear and therefore only in one direction, with ROS services it is possible to request a service and get a response. But there is no information about the actual progress of the requested service. When a ROS action is performed the action server sends continuous feedback to the client to inform about the actual status of the operation requested. This is especially useful if a request will take time and will not be finished instantaneously. The action server provides an action which has a name and a type. Multiple actions are allowed with the same name under a different namespace. The action client sends the desired goal (action to be performed) and monitors the progress. The action is depicted in Figure 6 below. [15]

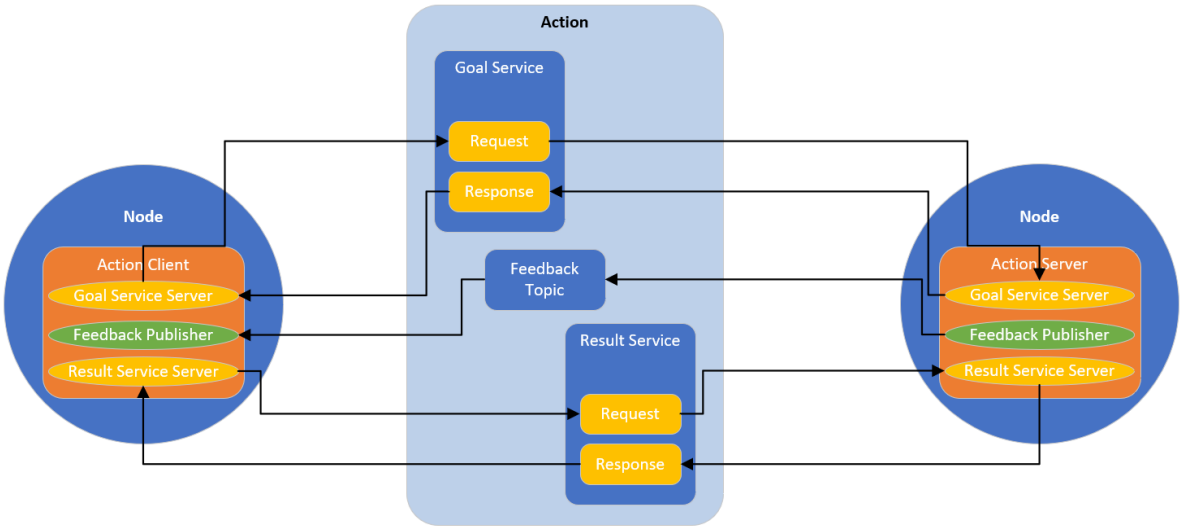

Figure 6: ROS node action own portrayal after [16]

2.3.7 Python API

The official supported Python API is *rospy*, it enables programmers to interface with ROS topics, services, and parameters in a quick and easy manner. But the design of *rospy* favors implementation speed over the runtime performance, therefore most of the performance-oriented code is written in C++. [17]

2.3.8 C++ API

The official high-performance library is *roscpp*, which does also supply access to topics, services and parameters. [18]

2.3.9 Data Distribution System (DDS)

In ROS no external middleware was used, the communication in general was handled by the ROS Core. But for the second generation the Open Source Robotics Foundation decided to build ROS2 on top of an existing middleware solution (Data Distribution System). The advantage is that ROS2 can use an existing well developed and tested implementation of that standard. ROS2 now aims to support multiple DDS implementations, for this reason the middleware interface was developed. And for each DDS implementation there is a specific middleware interface. The structure of the ROS2 middleware implementation is pictured below. [19]

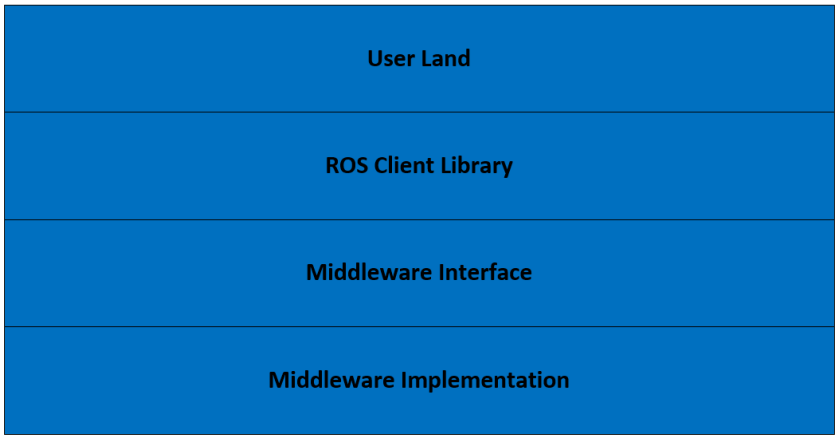

Figure 7: ROS2 layers own portrayal after [19]

The Data Distribution System provides a publish-subscribe transport which is quite similar to the ROS Core publish-subscribe transport from the first generation of ROS. But the benefit of DDS is that the ROS2 development team has much less code to maintain, and the behavior and the specifications are already documented. So, they can rely on all the tested functionality of DDS. [20]

2.3.10 ROS Bridge

To communicate between the two ROS versions a bridge is needed. It exchanges the messages between ROS and ROS2. The code is open-source, therefore it is possible to bridge data types which were created by the user by simply adding it to the source code and recompiling it. [21]

2.3.11 RVIZ

RVIZ is based on ROS and is a three-dimensional visualizer used to visualize robots and the environment they work in. It accesses ROS via a build in ROS-Bridge and can therefore visualize various data from ROS. It can also visualize all necessary data needed for Autoware, for instance LIDAR point clouds, images, bounding boxes of detected objects. The ROS python API can be used to publish data onto ROS that RVIZ can then visualize. RVIZ is available for ROS and ROS2. [22]

2.4 Docker

Docker is an open-source containerization platform. It allows to package an application into containers in comparison with a Virtual Machine (VM) the Docker image or container only represents the user space of the Operating System (OS) therefore the containers need less space than a VM image. The benefits of a Docker in comparison of a VM are listed below. [23]

Lighter Weight

Containers do not carry the payload of an entire OS instance and the container sizes are in the megabyte range and not in the gigabyte range as some VM images are. [23]

Resource Efficiency

A host pc can run many more docker images than VM images at once. [23]

Developer Productivity

Docker containers are faster and easier to deploy than VMs, this makes them easy to use for developers. Developers can also share the containers faster since they are smaller in space. [23]

2.5 V2X

Vehicle-to-Anything (V2X) is the connection between road users with the 802.11p wireless standard. With V2X, applications like platooning, congestion prevention and various safety features can be implemented into the vehicles. For this application a new standard for the automotive industry was created. To connect all the road users in the United States, Direct Shortrange Communication (DRSC) is used. In Europe, DRSC is known as ITS-G5. The Institute of Electrical and Electronics Engineers (IEEE) introduced the wireless standard 802.11p which is built on the 802.11a standard with several changes. For example, the bandwidth was reduced from 20 MHz to 10 MHz to ensure a larger guard band. Additionally, the transmission frequency was increased to 5.9 GHz. [3] [24] [25]

Among other things, the Physical Layer was also adopted to cover the high-speed requirements of the vehicles. The Doppler Effect on moving objects creates undesirable signal effects. Therefore, the guard interval, the time that elapses between transmissions, i.e. where there is no transmission, was increased from 0.8µs to 1.6µs. This precaution was taken to make the signal more robust. And the maximum data rate was halved from 54 Mbit/s to 27 Mbit/s. [26]

Three frequency bands have been allocated by the European Telecommunications Standards Institute (ETSI). Each of it serves a different purpose which is shown in the following table.

Table 1: ITS-G5 standards [27]

Standard	Frequency in GHz
ITS-G5A for safety related applications.	5.875-5.905
ITS-G5B for non-safety related applications	5.855-5.875
ITS-G5C for ITS applications	5.470-5.725

As can be seen in Table 1, the European Telecommunications Standards Institute (ETSI) allocated three frequency bands, Band A for safety related applications, Band B for non-safety related applications and Band C for Intelligent Transportation Systems (ITS).

2.6 Autoware

Autoware is a ROS/ROS2 based open-source software, which enables self-driving vehicles. It includes the software for an Autonomous Driving Stack like sensing, perception, decision, and vehicle control. Autoware was created by the Autoware Foundation and has multiple branches. Autoware.AI is based on ROS and Autoware.Auto is based on ROS2. Both were developed for different specific use cases, Autoware.Auto for instance was created for Valet Parking. [28]

Figure 8: Logo of the Autoware Foundation [28]

2.6.1 Versions

Autoware.AI

Autoware.AI was the first and therefore the original Autoware project from the Autoware Foundation, it is based on ROS and was launched as a research and development platform. It was the first "All-in-One" open-source software for autonomous driving. It contains modules for localization, detection, prediction, and planning. [29]

Autoware.Auto

Autoware.Auto is the successor from Autoware.AI but it is already based on ROS2 and managed by an open-source community manager. It applies best-in-class software engineering practices. The initial example use cases were Autonomous Valet Parking and Autonomous Depot Maneuvering. [28]

2.7 CARLA

CARLA is an open-source simulator for autonomous driving based on the Unreal Engine, it has been developed to support training, development and validation of autonomous driving systems. It provides

buildings, roads and cars which are free to use. In includes built-in sensors like LIDAR, camera, GNSS and depth sensors. The whole simulator can be controlled via the Python API for CARLA, and all the sensor data can be published onto ROS/ROS2 via the ROS-Bridge from CARLA. Therefore, the car inside the simulation can also be controlled from an AD Stack running on ROS/ROS2. [30] [31]

Figure 9: CARLA Simulator

2.7.1 CARLA Sensors

Since CARLA is an open-source project it allows you to implement your own sensors, to do so the Unreal Engine including the CARLA source code needs to be downloaded and the sensor part needs to be expanded. The already included sensors are described below.

LIDAR

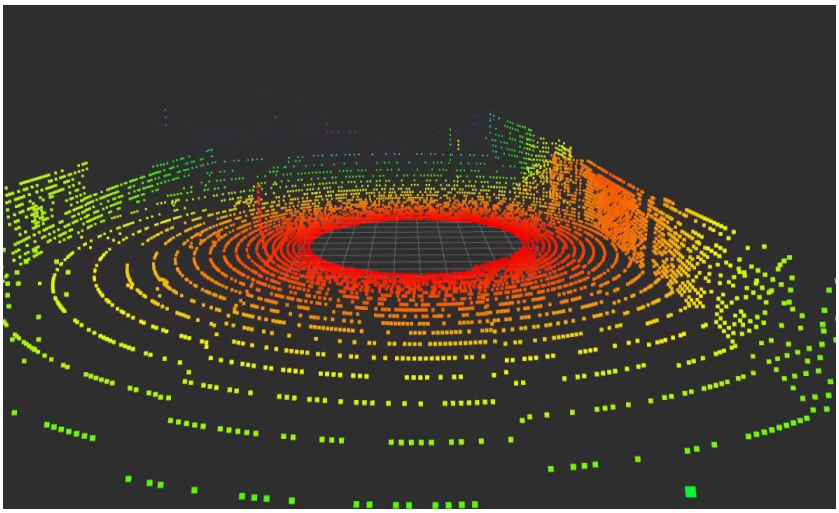

Figure 10: LIDAR sensor from CARLA in RVIZ

Visible in Figure 10 is the point cloud from the CARLA LIDAR sensor, it is published on ROS2 through the CARLA ROS Bridge in the poincloud2 format. The sensor simulates a rotating LIDAR using ray-casting.

CAMERA

Figure 11: CARLA RGB front camera

The RGB front camera is a regular camera capturing images from the scene in front of the ego-vehicle.

RADAR

Figure 12: RADAR overlay on camera

The radar sensor creates a conic view whereby it is translated into a 2D point map. CARLA provides more sensors than the sensors shown above which are listed below:

Collision detector, Depth Camera, GNSS Sensors, IMU Sensor, Lane invasion detector, Obstacle detector, RSS Sensor, Semantic LIDAR, Semantic segmentation camera, DVS camera, Optical-flow camera.

Whereby all sensors which get information directly from the gaming engine are less interesting for an AD Stack since in real life the AD Stack would not get the information about an obstacle or a lane invasion before the car has even sensed the object with its own sensors.

2.8 Setup

In this chapter the setup under which both stacks were implemented is explained.

2.8.1 Autoware.Auto

The following table shows the software installed for this thesis.

Table 2: Software versions installed

Autoware.Auto 1.0.0 Docker				
	OS	Docker	ROS2	CARLA
Version	Ubuntu 18.04	20.10.7	Foxy	1.9.11

The table above shows the versions used for the Autoware.Auto implementation in this thesis. The install process for Autoware.Auto is more complicated compared to the process needed to install Autoware.AI. The docker images are handled by an Agile Development Environment (ADE). Therefore, the installation process will be explained in more detail below.

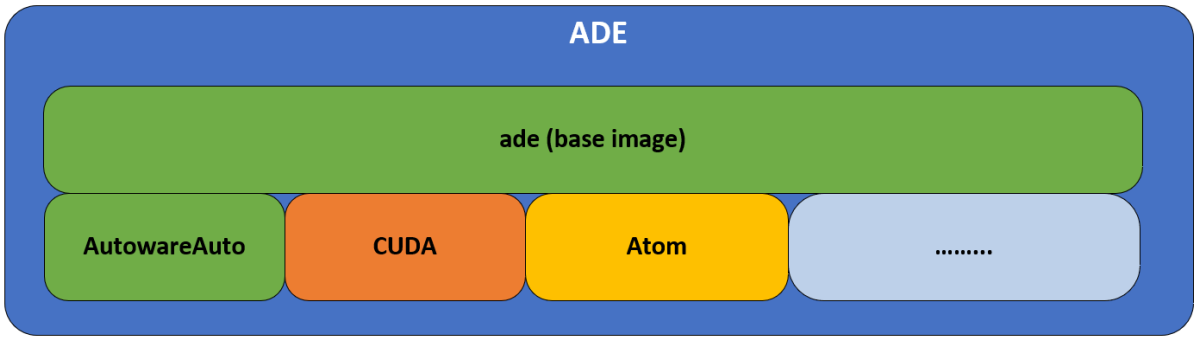

Figure 13: ADE image

As can be seen in Figure 13 the ADE manages the installation, it will pull docker containers with the preinstalled ROS distribution and Autoware.Auto, additionally packages like CUDA or Atom can be included. The downloaded folder will include *.aderc* files shown below.

Figure 14: aderc file for Autoware.Auto

The *.aderc* file will start the docker with the arguments suited to the docker volumes for the given release. Once the ADE is started it will pull the images for the Autoware.Auto distribution and for ROS2 Foxy. Additionally, the LGSVL simulator will be installed, however we do not have use for it since CARLA is used as simulation environment in this thesis. To start the containers the following command visible in the figure below is used.

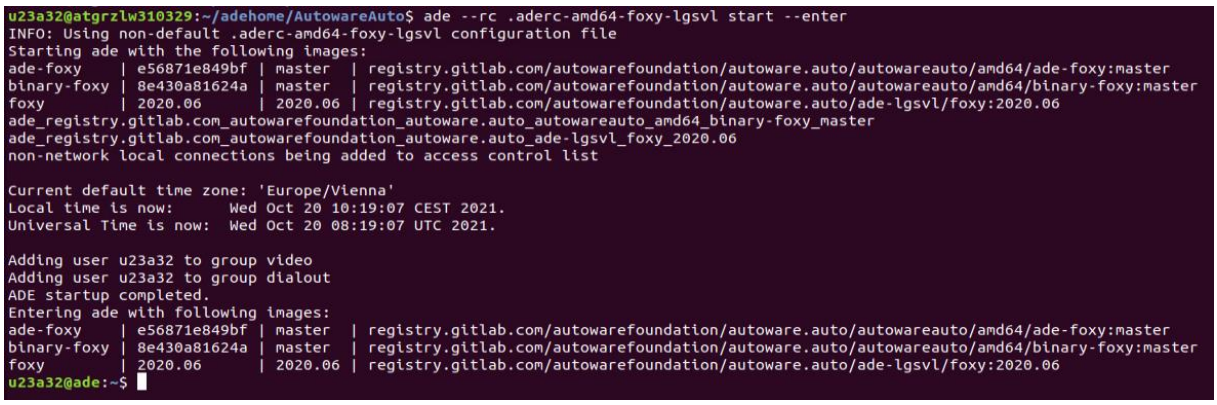

Figure 15: Starting ADE with images

The command in Figure 15 will start the ADE and it will pull the images specified in the *.aderc* file. Once the container is started the docker console will appear. You can simply start a new terminal command shell by entering *ade enter* and the containers can be stopped with *ade stop*. Important to know is that nothing in-between *ade start* and *ade stop* is persistent, this means all packages or libraries installed in between these two commands will not be available the next time the ADE with its containers is started. The ADE will always pull the images defined in the *.aderc* file which are the original images from the online repository. To get around this and make data persistent the ADE is installed inside an adehome folder on the host PC which is also the home directory inside the docker container. Therefore, once we save any file within the docker container into its home directory, it will also be available to the host PC where the docker container is running. CARLA communicates with ROS2 via the CARLA ROS

Bridge. This bridge is not installed on default, therefore all the libraries needed for it are installed at the docker container. Since the libraries inside the docker container are not stored in the home directory of the container, they are not persistent. Therefore, after installing libraries the docker container needs to be converted to a docker image. The new image then includes the previously installed libraries and can be shared to a different PC the next time. How the container is converted to an image will now be explained. The command to convert a docker container into an image is: *docker commit*

```
u23a32@atgrzlw310329:~/adehome/images_relevance$ docker ps
CONTAINER ID   IMAGE                                                                                                COMMAND                  CREATE
D          STATUS          PORTS     NAMES
cfdf35909cb6   registry.gitlab.com/autowarefoundation/autoware.auto/autowareauto/amd64/ade-foxy:master      "/ade_entrypoint /bi…"   12 min
utes ago   Up 12 minutes             ade
2aa45e748342   registry.gitlab.com/autowarefoundation/autoware.auto/ade-lgsvl/foxy:2020.06                 "/bin/sh -c 'trap 'e…"   12 min
utes ago   Up 12 minutes             ade_registry.gitlab.com_autowarefoundation_autoware.auto_ade-lgsvl_foxy_2020.06
1a126f55567a   registry.gitlab.com/autowarefoundation/autoware.auto/autowareauto/amd64/binary-foxy:master   "/bin/sh -c 'trap 'e…"   12 min
utes ago   Up 12 minutes             ade_registry.gitlab.com_autowarefoundation_autoware.auto_autowareauto_amd64_binary-foxy_master
```

Figure 16: Original docker containers

Above are three containers visible, all three of them have been committed with the command *docker commit* followed by the container ID. The created images are visible below and labeled with relevance in their name.

```
u23a32@atgrzlw310329:~/adehome/AutowareAuto$ docker images
REPOSITORY                                                                                TAG       IMAGE ID       CREATED          SIZE
ade-binary-foxy-relevance                                                                 version1  8e9afb3e743b   22 seconds ago   170MB
ade-lgsvl-foxy-relevance                                                                  version1  2263584645f1   57 seconds ago   251MB
relevance/autoware                                                                        version1  8447ca09c78e   22 minutes ago   9.27GB
registry.gitlab.com/autowarefoundation/autoware.auto/autowareauto/amd64/binary-foxy       master    6454a964d444   40 hours ago     170MB
registry.gitlab.com/autowarefoundation/autoware.auto/autowareauto/amd64/ade-foxy          master    63b9ade35412   8 days ago       5.1GB
registry.gitlab.com/autowarefoundation/autoware.auto/ade-lgsvl/foxy                       2020.06   7be8da9ce3bb   4 months ago     251MB
u23a32@atgrzlw310329:~/adehome/AutowareAuto$
```

Figure 17: Custom docker images

Since all the containers with the installed libraries are converted to images, they can be easily shared between PCs now. These images can always be used to get a fresh start with a working container. To use this docker images the *.aderc* file needs to be changed in order to use the right images visible in the next image below.

```
Open   .aderc-amd64-foxy-lgsvl-relevance   Save
~/adehome/AutowareAuto
export ADE_DOCKER_RUN_ARGS="--cap-add=SYS_PTRACE --net=host --privileged --add-host ade:127.0.0.1 -e RMW_IMPLEMENTATION=rmw_cyclonedds_cpp"
export ADE_GITLAB=gitlab.com
export ADE_REGISTRY=registry.gitlab.com
export ADE_IMAGES="
  relevance/autoware:version1
  ade-binary-foxy-relevance:version1
  ade-lgsvl-foxy-relevance:version1
"
```

Figure 18: Custom *.aderc* file with new images

As can be seen in Figure 18 the ADE images have been replaced to the custom created images before. Now when the image is started the libraries for the CARLA ROS Bridge are already installed and ready to go. The simulator CARLA was installed from the official website locally on the host PC and not on the ADE container. To run CARLA in server mode the following command is used:

./CarlaUE4.sh -carla-server

After CARLA is running in server mode, the CARLA ROS Bridge can be launched with an ego-vehicle.

ros2 launch carla_ros_bridge carla_ros_bridge_with_example_ego_vehicle.launch.py

The command above launches the ROS bridge between CARLA and ROS Foxy and sets up a specific town, car and sensors within the simulation environment CARLA.

Figure 19: CARLA ROS Bridge on the left and the CARLA server on the right

As can be seen in Figure 19 the command launches an ego-vehicle within a certain town, the simulator server can then be used to observe the whole traffic situation from an eagle's eye view. To spawn different actors like cars and pedestrians the CARLA API can be used with the following command:

python3 spawn_npc.py -n 80

Now 80 actor's or also known as non-player-character (NPC) are spawned on the map visible in Figure 20. The cars are controlled by a rule based algorithm, the pedestrians are controlled by an Artificial Intelligence (AI).

Figure 20: Spawned NPCs

To start a node/function from Autoware.Auto the launch files from ROS2 can be used and are written in python. The command to execute a launch file is the following: *ros2 launch <launchfile.launch>* An example of a launch file is below it starts the euclidean cluster node from Autoware.Auto.

```
def generate_launch_description():
    """Launch all the needed perception nodes"""
    # euclidean cluster node execution definition.
    euclidean_cluster_node_runner = launch_ros.actions.Node(
        package='euclidean_cluster_nodes',
        executable='euclidean_cluster_node_exe',
        namespace='lidar_front',
        parameters=[get_param_file('euclidean_cluster_nodes',
            'vlp16_sim_lexus_cluster.param.yaml')],
        remappings=[
            ("points_in", "points_nonground"),
            ("points_clustered", "cluster_points")  #3
        ])
```

2.8.2 Autoware.AI

Table 3: Software Versions Installed

Autoware.AI Version 1.14.0 Docker				
	OS	Docker	ROS	CARLA
Version	Ubuntu 18.04	20.10.7	Melodic	1.9.10.1

The whole system was downloaded as a docker image whereby all the necessary parts like ROS or libraries were already installed. The general design of Autoware.AI is shown in the image below.

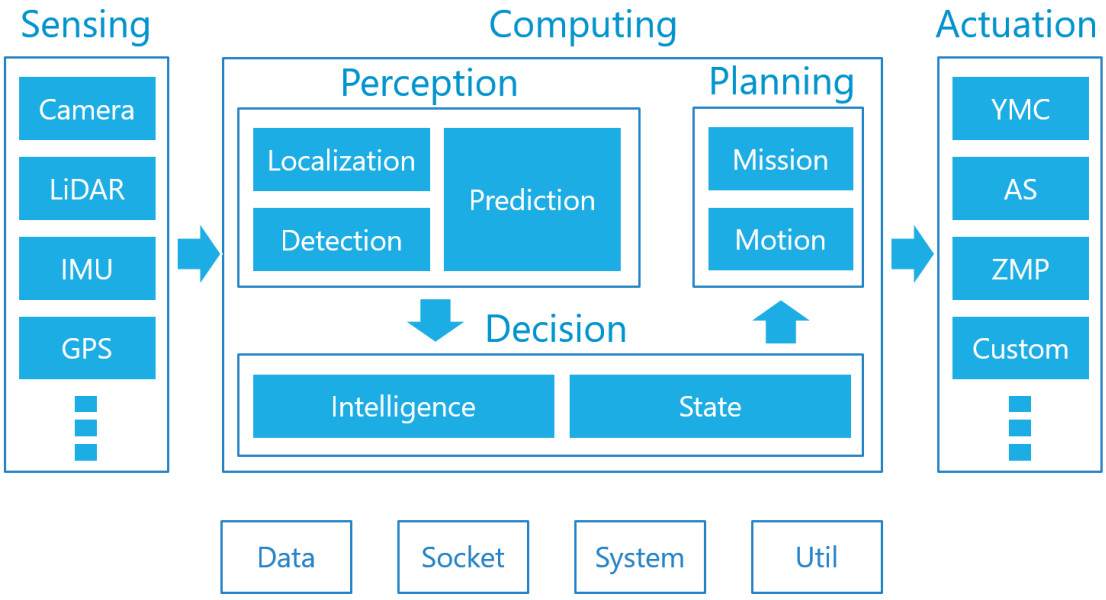

Figure 21: Autoware.AI Design [32]

Above is the basic design of Autoware.AI, it can be subdivided into three general categories, sensing computing and actuation. Sensing delivers the data to the computing block where the data is processed. The computing block can be split into perception, planning and decision. When all the decisions are made and the mission and motion is planned the commands are passed to the actuation layer. As the name already implies this layer controls the car either in simulation or a real vehicle. Autoware.AI comes with a runtime manager which allows to use the AD stack with a Graphical User Interface (GUI).

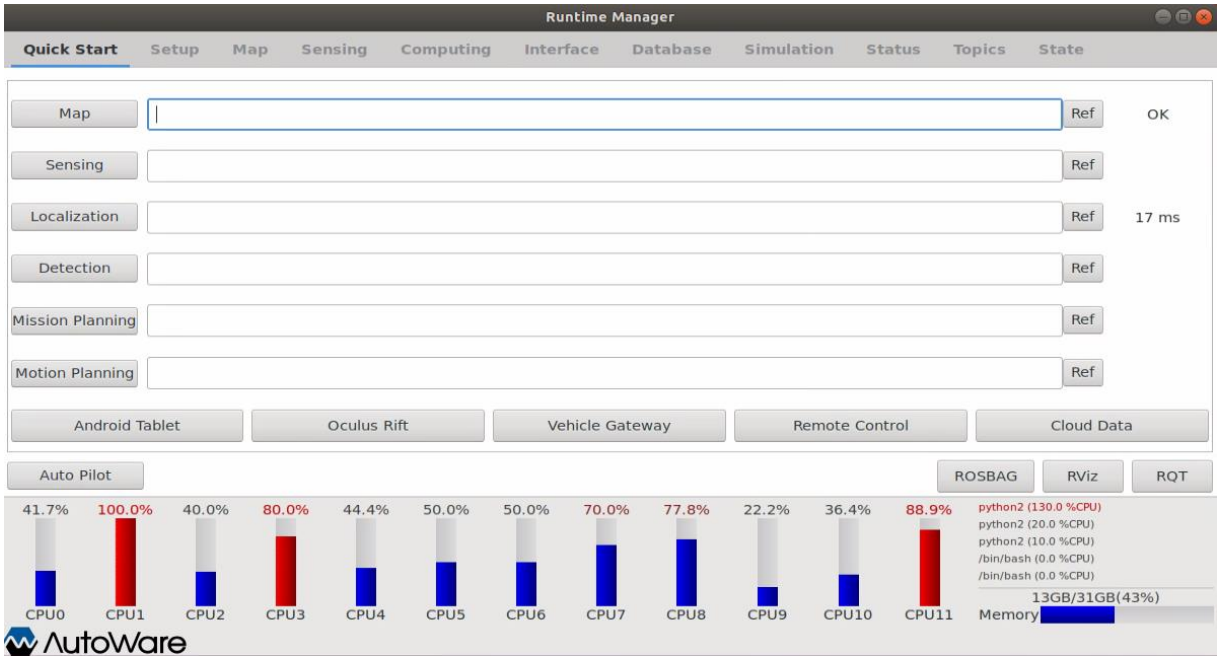

Figure 22: Runtime Manager Autoware.AI Quick Start Tab

The runtime manager can start different launch files on the quick start tab. Launch files can launch either nodes on ROS, or they start launch files which then start nodes on ROS. The nodes are the functions included in Autoware.AI. Nodes can be euclidean clustering or object detection via camera. In the launch files you can define the node you want to start. Below is an example of a launch file.

```
<launch>
  <!-- ndt_matching -->
  <include file="$(find lidar_localizer)/launch/ndt_matching.launch">
    <arg name="get_height" value="$(arg get_height)" />
  </include>
</launch>
```

A launch file can look like the code above, in Autoware.AI the launch files are written in the XML format since this is the format for ROS. The code above will start the ndt_matching.launch file which starts the ndt_matching node from Autoware.AI.

3 Autoware.Auto Implementation

In this chapter the LIDAR object detection stack from Autoware.Auto is explained and then implemented. Autoware.Auto or CARLA do not include V2X sensors yet, therefore the perception layer of Autoware.Auto is extended for the use of an V2X sensor. Autoware.Auto itself does not come with an object detection based on camera, it was primarily developed for Automated Valet Parking (AVP) and uses the LIDAR for detection and localization.

3.1 LIDAR Object Detection Basics

To detect objects with a LIDAR sensor preprocessing is needed to ensure a proper detection afterwards. After preprocessing the processed points are subdivided into ground, and non-ground points. Non-ground points are usually objects in the surrounding of the vehicle where the LIDAR is mounted. Ground points on the other side are the road where the vehicle is driving on. The next figure shows the basic block diagram of a LIDAR object detection.

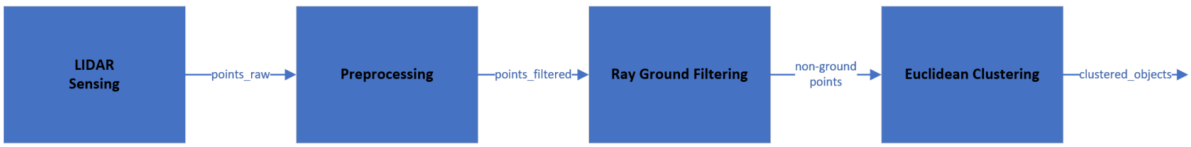

Figure 23: Block diagram object detection LIDAR

As can be seen in Figure 23 the first block is the LIDAR sensor which senses the raw points from its environment. Then the points get processed by the preprocessing block which can remove useless or problematic data. Afterwards the filtered points are sent into the ray ground filter to distinguish between ground and non-ground points. Once the ray ground filtering is done the euclidean cluster algorithm clusters the non-ground points to objects.

3.1.1 Preprocessing

Preprocessing can remove useless, problematic, or redundant data. The goal of preprocessing is to use the minimum amount of data to get the right results. By reducing the amount of data, the computational effort needed for the operations automatically decreases with it. Therefore, it is only an advantage to preprocess the LIDAR data. [33]

3.1.2 Range Based Filtering

Sometimes captured points far away from the ego-vehicle are less important. For example, while parking or while driving in an urban environment. Therefore, range-based filtering is used to decrease the amount of data that the LIDAR detection stack has to compute. [33]

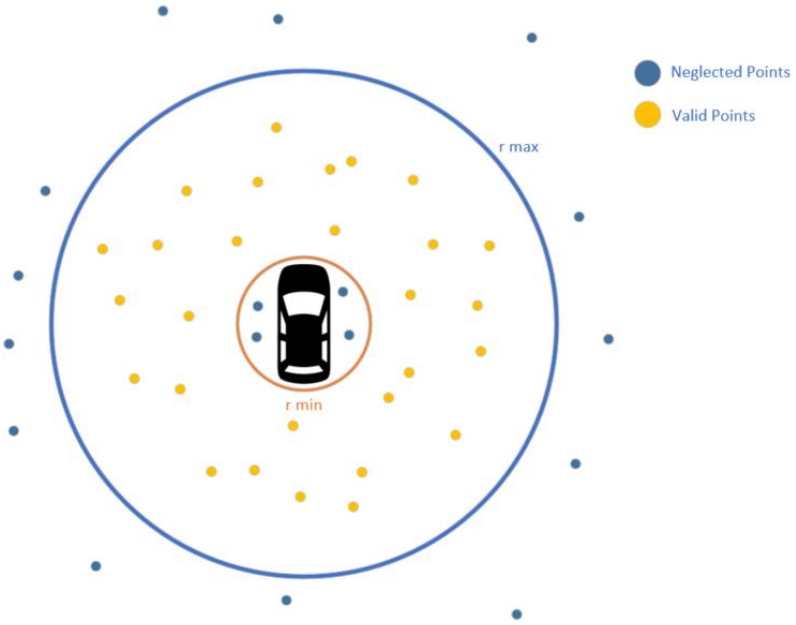

Figure 24: Range based filtering own portrayal after [33]

As can be seen in Figure 24, points outside of a certain range $r\ max$ are neglected. Sometime the LIDAR senses the roof or the engine hood of the vehicle where it is mounted to, to get around this a minimum radius $r\ min$ is used, whereby all points within $r\ min$ are neglected. [33]

3.1.3 Angle Based Filtering

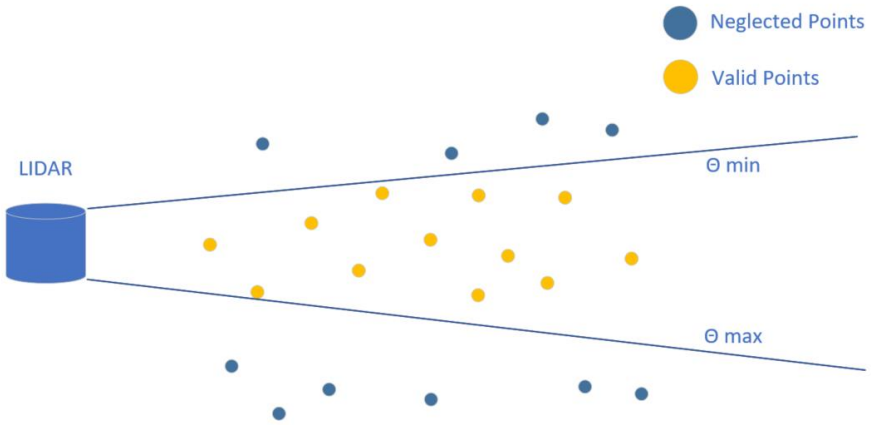

Figure 25: Angle based filtering own portrayal after [33]

With angle-based filtering all points captured outside of θ_{min} and θ_{max} are neglected. For instance, in highway situation where only one direction of travel is interesting for the ego-vehicle.

3.1.4 Down sampling

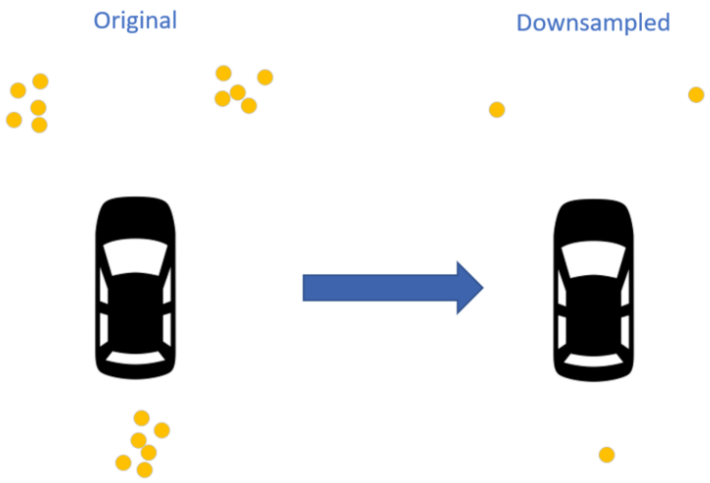

Figure 26: LIDAR down sampling own portrayal after [33]

Visible in Figure 26 is that the clustered points from the left side of the figure got down sampled by the algorithm, it reduces the computational complexity. The most widely used algorithm for this is the voxel grid approach, whereby a grid is placed over the LIDAR points, and all points within a certain box in the grid are then represented as one point with the corresponding grid coordinates. [33]

3.1.5 Standard Ray Ground Filtering

The ray ground filter distinguishes between the ground and non-ground points of the point cloud captured by the LIDAR. Since it is fast in comparison to other algorithms like factor graphs or voxels, Autoware.Auto uses it. [33]

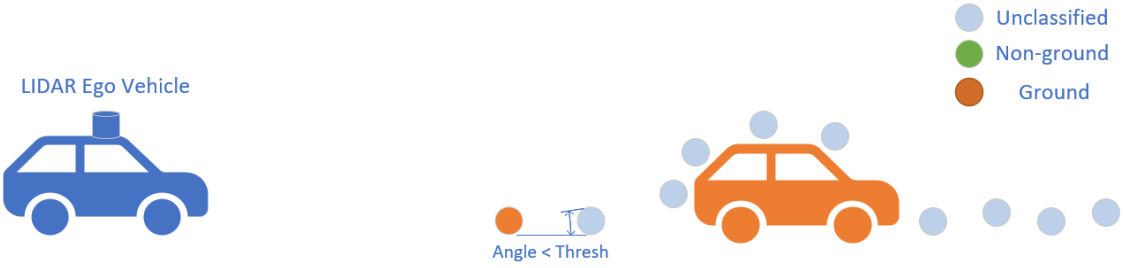

Figure 27: Standard ray ground filtering step 1 own portrayal after [34]

To start the algorithm, it needs to know where the ground is located with respect to the sensor on top of the ego-vehicle. Afterwards, the point sensed closest to the ego-vehicle and closest to the ground gets

labeled as ground point. Then it scans through all points in a ray with increasing distance and measuring the angle between the last ground point and the next point in the point cloud. If the next point is within a certain angle threshold it is considered to be a ground point. [34]

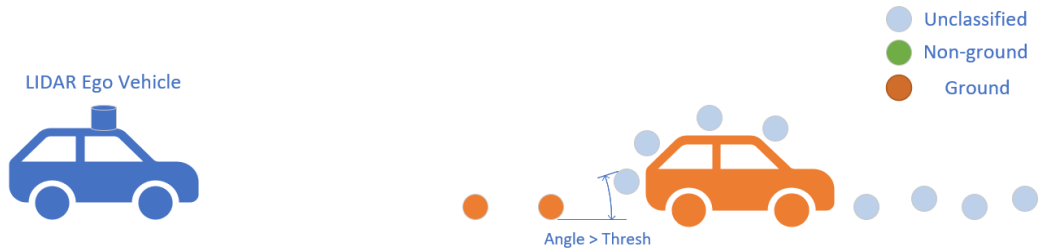

Figure 28: Standard ray ground filtering step 2 own portrayal after [34]

As can be seen in Figure 28 the angle to the next point from the last labeled orange ground point is larger than the threshold and therefore all points afterwards are considered as non-ground, this is shown in the next figure. [34]

Figure 29: Standard ray ground filtering step 3 own portrayal after [34]

As can be seen in Figure 29 all points after the threshold violation got labeled as non-ground. Autoware.Auto uses the algorithm from Autoware.AI with some key improvements like threshold statistics, domain knowledge and reinterpretation as a factor graph. Therefore, the algorithm from Autoware.AI is explained below. [34]

3.1.6 Ray Ground Filtering Autoware.AI Algorithm

The difference between the standard approach and the approach from Autoware.AI is that it has two cones, one local cone and a global cone. How the algorithm works is described below. [34]

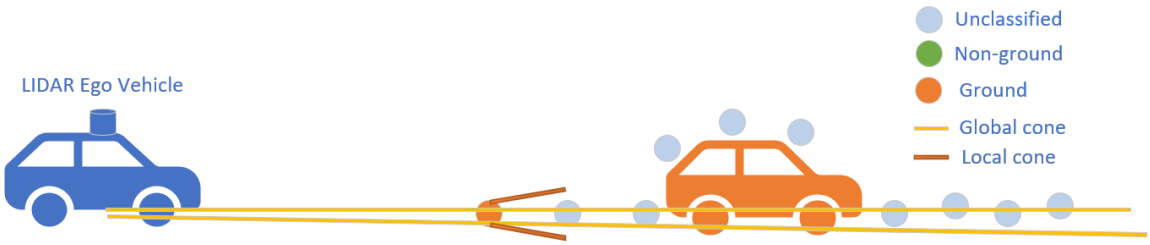

Figure 30: Autoware.AI ray ground filtering own portrayal after [34]

As can be seen above there are two cones a local cone starting from a point in the point cloud and a global cone starting from the ego-vehicle. The global cone is close to the actual ground based on the LIDAR height position. Once the first ground point is labeled, the local cone comes into the play. If the next LIDAR point is within the local cone, it gets automatically labeled like the parent point. In the case above it gets labeled as ground, since the first point is also ground. [34]

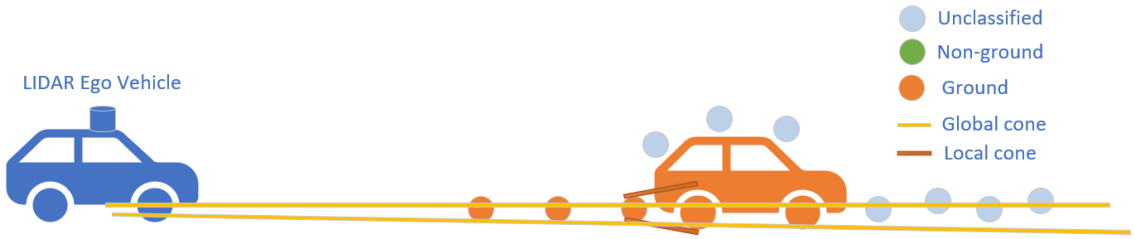

Figure 31: Autoware.AI ray ground filtering own portrayal after [34]

The first three points are all within the global cone and the local cone and are therefore labeled as ground.

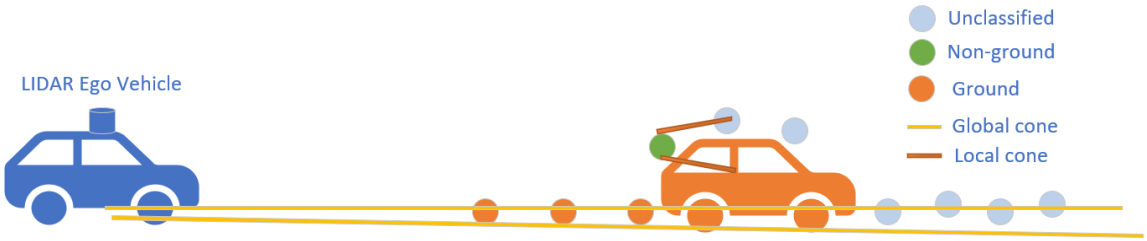

Figure 32: Autoware.AI ray ground filtering own portrayal after [34]

The fourth point is not within the local cone of the third point which is visible in Figure 31, and is therefore non-ground. All points on the roof of the car in front are within the local cone of the previous point and are therefore labeled as non-ground points. [34]

Figure 33: Autoware.AI ray ground filtering own portrayal after [34]

The next difference between the standard algorithm and the algorithm used in Autoware.AI is that it does not set all points after the first labeled non-ground automatically as non-ground. It always checks for both of the cones. As can be seen in Figure 33 the points after the car are labeled as ground, since

the first point directly after the car, does not fit within the local cone of the last point on the roof. All of them afterwards are labeled as ground. [34]

3.1.7 Ray Ground Filtering Autoware.Auto

Autoware.Auto uses the algorithm from Autoware.AI but with some minor adjustments. One problem of the original algorithm is that the lager the distance is the larger the global cone gets, and therefore also the ground gets.

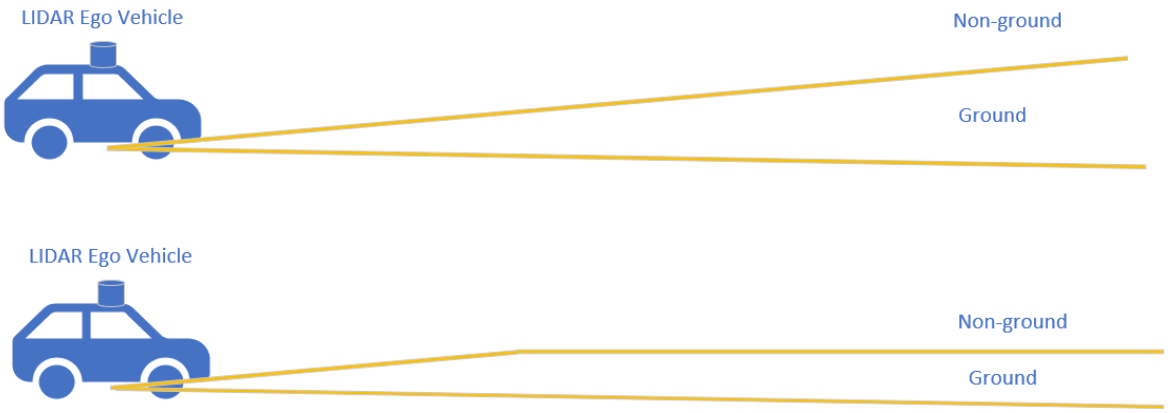

Figure 34: Autoware.Auto cone threshold own portrayal after [35]

As can be seen in Figure 34 the global cone gets bigger and bigger the further away you get from the ego-vehicle. To prevent the cone from always getting bigger a threshold is used, it improves the detection of ground and nonground points in a distance. Additionally, they added more domain knowledge by adding more labels to the ground filtering algorithm. This helps to increase the accuracy by distinguishing more than just instant ground or non-ground. Provisional labels get assigned in the algorithm and then the provisional labels get reviewed with the neighbors and additional information, therefore the result is more precise than the one from Autoware.AI. [33]

3.1.8 Euclidean Clustering

A key element for automated driving is to detect objects on the road, therefore the LIDAR points need to be converted into objects. Once the ground points are separated from the non-ground points, the object detection can be started. The euclidean cluster algorithm takes the non-ground points and generates objects out of it. The algorithm works as follows and comes from the following source [36].

1. Creating a kd-tree representation for the input point cloud dataset P

2. Setup of an empty list of clusters C, and a queue of the points that need to be checked Q

3. For every point $p_i \in P$, perform the following steps:

- Add p_i to the current queue Q
- For every point $p_i \in Q$ do:
    - search for the set P_i^k of point neighbors of p_i in a sphere with radius $r < d_{th}$
    - for every neighbor $p_i^k \in P_i^k$, check if the point has already been processed, and if not add it to Q
- When the list of all points in Q has been processed, add Q to the list of clusters C, and reset Q to an empty list

4. The algorithm terminates when all points $p_i \in P_i^k$ have been processed and are now part of the list of point clusters C

This algorithm is illustrated in a visual way below.

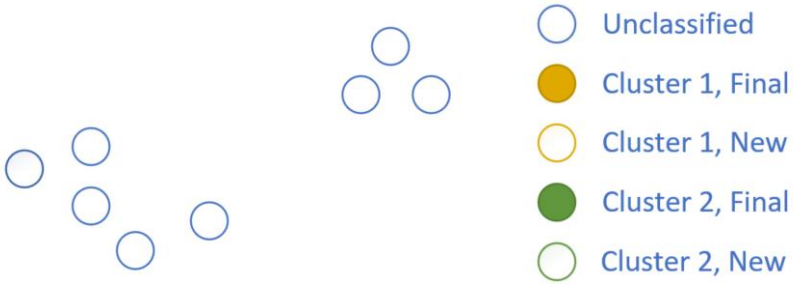

Figure 35: Step 1 euclidean clustering empty cluster own portrayal after [33]

As can be seen in in Figure 35 the algorithm starts with an empty cluster. Then a random point gets added to the first cluster. In this case the point all the way to the left marked with the color (Cluster 1, New).

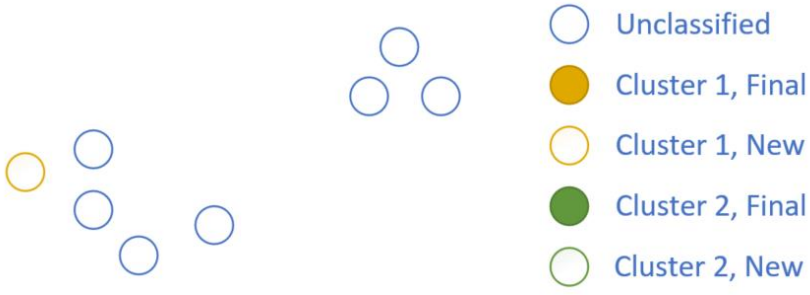

Figure 36: Step 2 euclidean clustering, adding a point to a cluster own portrayal after [33]

Then the neighbors of this point within a certain radius need to be found. Once all neighbors of a point are found and labeled as (Cluster 1, New) the initial point gets added to (Cluster 1, Final) visible in the next figure.

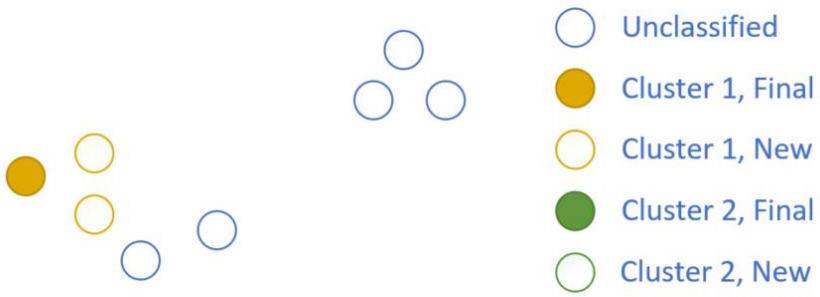

Figure 37: Step 2 euclidean clustering find neighbors own portrayal after [33]

Then the algorithm jumps to the next point which is labeled as (Cluster 1, New) and searches for its neighbors. This goes on as long there are neighbors within a certain radius.

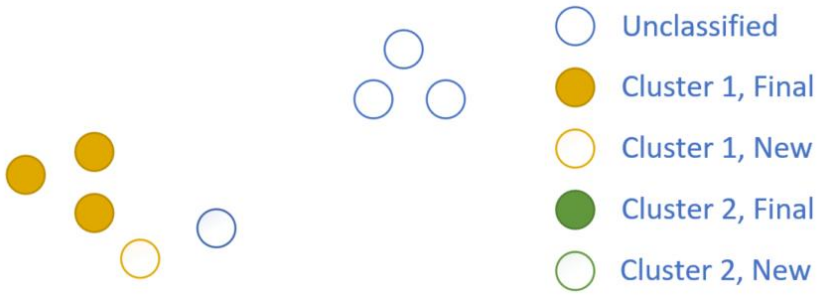

Figure 38: Step 2 and Step 3 euclidean clustering repeating own portrayal after [33]

Two more points in the first cluster were labeled with (Cluster 1, Final).

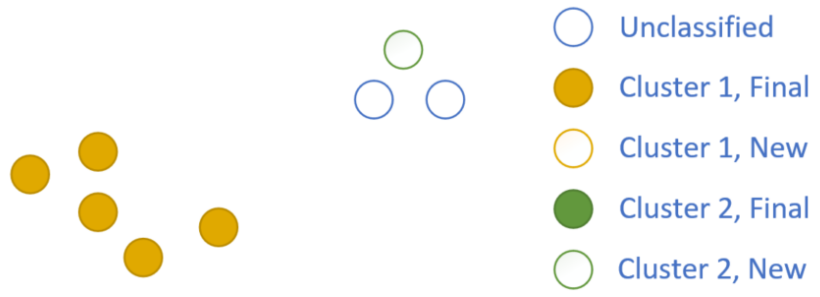

Figure 39: Euclidean clustering, cluster 1 finished, cluster 2 started own portrayal after [33]

Once the cluster has no new points in the neighborhood the cluster is finished. Then the next random point in the point cloud which was not processed yet is added to the (Cluster 2, New) and the algorithm starts over. This continues until all points within the point cloud have been processed.

3.1.9 Shape Extraction

Once the point cloud is clustered the objects appear as point blobs. The problem with that blob is that it is computationally intensive to check blobs for collision, they are unbounded and cause memory overhead. Object detection in general informs about possible collisions. The final object representation should be as follows. [33]

- Easy to collision check
- Easy to compute
- And representing the actual object reasonably

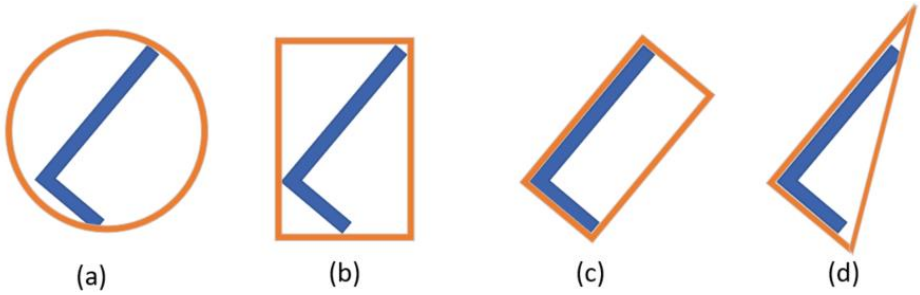

Figure 40: Different bounding boxes own portrayal after [37]

Visible in Figure 40 are four different kinds of bounding boxes. (a) sphere, (b) axis-aligned bounding box (AABB), (c) oriented bounding box (OBB), and (d) convex hull. The oriented bounding boxes are a good compromise when paying attention for easy collision checking and are easy to compute while always representing the actual object. [33]

3.1.10 Bounding Boxes in Autoware.Auto

It uses an efficient L-Shape fitting algorithm whereby it computes the principal components of the blob and sorts it along the computed axis so that multiple LIDARs can be used and that it appears that the blob comes from one LIDAR. [38] [33]

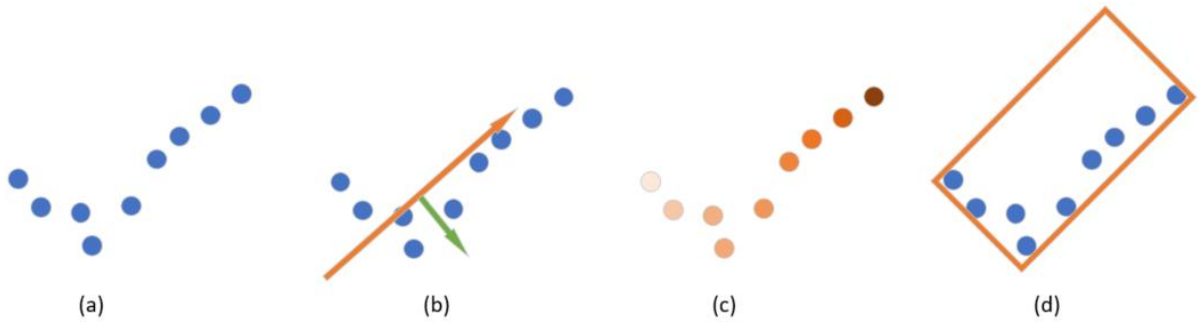

Figure 41: Autoware.Auto bounding box generation own portrayal after [33]

The points captured by the LIDAR are visible in (a), in (b) a principal component analysis (PCA) is performed to find the major and minor axes of the bounding boxes. The major and minor axis of the PCA is marked with the arrows. In (c) the points were ordered in the direction of the major axis from the principal component analysis. This the sorting is done that it appears that the LIDAR points come from only one single LIDAR. If only one LIDAR is used this step can be neglected. And in (d) the bounding box is placed in the direction from the major and minor axis that it fits the points. [33] [38]

3.2 LIDAR Object Detection with Autoware.Auto and CARLA

To generate objects from LIDAR points in Autoware.Auto the following nodes/launch files represented as blocks in the next figure are launched. The connection arrows in between are the published topics.

Figure 42: Autoware.Auto LIDAR object detection block diagram

First the topic /points_raw is sent into the ray_ground_classifier which separates the ground points from the non-ground points. Then the /points_no_ground are sent to the euclidean_cluster_nodes block which clusters them and automatically generates a bounding box around them. All of this is shown in the next figures.

Figure 43: CARLA ego-vehicle

The Figure 43 shows the ego-vehicle in CARLA with a non-visible LIDAR sensor on top. It is scanning the environment around it. In front of the ego-vehicle on the road is a car on the left and a motorcycle on the right side.

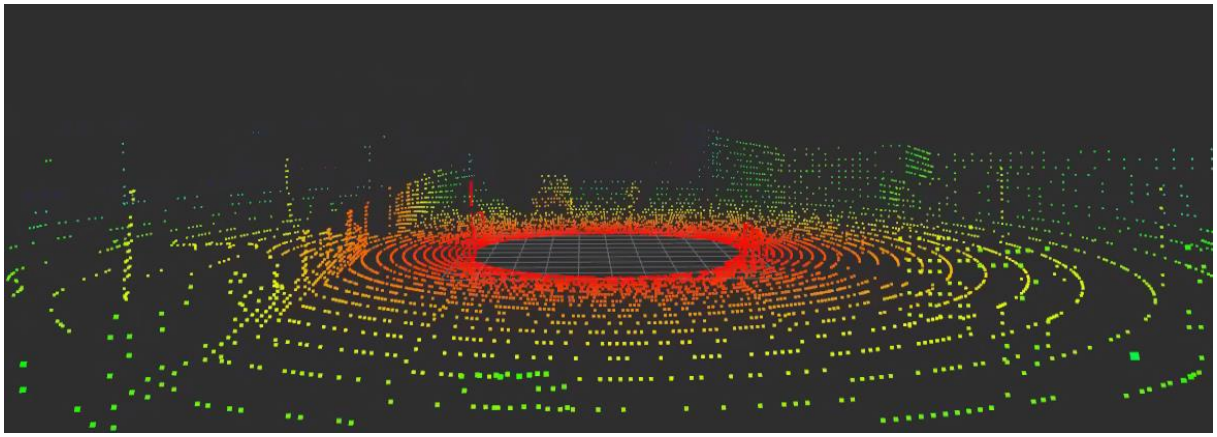

Figure 44: RVIZ LIDAR point cloud raw points

Figure 44 shows the topic /points_raw which is the output from the LIDAR. This topic is the input for the object detection stack. This is unfiltered raw data of the LIDAR. The car and the motorcycle are barely visible in the point cloud.

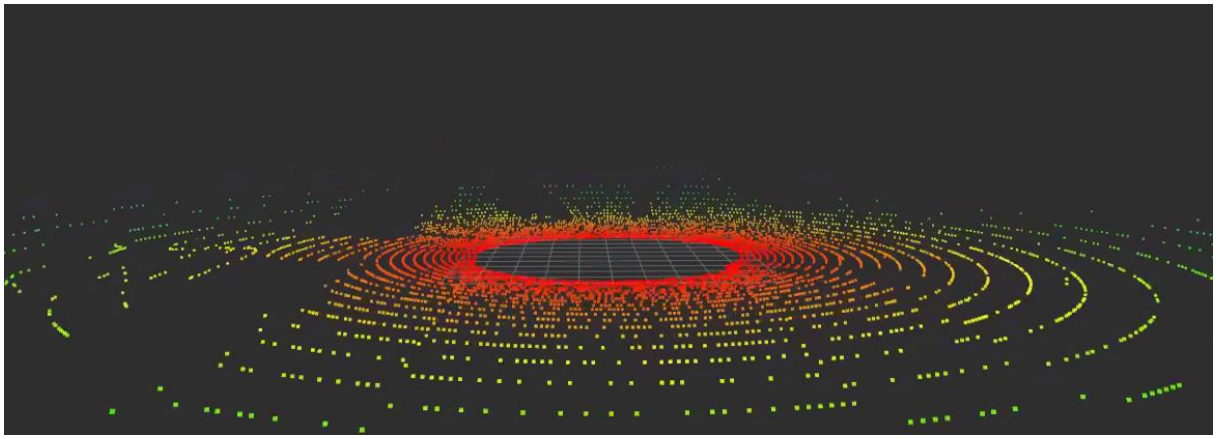

Figure 45: RVIZ LIDAR point cloud ground points

After the ray ground filter, the ground points got separated from the non-ground points visible in the image above. Important for the ray ground filter node is the actual sensor height. It took a while to set the right height that all non-ground objects get detected correctly. The quality of object detection in this case depends on good ground filtering. The quality of object detection will benefit if the ground filtering is done correctly and in a qualitative way.

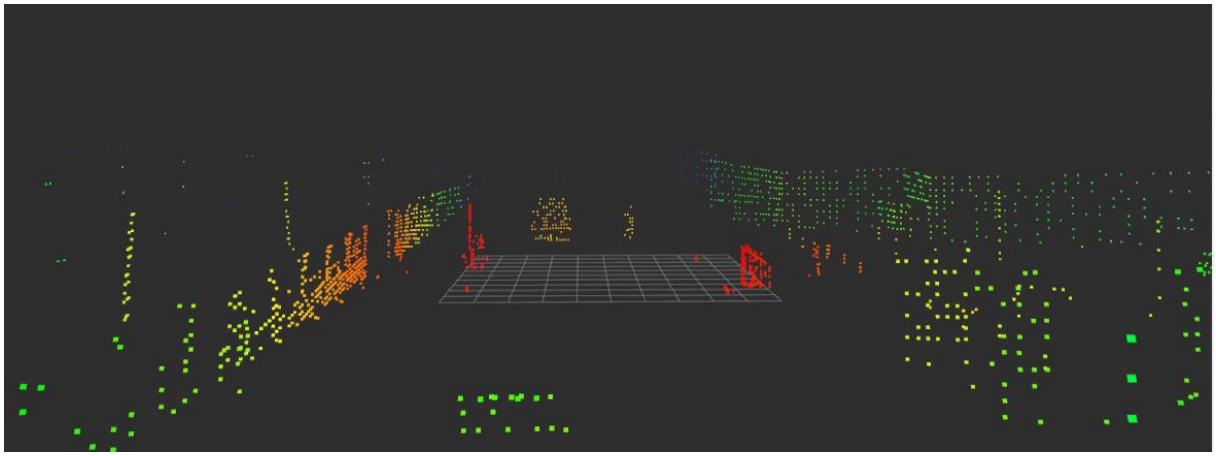

Figure 46: LIDAR point cloud non-ground points

Above all the non-ground points detected are visible. The car on the left and the motorcycle on the right are now clearly visible.

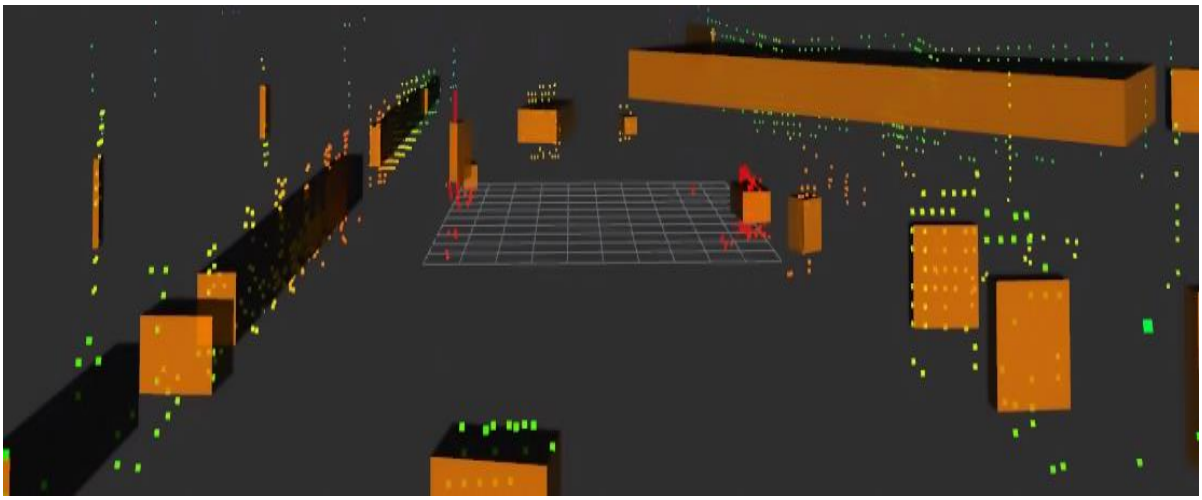

Figure 47: Euclidean clustering and bounding boxes + non-ground points

After euclidean clustering and bounding box fitting the result is visible in the figure above. Whereby the car and the motorcycle in front of the ego-vehicle got detected accurately. As can be observed in the image not all bounding boxes fit correctly. Not all points of the point cloud or clusters got totally wrapped by the bounding box. Some points float above, below or around the bounding box.

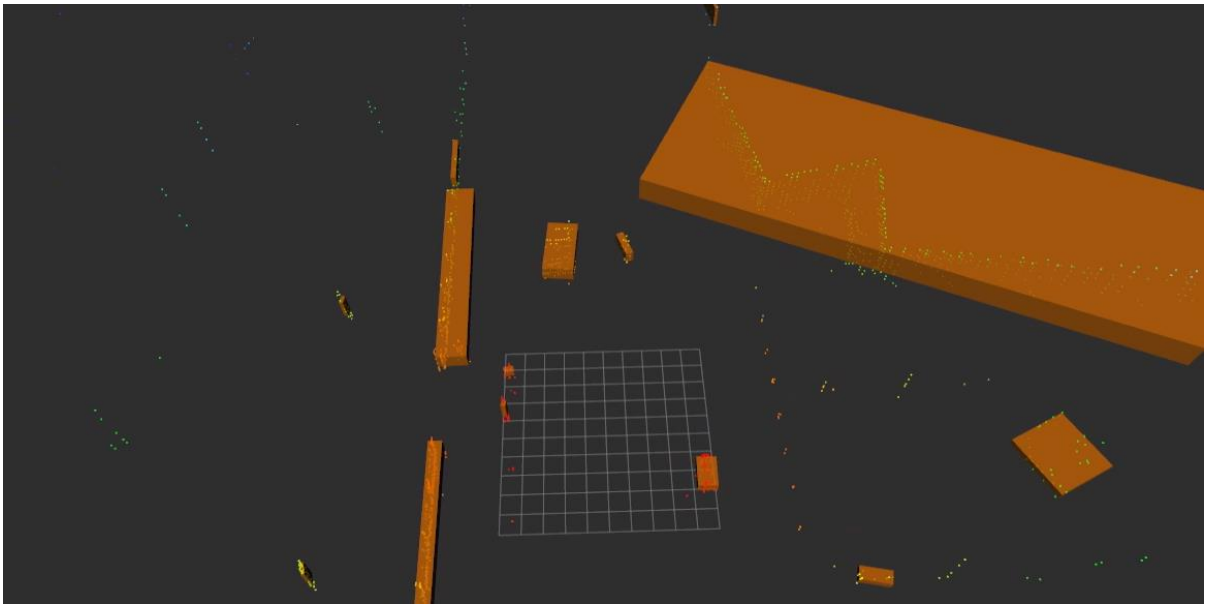

Figure 48: Bounding boxes error

The alignment of the bounding boxes does not always correlate with the actual object pose sensed by the LIDAR, as visible in the figure above. The car on the left and the motorcycle on the right have both a slight rotation. The next problem is the detected house from the CARLA simulation in the right upper corner in the figure above. The created bounding box is protruding into the road. This can cause a serious problem when driving autonomously. Since this is then perceived as an object, the ego-vehicle would either crash into the bounding box virtually or it will stop before that box. The behavior of course depends on the AD stack and how it reacts in such situations. Since the bounding boxes get computed with each frame, it might be that the bounding box changes its pose with the next frames.

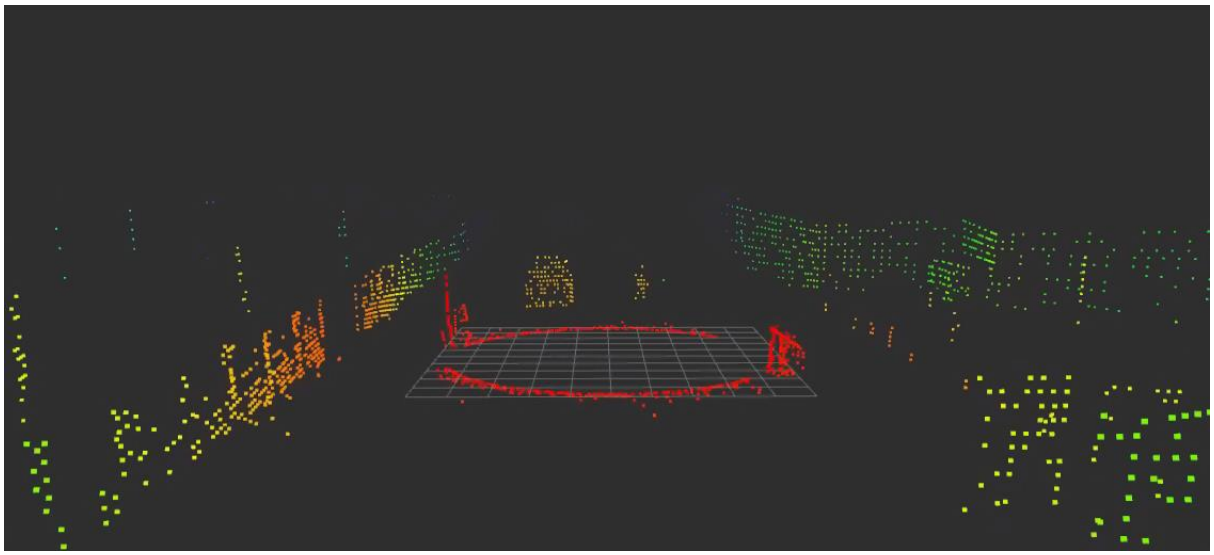

Figure 49: Wrong calibrated sensor non-ground points

Figure 49 shows the effect of a wrong calibrated sensor height. The difference to Figure 46 is that a circle of points around the ego-vehicle is labeled as non-ground points but are actually ground points. Therefore, the bounding box detection afterwards does not work correctly visible in Figure 50. The actual height of the sensor is 2.4 m and the results from Figure 49 come from a setup with a sensor in 2.1 m height. A difference in 0.3 m can lead to enormous errors for the ground filtering and therefore also for the object detection.

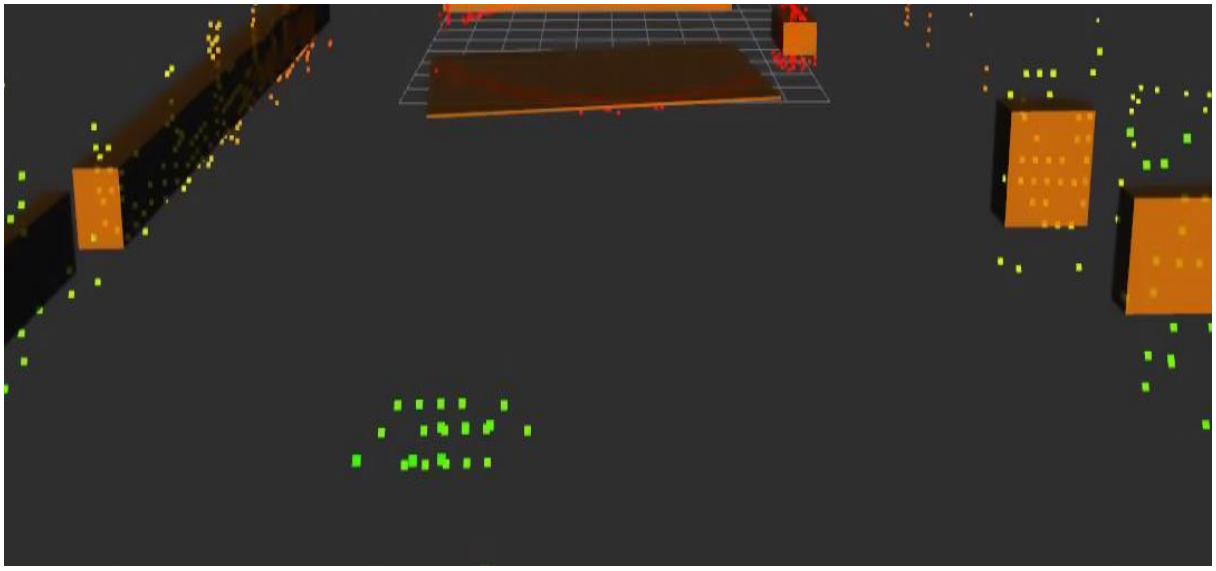

Figure 50: Euclidean clustering bounding box detection with wrong sensor height

As visible above the stack identifies objects next to the ego-vehicle in the simulation, whereby there are no objects in the CARLA simulation.

3.3 V2X Perception Layer Extension

Autoware.Auto does not come with an implemented V2X sensor, since this functionality is missing in the perception layer, an ideal V2X sensor without radio channel simulation will be developed in combination with the simulator CARLA. It is important for automated vehicles to know the position of other vehicles on the road to make sure not to crash into each other. The needed parameters for that are the following.

- Positions of the vehicles
- Orientation of the vehicles
- Dimensions of the vehicles

Therefore, the positions, dimensions and orientations of the vehicles in the simulation need to be extracted from CARLA. There are two possibilities to do so.

First

The first one would be to add a new sensor file directly to the source code of CARLA and re-building it with the Unreal Engine.

Second

The second one would make use of the existing CARLA ROS bridge. Once the bridge is started, CARLA publishes a list of objects and actors. The actors list includes all actors spawned with a unique ID and the role name of the actor. The role names for instance are ego-vehicle or autopilot. But the actors list does not include information about the position, dimension or orientation which is important for the V2X sensor. This information, including the unique ID of the objects, is stored in the objects list. Where all objects within the simulation are listed, this includes the actors. The objects list in turn does not label the objects inside with a role name. They only have an ID which does not specify if it is a vehicle or a traffic light, but it is the same ID as in the actors list. Since we need to know which objects are vehicles and which are something different the actors list needs to be searched for the actors with the role name autopilot and ego-vehicle. Then the unique ID from the actors with the label ego-vehicle or autopilot needs to be stored to extract the information about the position from the objects list.

The following implementation uses the second variant of implementation explained above and has information directly from the gaming engine, no radio channel is simulated, all vehicles in the simulation will be received by the V2X sensor. The algorithm is shown below and was realized with python and the python API from ROS2:

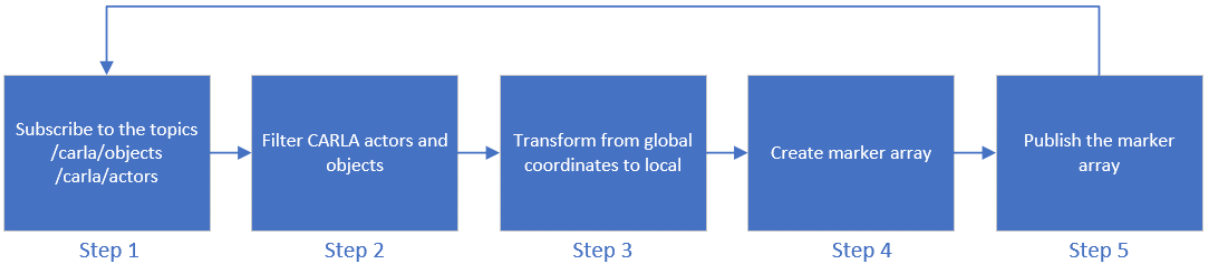

Figure 51: Software steps for V2X sensor

The block diagram above visualizes the software steps for the V2X sensor. Whereby all steps will be explained in detail now.

3.3.2 Step 1 Subscribing to Topics

First subscribing to the objects list and actors list with:

```
self.subscription = self.create_subscription(
    CarlaActorList,
    '/carla/actor_list',
    self.listener_callback,
    10)
self.subscription1 = self.create_subscription(
    CarlaActorList,
    '/carla/objects',
    self.listener_callback,
    10)
```

Now the objects and actors are available with each frame and can be used by the software.

3.3.3 Step 2 Filtering Data

In step two the data needs to be filtered. The objects list delivers all necessary information for the V2X sensor but not the labeling information if it is a car or some other object in the simulated world. The actor list on the other hand has the information what actor is a vehicle and what not. Therefore, the actors list is searched, and the ID, which is the same in the objects list and the actors list is then stored. This ID can then be used to find the right objects from the objects list.

To store the vehicle IDs a list is created which gets refreshed each frame:

```
v2xIDList = []
```

Then the list will be searched for the ID of with the label ego-vehicle or autopilot since these are important for the V2X sensor:

```
def listener_callback(self, msg):
    self.v2xIDList.clear()
    for i in range(0, len(msg.actors)):
        if(msg.actors[i].rolename == "ego_vehicle"):
            self.egoVehicleId = msg.actors[i].id
        if (msg.actors[i].rolename == "ego_vehicle"  or msg.actors[i].rolename
            == "autopilot"):
            self.v2xIDList.append(msg.actors[i].id)
```

Now all IDs are stored in the ID list.

3.3.4 Step 3 Coordinates Transform

CARLA publishes all information about the actors within the simulation in a global coordinate system, but the ego-vehicle and the AD stack need this position relative to its own position. All sensors from

CARLA like LIDAR, camera and radar are published in a relative position to the ego-vehicle. Therefore, the positions need to be transformed to a local coordinate system with respect to the ego-vehicle.

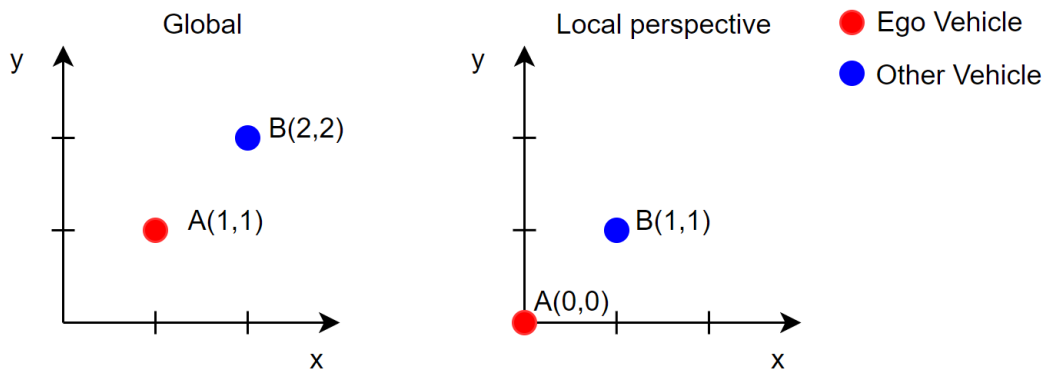

Figure 52: Global CARLA coordinates and local ego-vehicle coordinates

The coordinates transformation is simplified with only two dimensions in Figure 52. On the left side are the global coordinates system from CARLA and on the right side are the transformed coordinates from the perspective of the ego-vehicle. The transformation is calculated as follows:

$$x' = x - x_{ego}$$
$$y' = y - y_{ego} \quad (3.1)$$

Then we get the new local positions with respect to the ego-vehicle as follows:

$$1 = 2 - 1$$
$$1 = 2 - 1 \quad (3.2)$$

Below is the CARLA simulator with cars in the surrounding, to see what this transformation has done the result is visualized with RVIZ.

Figure 53: CARLA ego-vehicle

Visible in Figure 53 is the CARLA ego-vehicle with two other vehicles on the left side of it.

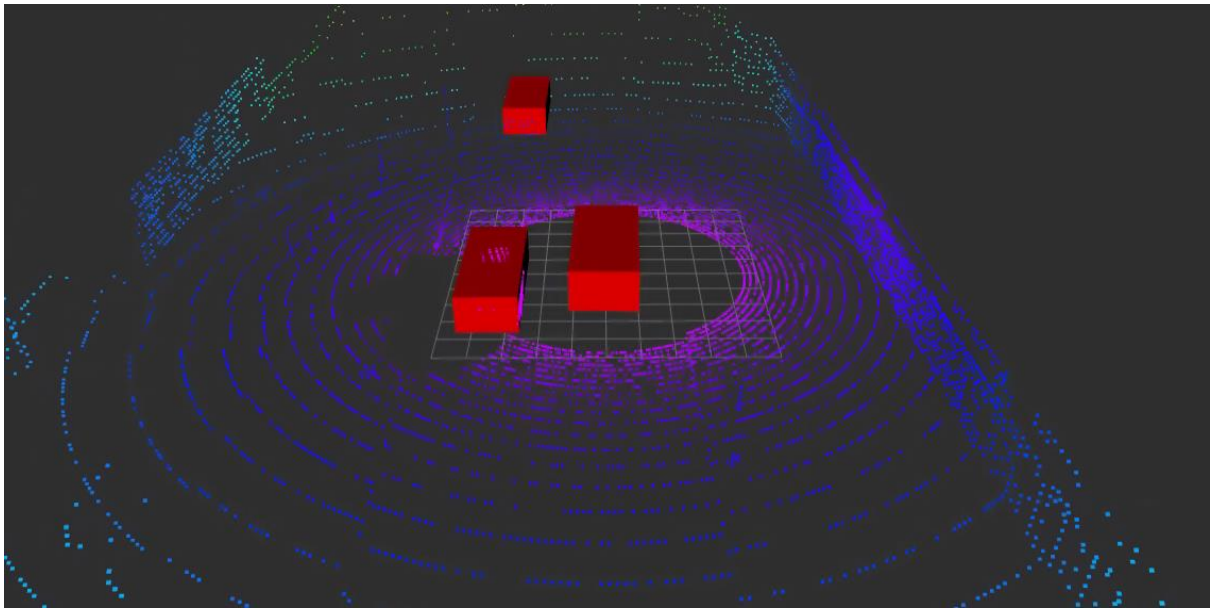

Figure 54: V2X and LIDAR data

As the figure above shows the point conversion from the global coordinates to the local coordinates of the perspective of the ego-vehicle did work and everything looks right. But if you look at it more closely and start to move with the ego-vehicle you will recognize two errors. The first one is the rotation of the vehicles and the second one is the position of the vehicles if the ego-vehicle changes its rotation. For the next image the ego-vehicle will turn approximately 45 degrees to the right.

Figure 55: Ego-vehicle rotated approximately 45 degrees to the right

As can be seen in Figure 55 the ego-vehicle has turned about 45 degrees to the right. In the image below we see how this rotation affects the markers.

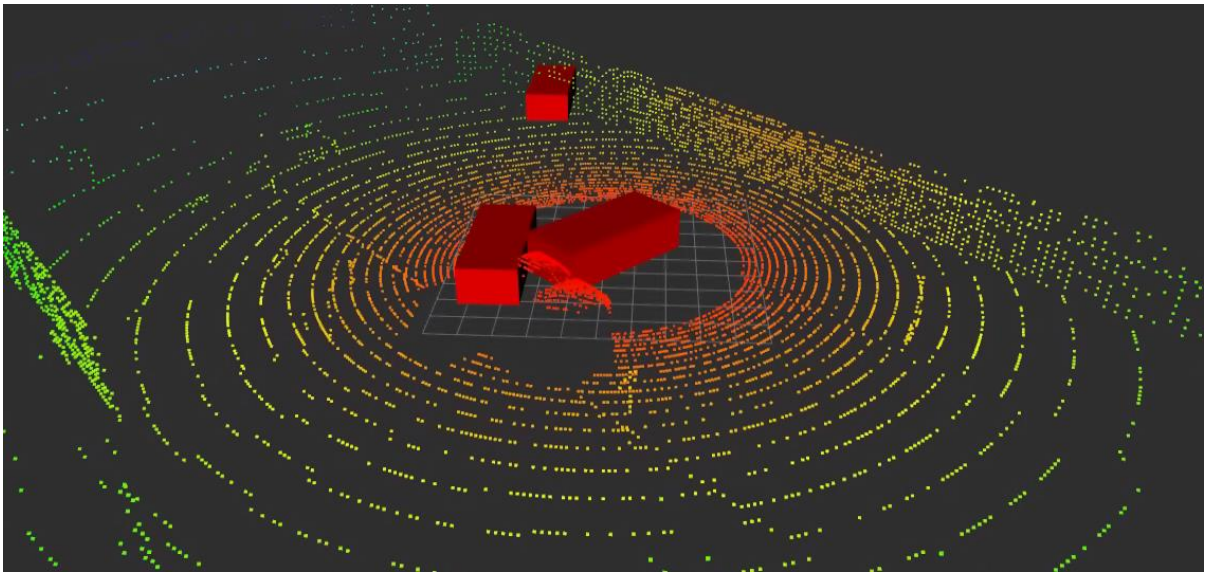

Figure 56: Markers ego-vehicle turned to the right by approximately 45 degree

The ego-vehicle block should always be aligned with the y axis of the grid underneath. In the image above there are two errors visible, the first one is that the ego-vehicle has rotated and is not aligned with the grid below. The second one is that the other vehicles in the surrounding have moved and do not correlate with the captured LIDAR points. The vehicle in front of the ego-vehicle is crashing virtually into the wall detected by the LIDAR. The second problem is explained and solved below:

The local coordinates system is always aligned with the ego-vehicle and therefore if the ego-vehicle turns to the right also the points of the other vehicles will turn to the right. The other vehicles get always transformed from the global coordinates system to the local with respect to the ego-vehicle without considering the rotation of the ego-vehicle. Therefore, the cars need to change their x and y position relative to the ego-vehicles rotation.

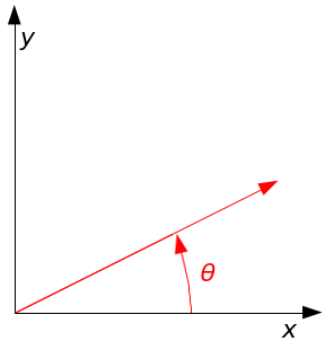

Figure 57: Ego-vehicle perspective for other cars [39]

All the other cars can also be represented as vectors in an coordinates system where the tip of the vector is the actual position. These vectors need to be moved by the angle of the rotation of the ego-vehicle that the actual representation in RVIS correlates with the 3D representation in CARLA. To rotate a vector the rotation matrix is used:

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \tag{3.3}$$

The matrix above realizes a rotation of a vector with the angle θ of the ego-vehicle

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} \tag{3.4}$$

And therefore, the new coordinates are:

$$\begin{aligned} x' &= x\cos(\theta) - y\sin(\theta) \\ y' &= x\sin(\theta) + y\cos(\theta) \end{aligned} \tag{3.5}$$

With the formulas above one can calculate the new positions of the surrounding vehicles with respect to the rotation of the ego-vehicle.

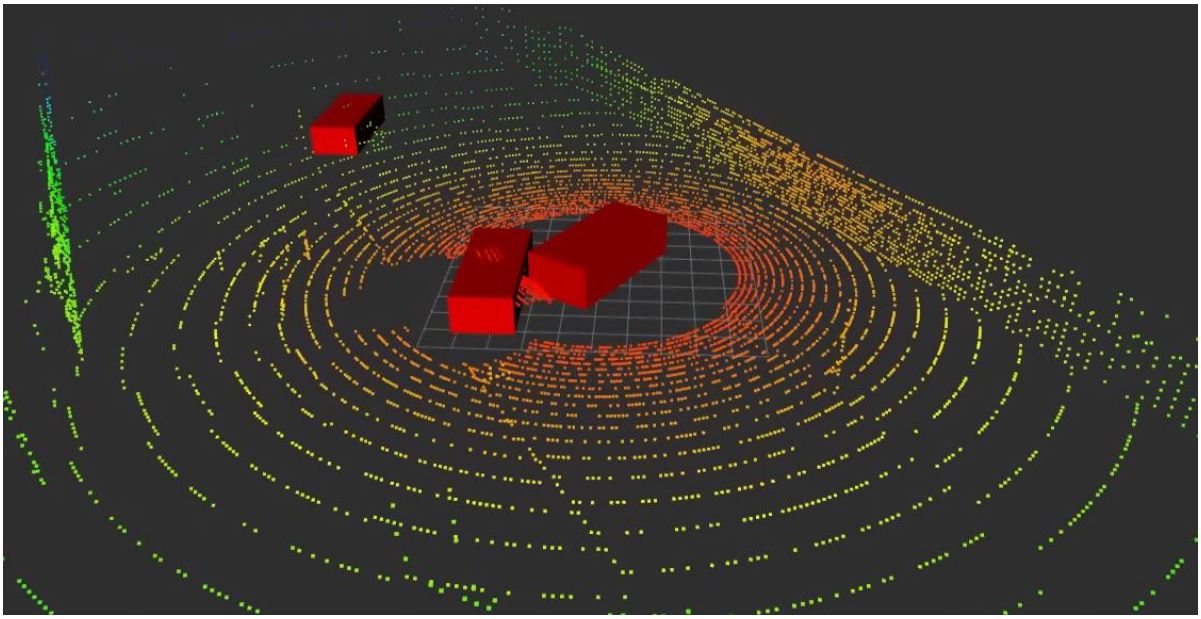

Figure 58: Corrected positions with respect to the ego-vehicle rotation

Now the positions of the vehicles in CARLA and the positions of the vehicles captured by the LIDAR correlate. But the rotation of the objects still is not right. Therefore, the rotations of the objects also need to be converted with respect to the rotation of the ego-vehicle. And the ego-vehicle needs to be the ground truth of all rotations. The first step is to set the rotation of the ego-vehicle to zero so that it aligns always with the y axis.

Figure 59: CARLA ego-vehicle

Now the ego-vehicle is slightly turned right this should be visible in RVIZ.

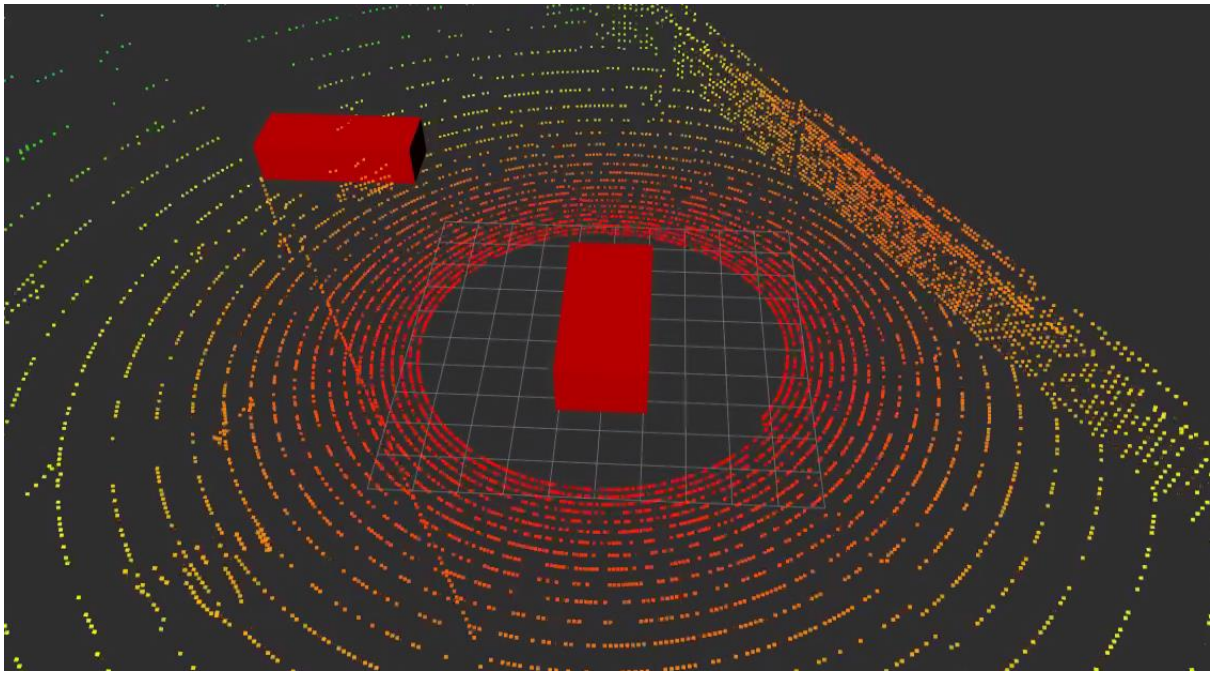

Figure 60: Ego-vehicle set to ground truth

Now the ego-vehicle in Figure 60 is aligned with the y-axis of the RVIZ grid and therefore right. But the other vehicle in the upper left corner is not rotated the right way. The problem is that the orientation of the objects is still in global perspective, therefore. The whole rotation of all the cars needs to be with respect to the ego-vehicle. Now the ego-vehicle is rotated to the left in CARLA.

Figure 61: Ego-vehicle

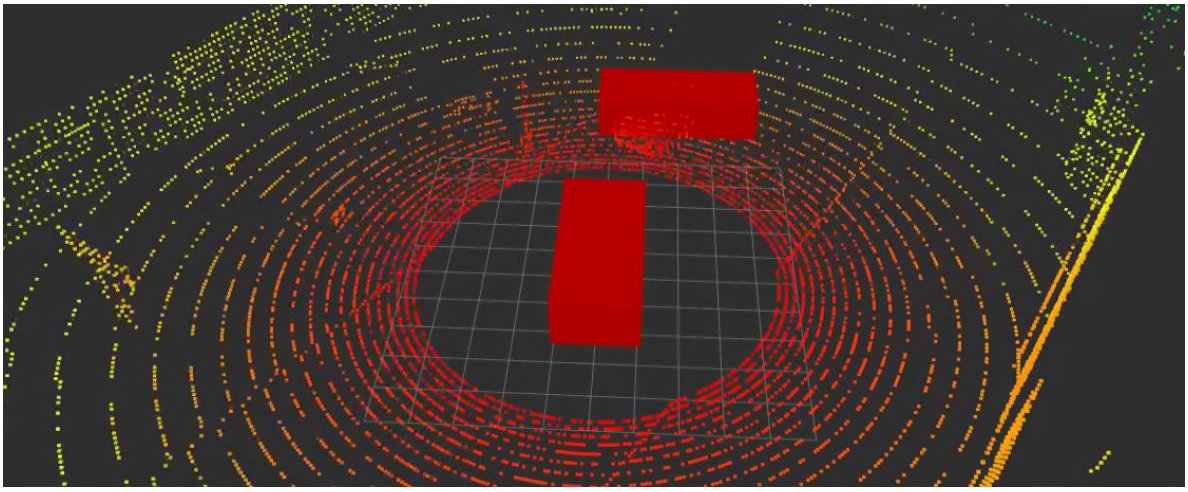

Figure 62: Ego-vehicle ground truth and another car rotation

In Figure 62 it is visible that the ego-vehicle has been rotated against the clock and the other car has rotated in that direction too, in real life this would not look like this in the view from the ego-vehicle. Therefore, the orientation needs to be transformed in the local perspective from the ego-vehicle. The orientation of the objects in CARLA and RVIZ is defined by the quaternion orientation.

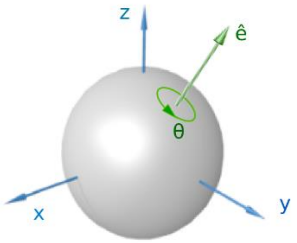

Figure 63: Quaternion rotation around unit vector $\vec{e}$ with an angle of θ [40]

The quaternions represent a rotation in 3D which can be represented with Euler angles. The quaternion is given by the following formula and represents a rotation with Euclidean coordinates: [41]

$$Q = [q_w \;\; q_x \;\; q_y \;\; q_z] \tag{3.6}$$

Around the vector:

$$\vec{e} = [q_x \;\; q_y \;\; q_z] \tag{3.7}$$

Whereby:

$$Q = \left[\cos\frac{\theta}{2} \quad \sin\frac{\theta}{2}e_x \quad \sin\frac{\theta}{2}e_y \quad \sin\frac{\theta}{2}e_z\right] \tag{3.8}$$

With the condition:

$$1 = q_w^2 + q_x^2 + q_y^2 + q_z^2 \tag{3.9}$$

From this follows the rotation around the vector $\vec{e}$:

$$\theta = 2\cos^{-1} q_w \tag{3.10}$$

In this scenario only a rotation around the z-axis is needed therefore the Euclidean coordinates q_x and q_y will always be zero. CARLA publishes the orientation of the ego-vehicle and all other vehicles in Euclidean coordinates, and it is rather complex to transform the orientation of the other cars with respect of the perspective of the ego-vehicle. For this reason, the Euclidean scalar for rotation q_w from the ego-

vehicle and the other vehicles will be converted to the Euler angle θ and then deducted. And afterwards it will be transformed back into the quaternion representation.

$$\theta_{ego} = 2\cos^{-1} q_{w_{ego}}$$
$$\theta_{car} = 2\cos^{-1} q_{w_{car}} \tag{3.11}$$

Then the angle of the car in the surrounding of the ego-vehicle gets deducted from the angle of the ego-vehicle:

$$\theta_{diff} = \theta_{car} - \theta_{ego} \tag{3.12}$$

This difference angle now needs to be transformed back into quaternion representation. The equation (3.10) leads to the following transformation back:

$$q_{w_{rotated}} = \cos\left(\frac{\theta}{2}\right) \tag{3.13}$$

And since the condition in equation (3.9) needs to be true the new $q_{z_{rotated}}$ is also computed:

$$q_{z_{rotated}} = \sqrt{1 - q_w^2} \tag{3.14}$$

In this conversion q_x and q_y can be neglected since the cars only rotate around the z-axis in this case.

Figure 64: CARLA ego-vehicle with other vehicles

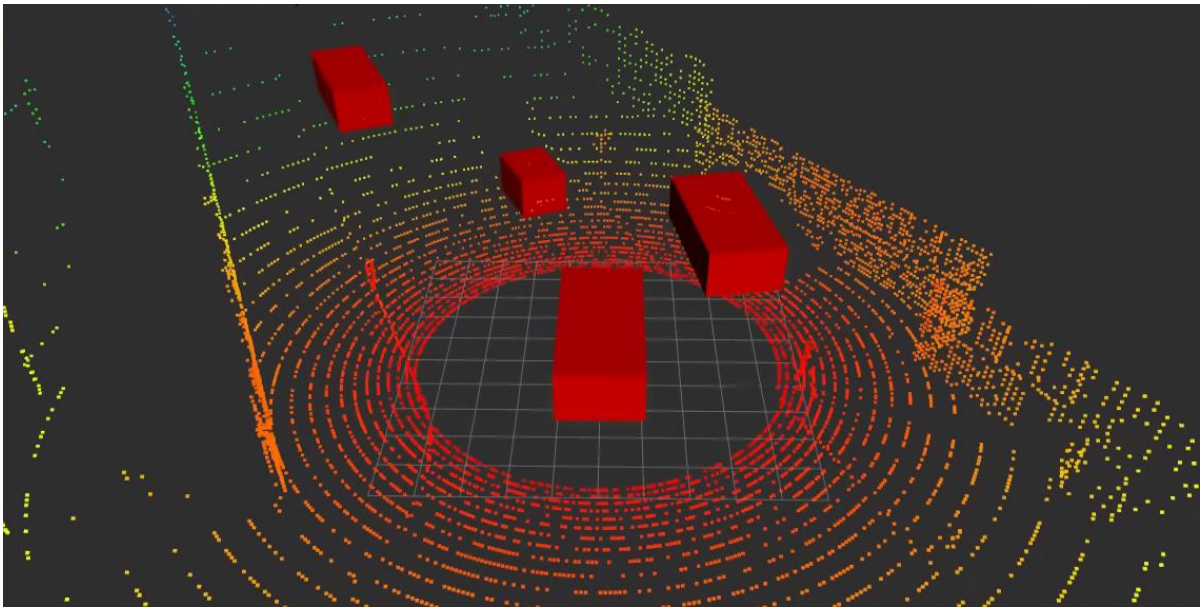

Figure 65: RVIZ result after transformation

Now all vehicles have the right position and the right rotation with respect to the ego-vehicle.

3.3.5 Step 4 and Step 5 Publishing Data

RVIZ allows to visualize different shapes by publishing them onto ROS2. For the vehicles boxes are chosen, since a box represents a car the best way for a non-sophisticated shape. The following code registers a publisher for the marker array with all the boxes which represent cars in RVIZ.

```
super().__init__('minimal_publisher')
self.publisher_ = self.create_publisher(MarkerArray, 'RVIZ_MARKER_ARRAY', 10)
timer_period = 0.5  # seconds
self.timer = self.create_timer(timer_period, self.timer_callback)
self.i = 0
```

Then the marker array will be created with each frame and published onto ROS2. RVIZ can then visualize the marker array. The source code for this whole section is listed in the appendix.

3.3.6 Comparison

Now the LIDAR detection and the V2X detection is compared in different environment situations within CARLA. First a situation where the car stops at a traffic light where cars are behind a fence on the right.

Figure 66: CARLA simulation with two cars behind a fence

As can be seen in the figure above the ego-vehicle is waiting at an intersection with a red traffic light and two cars are positioned behind a fence on the right side.

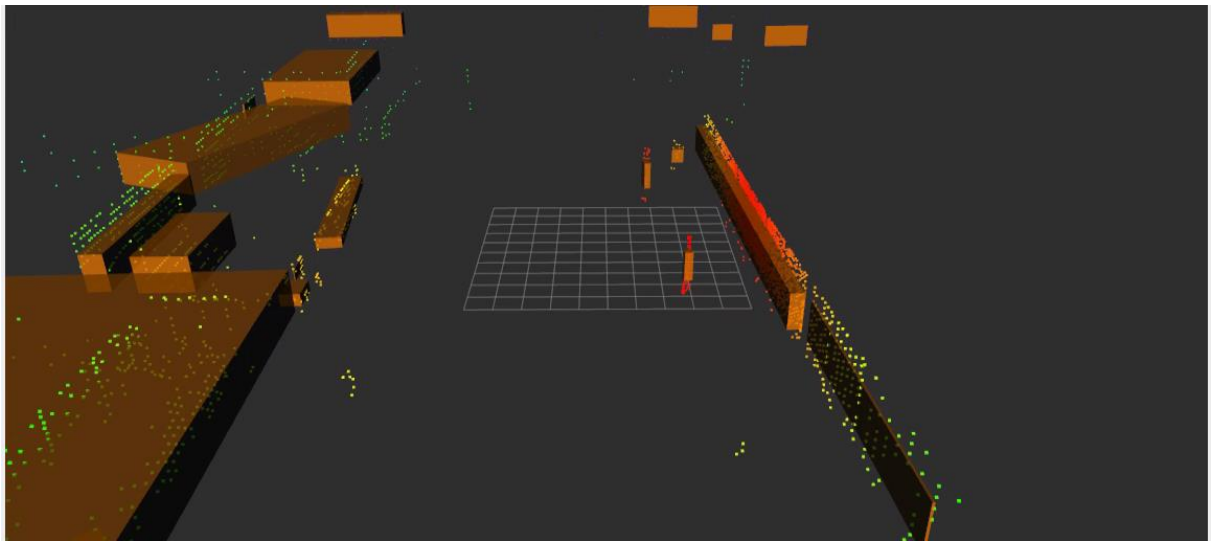

Figure 67: Autoware.Auto LIDAR object detection

Visible in Figure 67 are the detected objects (orange boxes) from the Autoware.Auto LIDAR detection. The representation of the point cloud and the bounding box is in driving direction of the ego-vehicle. The fence prevents the LIDAR from seeing the vehicles behind it, in some cases completely and in some cases partially. Whereby the V2X sensor does detect all the cars correctly behind the fence. Visible in the next picture.

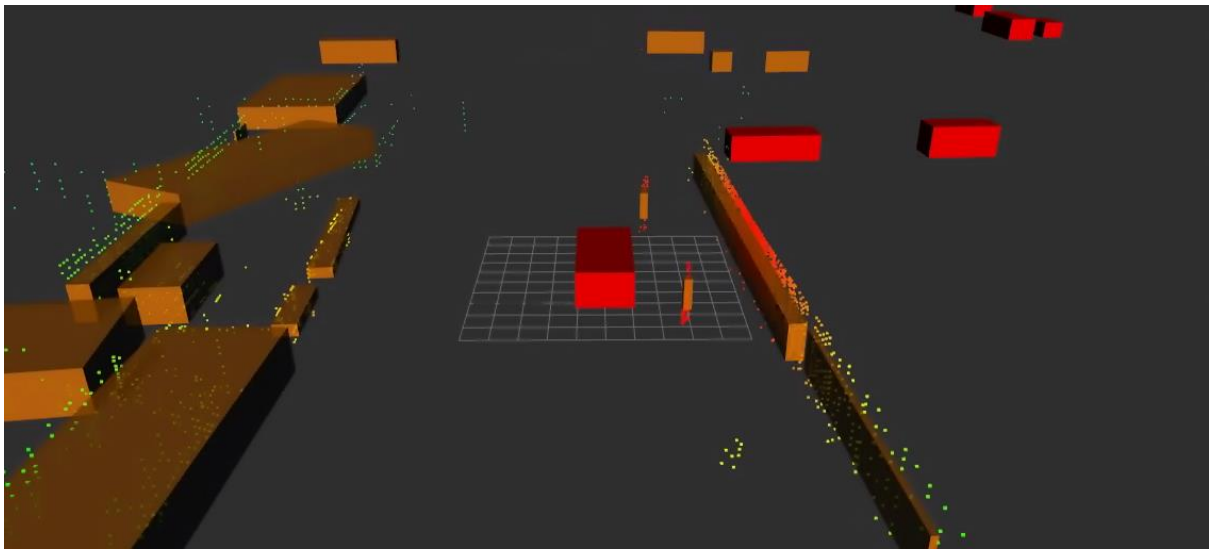

Figure 68: Autoware.Auto LIDAR detection + own ideal V2X sensor

As can be seen the vehicles behind the fence are detected by the V2X sensor (red boxes) and not by the LIDAR detection from the Autoware.Auto stack. To mention it again, the V2X implementation is ideal and receives all vehicles which are in the simulation without simulating a radio channel.

Figure 69: CARLA scene with many cars

In Figure 69 is the ego-vehicle with many cars in the surrounding, whereby the LIDAR only detects a few of it. Some are not visible either by range or the objects are not visible for the LIDAR as they are covered by other objects or vehicles.

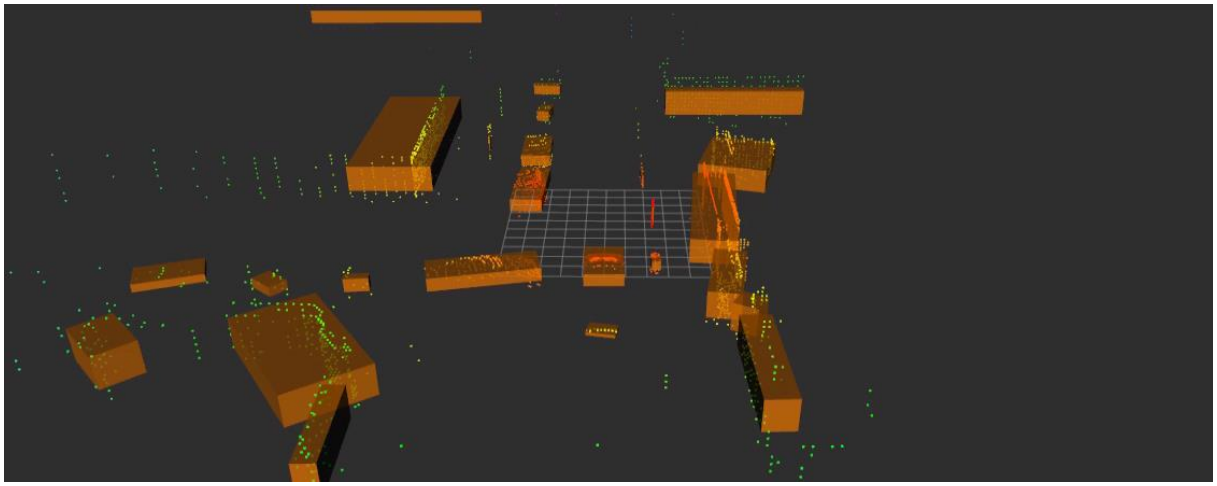

Figure 70: LIDAR detected objects

The RVIZ image above is captured in the driving direction of the ego-vehicle from CARLA. As can be seen only two objects are detected in the back of it, several on the left and several in front of it on the oncoming traffic lane. In the next picture is visible that much more vehicles are surrounding the ego-vehicle.

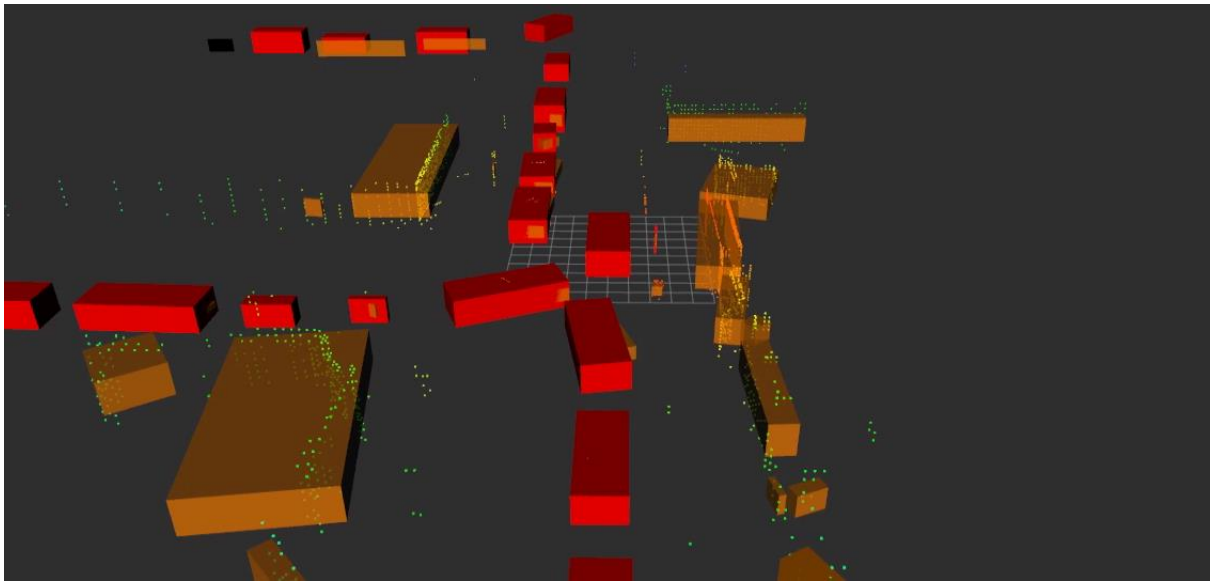

Figure 71: Autoware.Auto object detection (orange boxes) and V2X sensor (red boxes)

Visible is that the V2X sensor has received many more objects than the LIDAR sensor has detected in combination with the object detection of Autoware.Auto.

3.4 Discussion of Results

The ray ground filtering does work very well in Autoware.Auto, the bounding box detection has its limits since it does not always create the bounding boxes according to the rotation of the real objects sensed. Sometimes the bounding boxes are way bigger than the object itself, for instance house walls. If the wall is diagonal aligned to the road most of the time the bounding box created protrudes into the road whereby the actual house wall does not. The cars in the surrounding of the ego-vehicle are detected well as long as they are not shielded, and the LIDAR cannot sense them. For example, if a fence is between the ego-vehicle and the other car in simulation. The V2X implementation has its advantages here because it receives many more vehicles than detected by the LIDAR. Since the implemented V2X sensor is ideal, these findings cannot be transferred to real life, the next step would be to implement a radio channel simulation.

4 Autoware.AI Implementation

Autoware.AI is the predecessor from Autoware.Auto, it was the first all in one AD Stack available as open-source software. It was the first stack developed from the Autoware Foundation and is therefore sometime referred to as the Autoware. At this chapter the full stack of Autoware.AI is explained and implemented in combination with CARLA. The whole configuration of this stack, for example the launch files, are based on the carla autoware agent from GitHub [42]. The full block diagram with all the connected nodes for the working driving stack is too large to fit a A4 page, therefore the layers are explained separately. [43]

The full stack consists of several layers which are listed below:

- Map
- Sensing
- Localization
- Detection
- Mission Planning
- Motion Planning

In addition to the layers above the function of the CARLA autoware bridge and RVIZ is explained.

4.1 CARLA Autoware Bridge

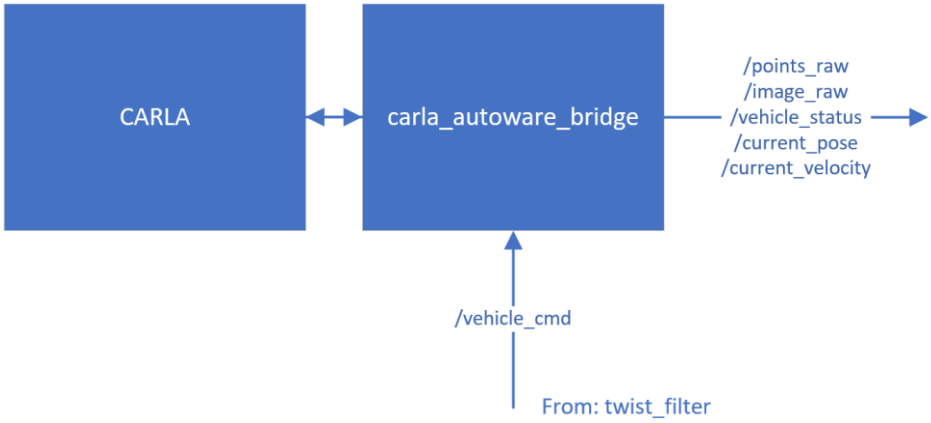

Figure 72: CARLA autoware bridge

The carla_autoware_bridge node connects CARLA to ROS and publishes important information about the ego-vehicle onto it. The topic to control the ego-vehicle is /vehicle_cmd, this topic controls the gas pedal, brake pedal and steering wheel from the ego-vehicle inside the CARLA simulation.

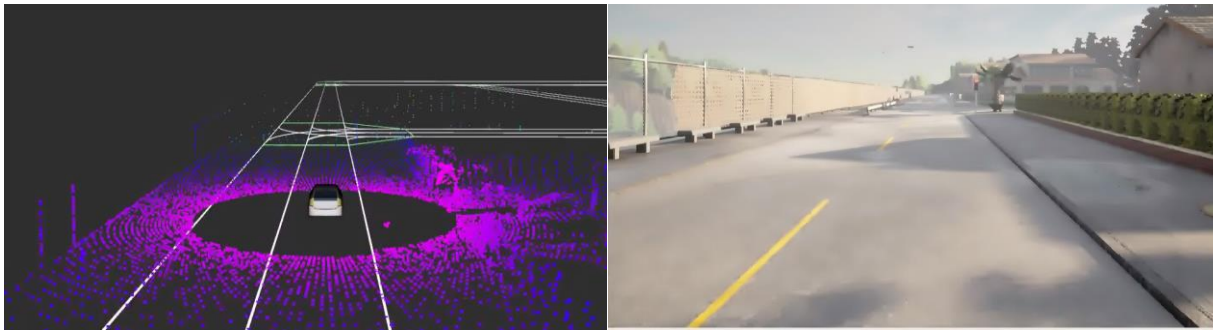

Figure 73: Topics /points_raw on the left and /image_raw on the right

On the left side the ego-vehicle is visible with the /points_raw from the LIDAR in pink/blue which are published from the carla_autoware_bridge. On the right side the image from the front camera of the ego-vehicle is visible. This image is published under /image_raw on ROS. The /vehicle_status topic includes, among other things, information about the current gear, gas or brake pedal position. The /current_pose is the pose of the ego-vehicle and the /current_velocity is the current velocity of the ego-vehicle. With /vehicle_cmd the ego-vehicle within CARLA is then controlled by the Autoware.AI stack.

4.2 RVIZ

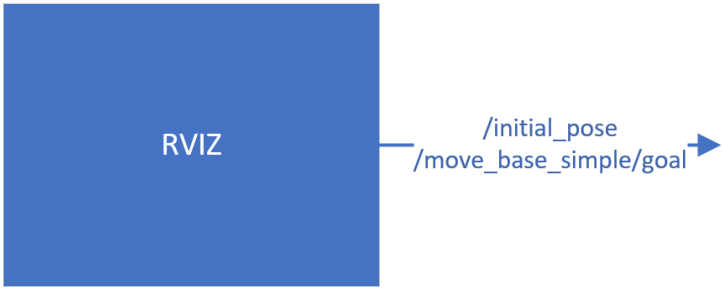

Figure 74: RVIZ

For the mission planning layer, RVIZ needs to define the initial pose of the ego-vehicle on a map and the goal position for the mission planning. Once this is defined the trajectory is computed which leads the ego-vehicle to the desired location on the map in compliance with the traffic regulations. All of this is explained in more detail afterwards.

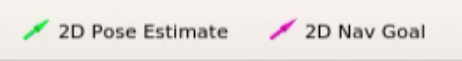

Figure 75: RVIZ pose estimate and navigation goal buttons

With the buttons above the pose estimate and the navigation goal can be entered, why this is needed is explained afterwards.

4.3 Map

In the map layer, all necessary maps for the AD Stack are loaded and configured. The map also referred to as High Definition Map or HD Map contains centimeter level accurate information of the environment especially designed for the aid of Autonomous Vehicles. The map is used in each layer of the AD Stack for sensing, perception, localization, decision making, planning and control. It contains information about the direction of travel, lanes crossings, stop lines, speed limits and more explained below. In Autoware.AI, the HD Map is divided into two different files. The first file is the Vector Map which contains the road information and the second is the Point Cloud Map which contains the environment captured by a LIDAR. [44] [45]

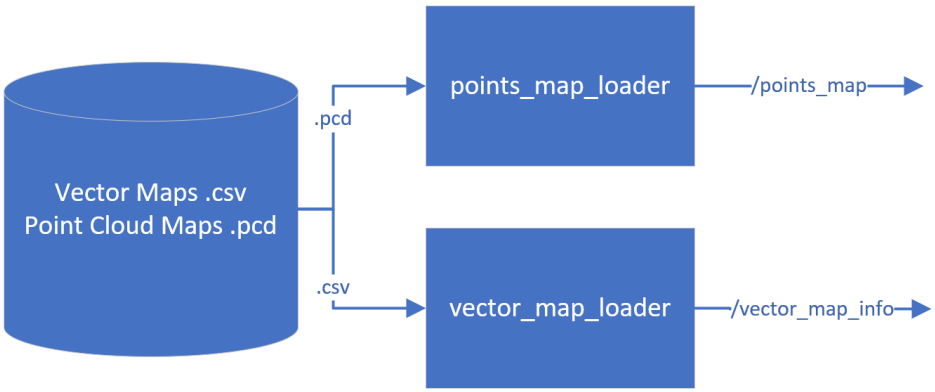

Figure 76: Map block diagram

The vector_map_loader fetches the .csv data with all the road information and then publishes it onto ROS under the topic /vector_map_info. The points_map_loader fetches the Point Cloud Map in the .pcd format and publishes it under /points_map onto ROS.

4.3.1 HD Maps

The information of the HD Map in general can be split in five categories/layers which are described below. [44]

Geometric

The geometric layer includes lines, strings and polygons in shape of the road, stored as point coordinates in the HD Map. It also contains information about the environment like a LIDAR point cloud. In Autoware.AI this information is contained in the Point Cloud Map. [44]

Semantic

The semantic layer contains information about the direction of travel, crossing or stop lines, the speed limit and further. This information is included in the .csv file in Autoware.AI. [44]

Road/Lane Network

This part contains information about different sections of the road like the preceding lane, adjacent lanes, and conflicting areas for instance intersections. It defines how the lanes are connected with each other and is important to generate routes. [44]

Experimental

The experimental layer contains information about the actual traffic flow, this layer is updated by all the cars on the road while driving or traffic regulators. [44]

Realtime

The realtime layer contains information about traffic jams, accidents, road closures or construction sites on the road. This layer allows the car to react on real time events on the road. [44]

4.3.2 Vector Maps

Vector Maps are used to present the HD Map Layer of Semantic and Road/Lane network in Autoware.AI. The CARLA installation includes such maps for the different towns in CARLA. The supported formats from Autoware.AI are the Aisan Vector map in the .csv format and the lanelet2 map in the .osm format. One disadvantage about the Aisan Vector Map is that the format is proprietary and all future Autoware projects will support the lanlet2 map. [44] [45]

Figure 77: CARLA Town 1

Visible above is the CARLA simulator with an intersection in Town 1 and in the figure below is the representation of this intersection with the Aisan Map and the Lanelet2 map.

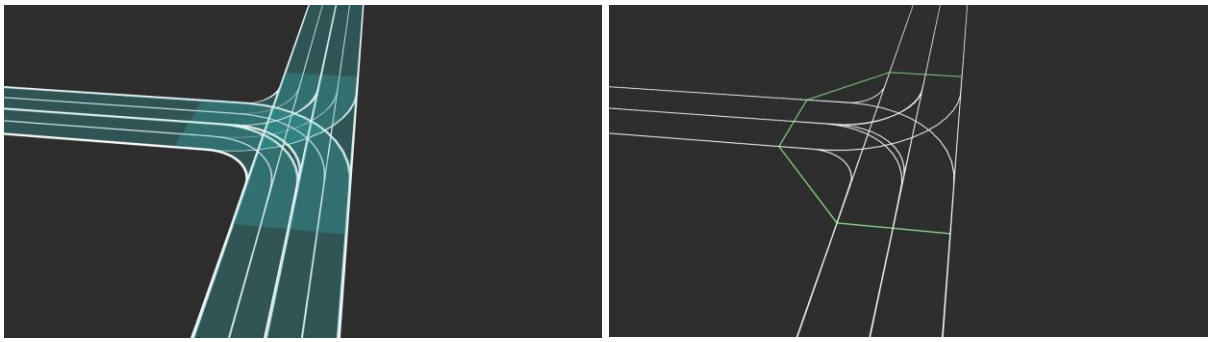

Figure 78: CARLA Town 1 Lanelet 2 map on the left and Aisan Map on the right

In Figure 78 you can see the Lanelet2 Map on the left and Aisan Map on the right. This is visualized in RVIZ.

4.3.3 PCD Maps

The Point Cloud Map represents the geometric layer of the HD Map, it includes the environment as a recorded point cloud captured by a LIDAR.

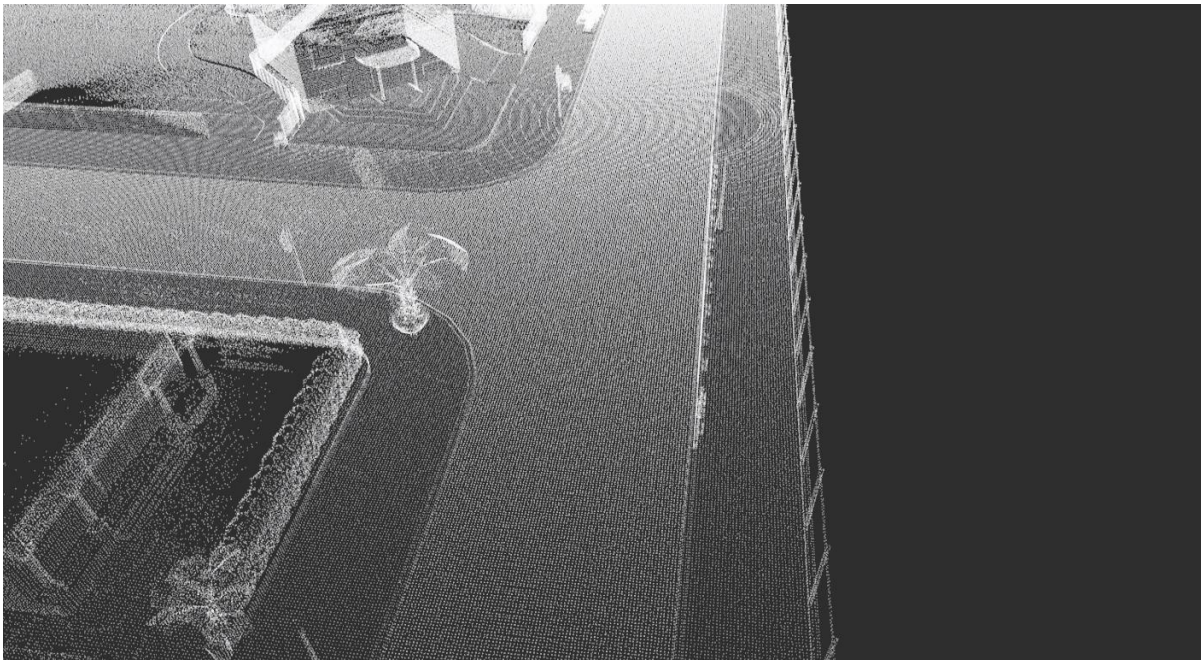

Figure 79: Point cloud representation of intersection of CARLA Town 1

Figure 79 shows the point cloud representation of the intersection from Figure 77 visualized in RVIZ. This point cloud in combination with the normal distribution transform is used for the localization in Autoware.AI. [44] [45]

4.4 Sensing

The sensing block launches the ray_ground_filter which is already explained in section 3.1.6, it delivers the ground and non-ground points from the perspective of the ego-vehicle. The non-ground points are visualized in Figure 81.

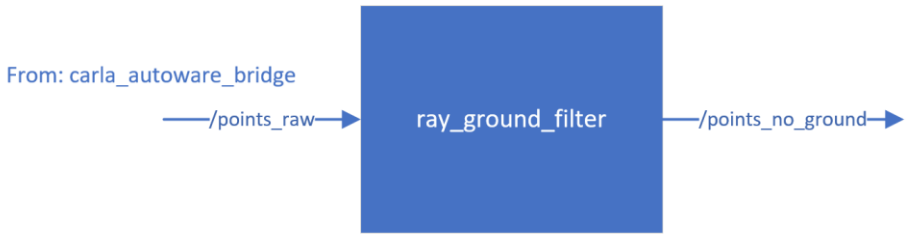

Figure 80: Sensing launch file block diagram

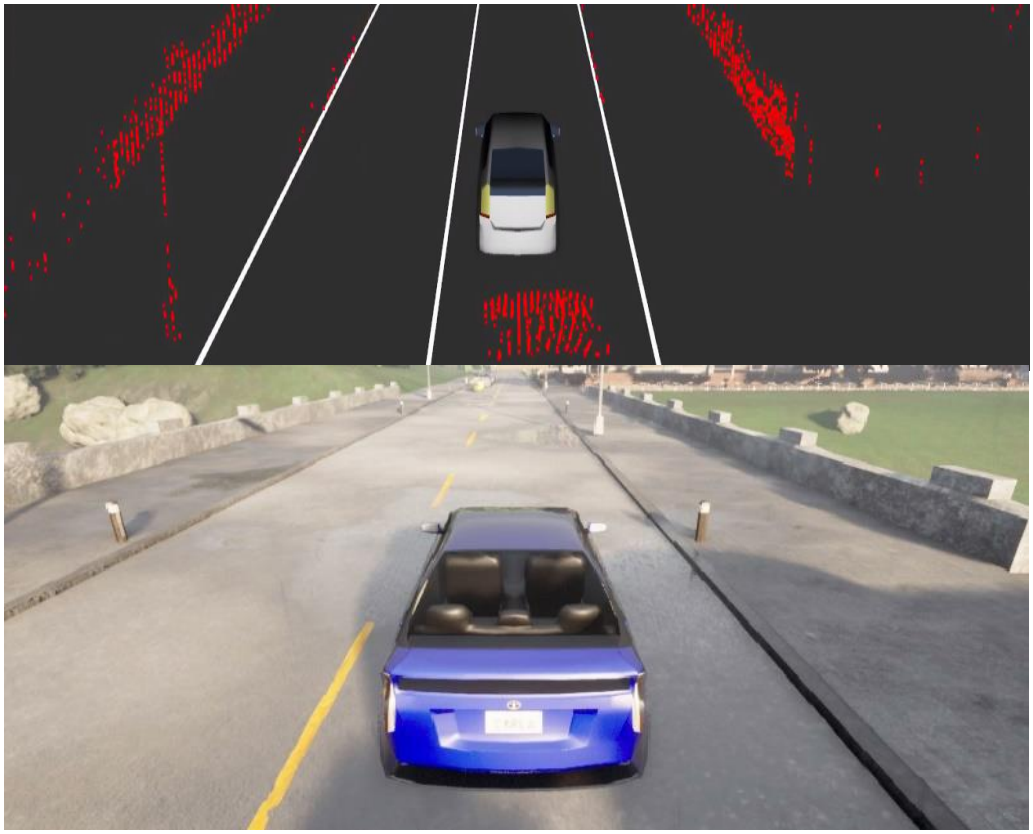

Figure 81: Points non-ground above and the CARLA ego-vehicle below

The upper part of Figure 81 depicts the non-ground points from the ray_ground_filter in red. How this filter works is already explained in 3.1.6 and will not be discussed here.

4.5 Localization

Localization determines actual position of the ego-vehicle on a map.

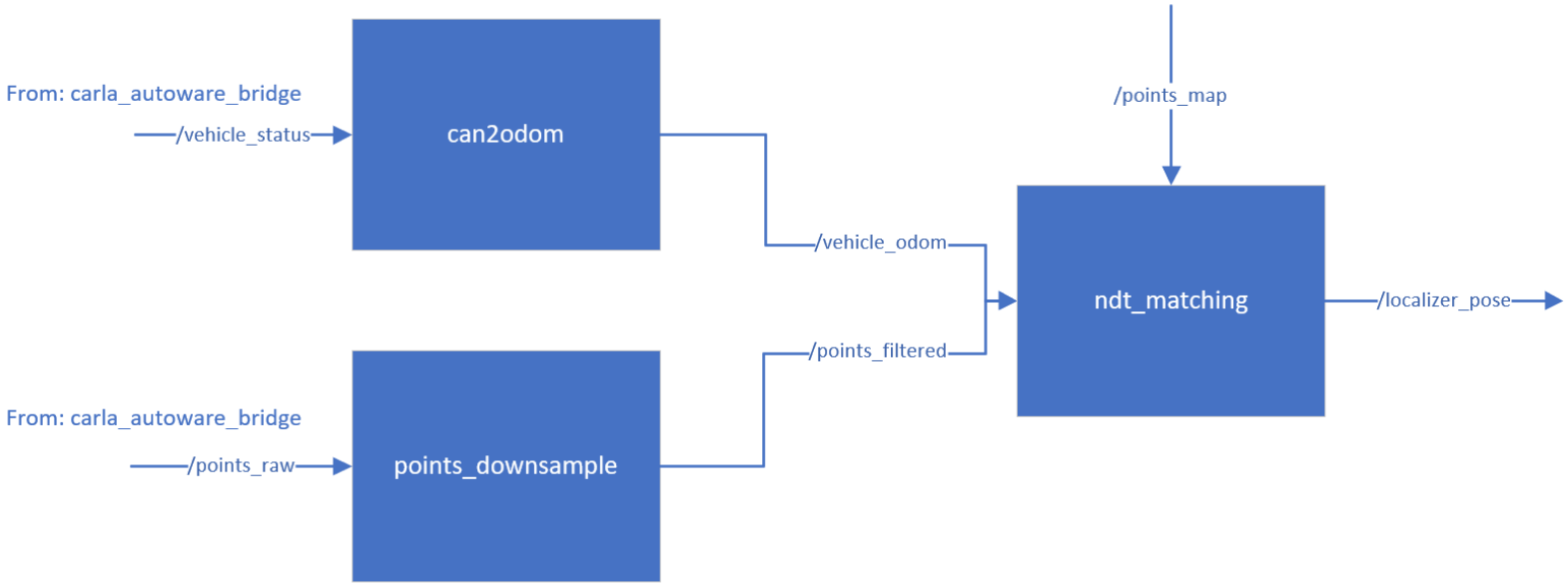

Figure 82: Localization launch file block diagram

The Localization layer launches multiple nodes/launch files which are displayed above. The block can2odom converts the /vehicle_status to odometry information about the ego-vehicle and republishes it under /vehicle_odom. The block points_downsample, down samples the incoming /points_raw and republishes them as /points_filtered. How the down sampling process works is explained in 3.1.4. The ndt_matching block then takes all the information from the odometry, the down sampled LIDAR /points_raw and the Point Cloud Map from the .pcd file with the topic /points_map. Then it tries to find the exact location from the ego-vehicle based on Normal Distribution Matching which will be described now.

4.5.1 Normal Distribution Matching

Normal Distribution Matching is used to localize the position of the ego-vehicle within a point cloud map. It uses the probability density function [46] [47]:

$$f(x) = \frac{1}{\sqrt{2\pi^k|\sum|}} e^{-\frac{1}{2}(x-\mu)\sum^{-1}(x-\mu)^T} \tag{4.1}$$

Whereby $\sum$ is the covariance of the distribution and μ is the mean of the distribution.

2D Normal Distribution Transform (NDT)

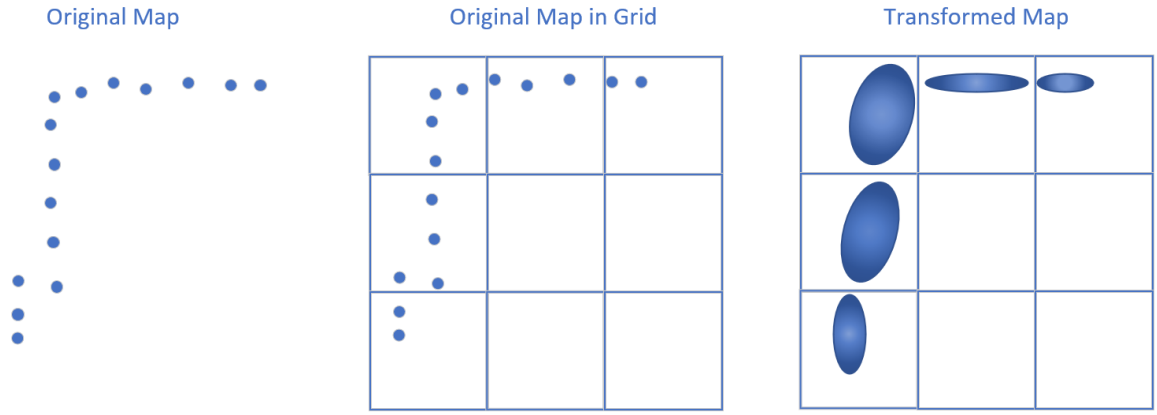

Figure 83: 2D Normal Distribution Transform own portrayal after [46] [47]

The original map on the left side gets a grid overlay which is visible in the middle. Each cell from the grid then represents a normal distribution of points. On the right side the transformed map is visible. [46] [47]

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{4.2}$$

$$\sum = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \mu)(x_i - \mu)^T \tag{4.3}$$

The mean μ and the covariance $\sum$ are computed by using each point x in a cell with N number of points. To locate the ego-vehicle, the points from the actual LIDAR are correlated with the probability density to find out if the point actually belongs into that cell. [46] [47]

NDT Alignment Problem

The probability density function $p(x)$ of a cell C for the point x is correlated with the probability that the actual scan point belongs into the cell C. If a high number of the points from the ego-vehicle LIDAR

have a high probability of being occupied in the map grid one can assume that there is a good alignment from the points of the actual scan. [46] [47]

$$p(x) = \frac{1}{C} e^{-\frac{1}{2}(x-\mu)\Sigma^{-1}(x-\mu)^T} \tag{4.4}$$

To achieve the optimal alignment of the sensed LIDAR points within the transformed recorded point cloud, the following formula must be maximized. [46] [47]

$$\sum_{i=1}^{N_s} e^{-\frac{1}{2}\left(T\left(x_{s_i},p\right)-\mu_i\right)\Sigma_i{}^{-1}\left(T\left(x_{s_i},p\right)-\mu_i\right)^T} \tag{4.5}$$

Where $T\left(x_{s_i},p\right)$ is the transformation function which transforms x_s, which are the points actually sensed by the LIDAR, from the actual sensor frame into the map frame by using the pose p. Wherby μ_i and Σ_i are the mean and the covariance of the transformed point. [46] [47]

NDT Alignment Optimization

The equation (4.5) needs to be maximized for optimal alignment of the scan points. With this information an optimization technique can be applied whereby the LIDAR scan points as a group get rotated and shifted into a direction whereby the maximum of the equation is searched. [46] [47]

$$score(p) = \sum_{i=1}^{N_s} e^{-\frac{1}{2}\left(T\left(x_{s_i},p\right)-\mu_i\right)\Sigma_i{}^{-1}\left(T\left(x_{s_i},p\right)-\mu_i\right)^T} \tag{4.6}$$

The score is computed each time the scan points are shifted as a group. This goes on until the score is maximized. [46] [47]

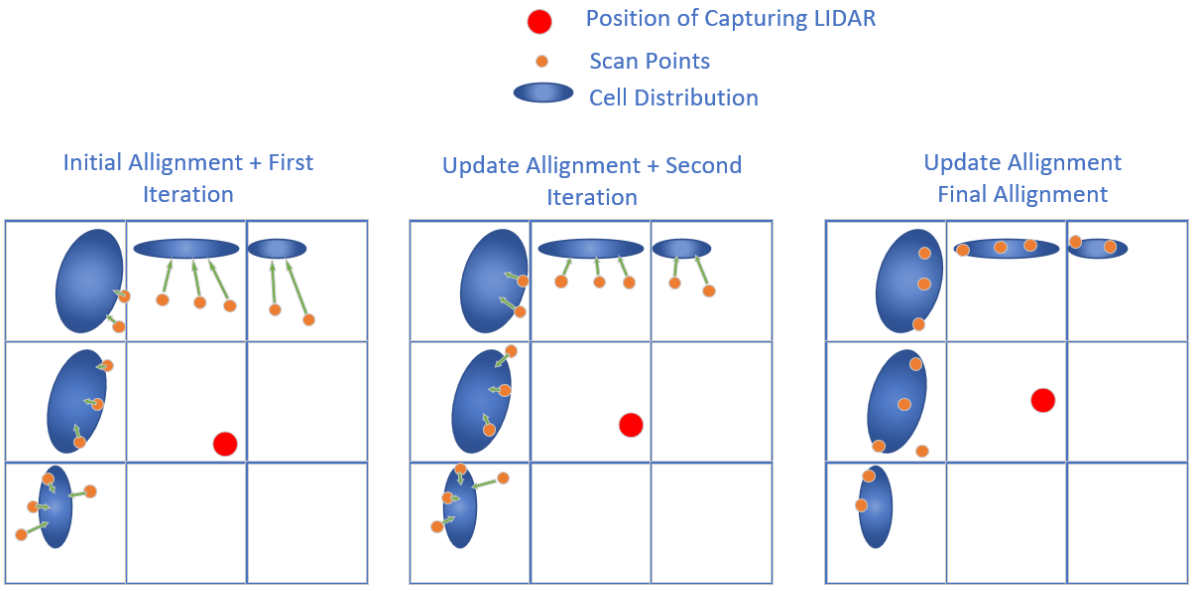

Figure 84: Normal Distribution Matching algorithm own portrayal after [46] [47]

At the initial alignment on the left, the LIDAR points in orange are correlated with the probability distribution in each cell (score is computed). The green arrow points into the direction of the maximum probability. Then the orange points are shifted and rotated as a group in the direction where the score will increase. Once the scan points are shifted also the theoretical position of the capturing LIDAR is shifted (red dot). When the alignment is updated (visible in the middle) the new score is computed, and this step is repeated until the maximum is found. On the right the final alignment is shown with the maximum probability, and the red dot shows the position of the capturing LIDAR. The red dot in the Figure 84 is the arrow in Figure 85. With this algorithm the position and pose of the capturing LIDAR can be found by correlating the sensed points with a stored point cloud with normal distribution matching also referred to as NDT matching. [46] [47]

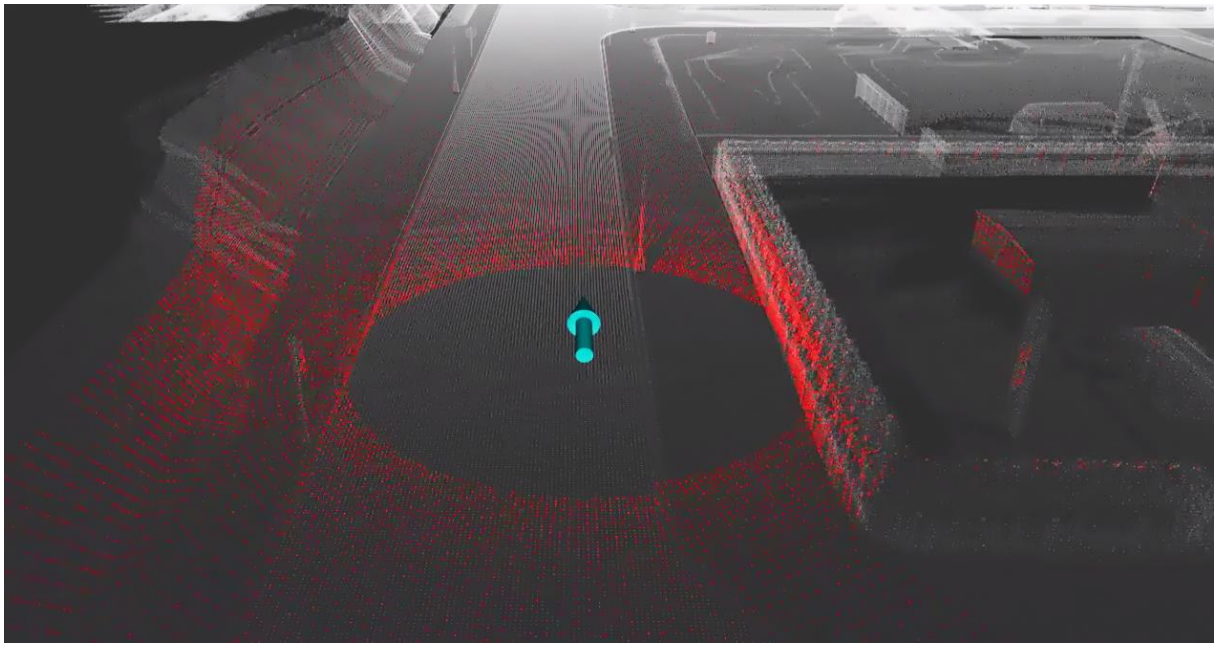

Figure 85: NDT matching

The white LIDAR point cloud is the previous recorded map, this map is transformed visible in Figure 84. Then the scanned points in red from Figure 85 get correlated with the transformed recorded map in white trough the NDT algorithm. The estimated pose of the vehicle gets published and is visible as the turquoise arrow in the middle of the figure above. Autoware.AI additionally uses odometry information to deliver a more precise information about the pose of the ego-vehicle.

4.6 Detection

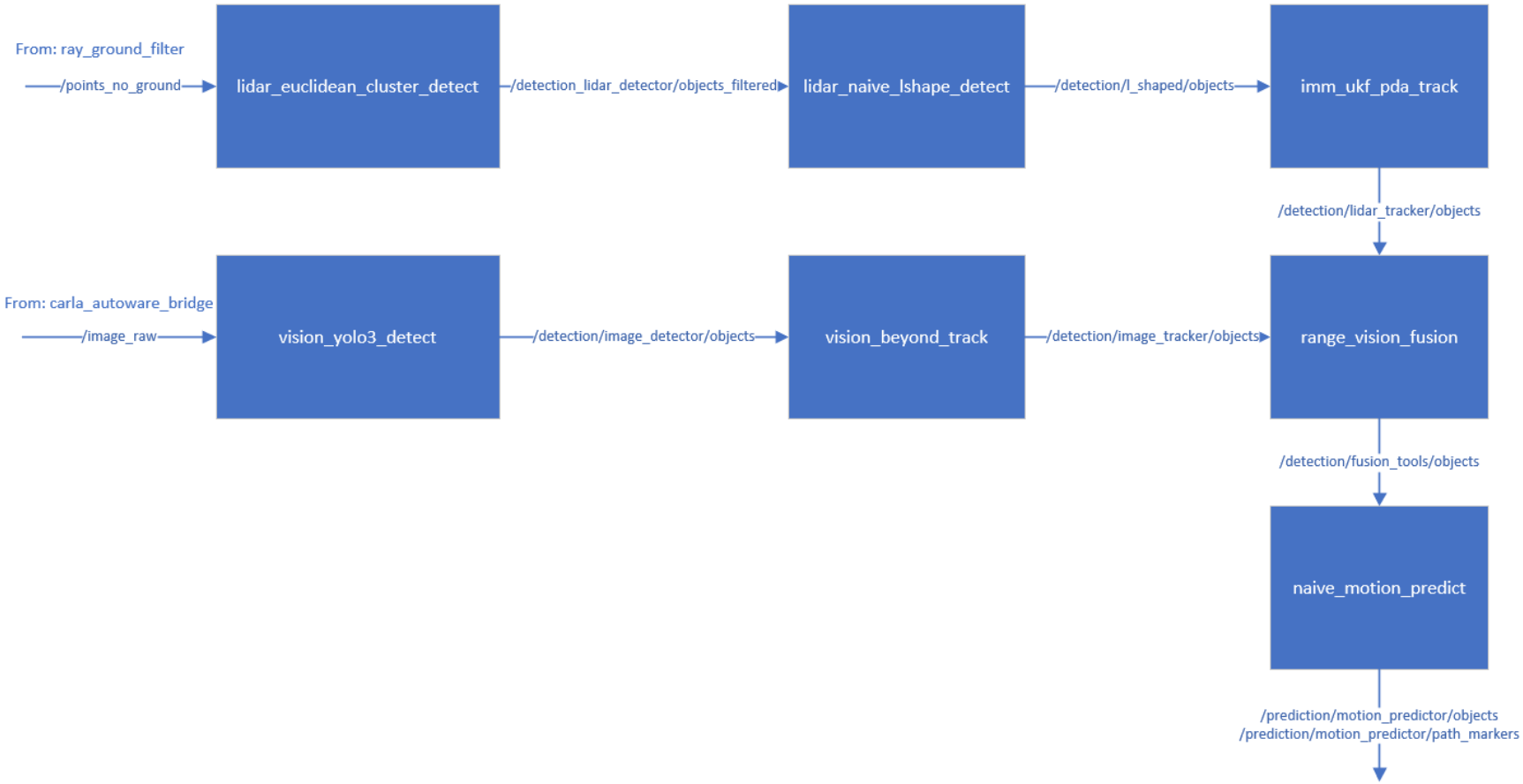

Figure 86: Autoware.AI detection block diagram

The block diagram in the Figure 86 shows all the nodes which handle the object detection.

4.6.1 LIDAR Detection

The topic /points_no_ground from the ray_ground_filter is the input for the lidar_euclidean_cluster_detect node. This node clusters the LIDAR points and afterwards the clustered objects are published under the topic detection_lidar_detector/objects_filtered. The clustered objects are the input for lidar_naive_lshape_detect which is generating bounding boxes for the clustered points. The algorithms used are already explained in detail in section 3.1.8 and 3.1.9 and will not be discussed again. The generated objects are visible in the image below.

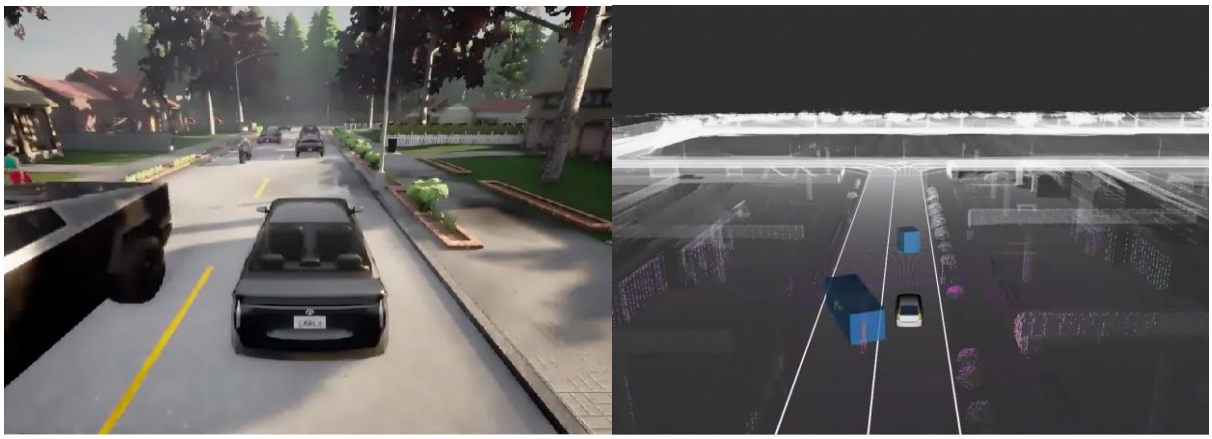

Figure 87: LIDAR objects detected

On the left side is the actual environment in the simulation environment CARLA, on the right side the RVIZ representation with the actual LIDAR perception and detection visible. The blue boxes on the right are the detected objects from the LIDAR detection. The node imm_ukf_pda_track then adds the speed, pose and the distance of the objects with respect to the ego-vehicle.

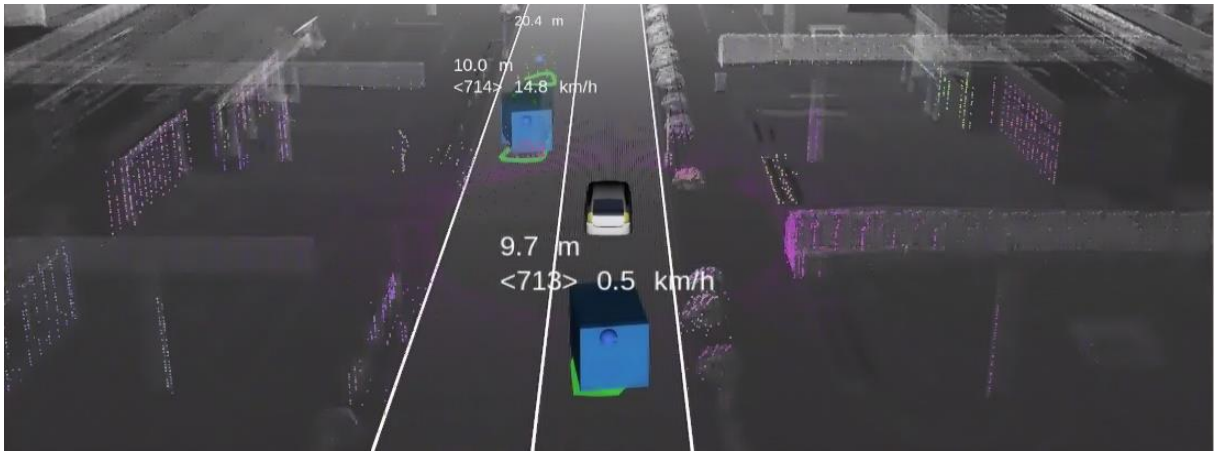

Figure 88: UKF PDA tracker

As can be seen in Figure 88 the distance and the speed are added to the objects. It uses the Bayesian Environment Representation for predicting and tracking, how this algorithm works precisely can be found in [48].

4.6.2 Camera Detection

Camera detection node vision_yolo3_detect, uses the YOLOv3 open-source neural network for object detection. The model is pre trained and it is capable of performing the needed computations in real time.

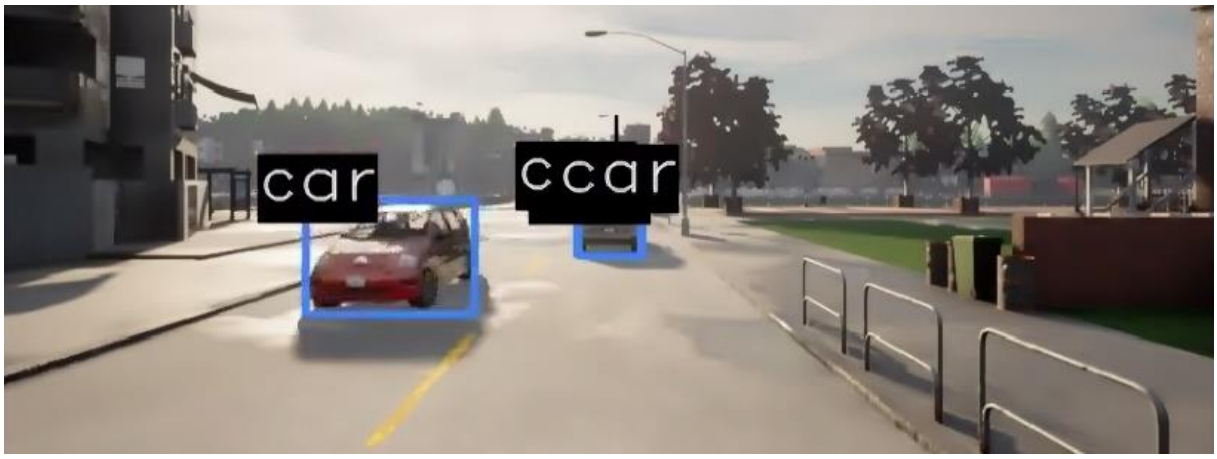

Figure 89: YOLOv3 detection

As can be seen in the image above the node detects the cars marks it with a rectangle and labels them with the name of the object detected. The node vision_beyond_track transforms the 2D objects from YOLOv3 into a 3D representation. It is based on the paper from Sarthak Sharma and Junaid Ahmed about Leveraging Geometry and Shape Cues for Online Multi-Object Tracking and is important for the fusion of the objects from LIDAR and camera. [49] [50]

4.6.3 Fusion

The node range_vision_fusion fuses the detected objects from camera and LIDAR. A match of the objects will be considered if the 3D projection from the vision_beyond_track is overlapping at least 50% with the object detected by the LIDAR. Once a match is found the label from the camera detection will be attached to the fused object in 3D space. This is visible in the next figure. [51]

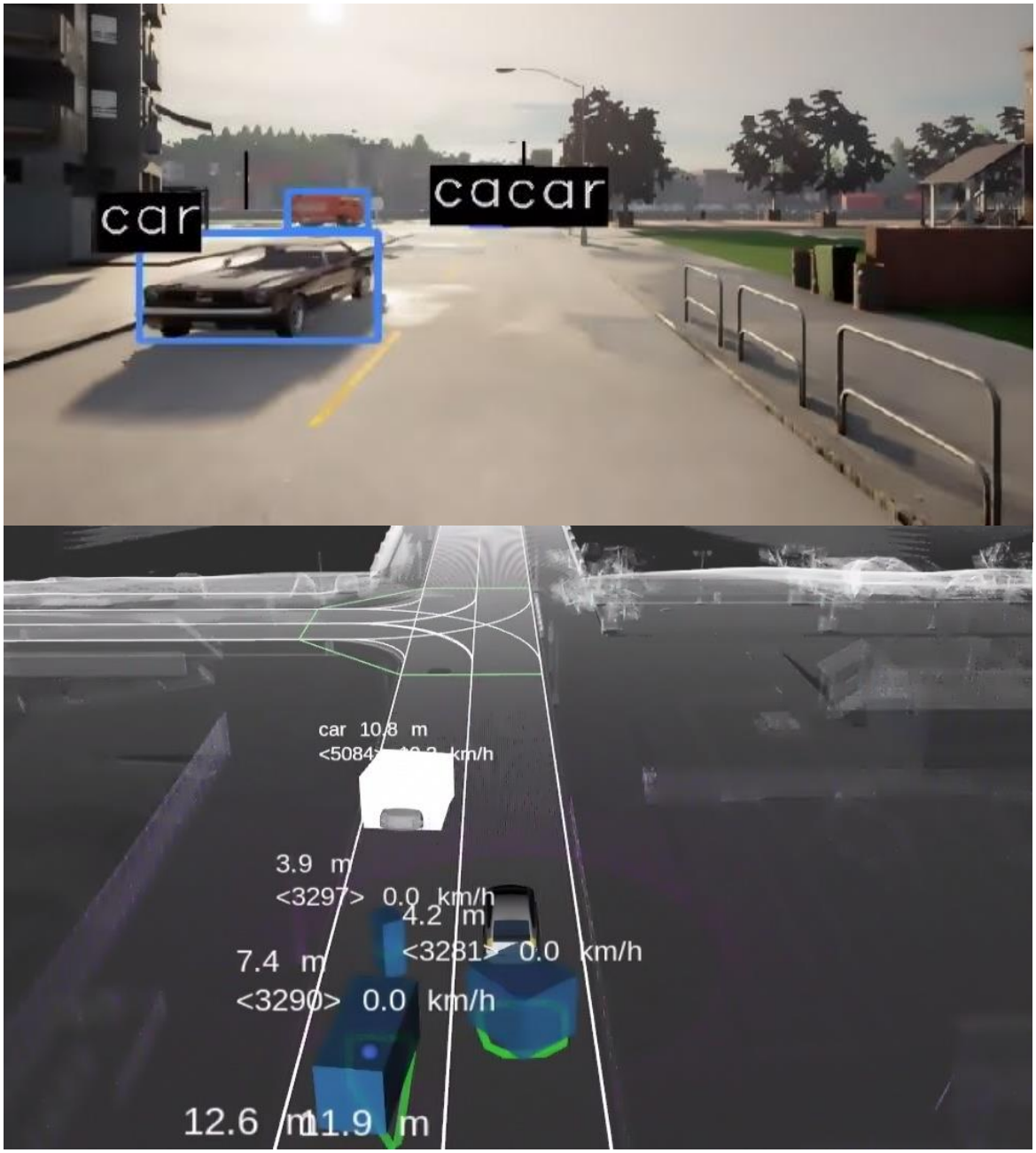

Figure 90: LIDAR camera fusion

The upper part of the image shows the front camera of the ego-vehicle with the detected objects from the image detection. Directly below with the blue bounding boxes are the detected objects from the LIDAR. The white bounding box displays the fused object from camera and LIDAR. In this case only one car was captured sufficiently well by the LIDAR and camera to be fused. The label from the camera detection was then added to the 3D object representation. Since the ego-vehicle only has a front camera all the cars behind in were not labeled.

4.6.4 Prediction

The node naive_motion_predict uses the fused objects from the range_vision_fusion node and predicts the path.

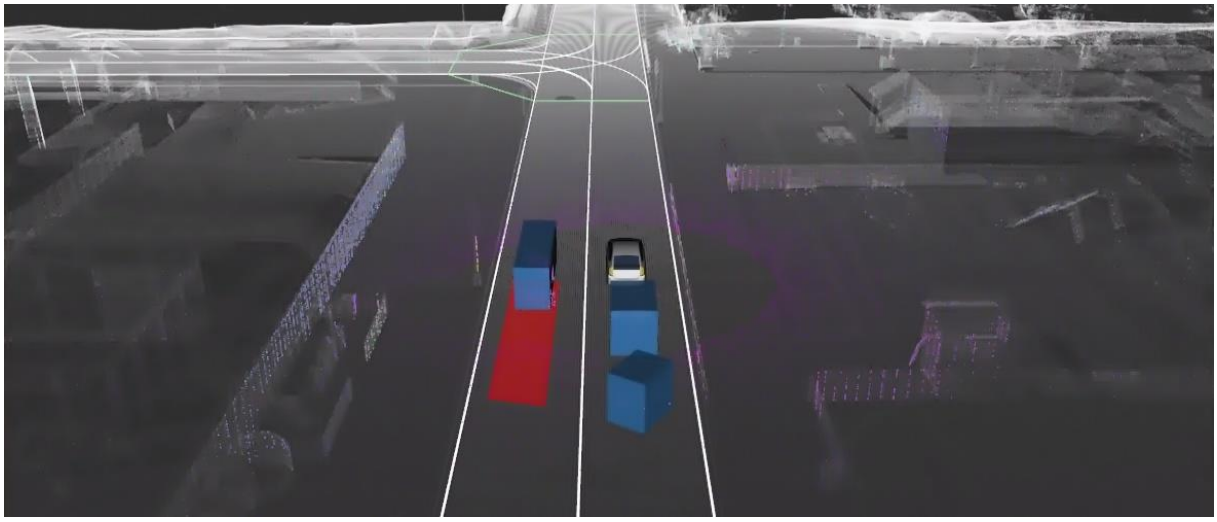

Figure 91: Path prediction

The future path in red of the objects in blue is predicted. This prediction will then be used in the motion planning.

4.7 Mission Planning

Mission planning is all about finding the right trajectory for the ego-vehicle.

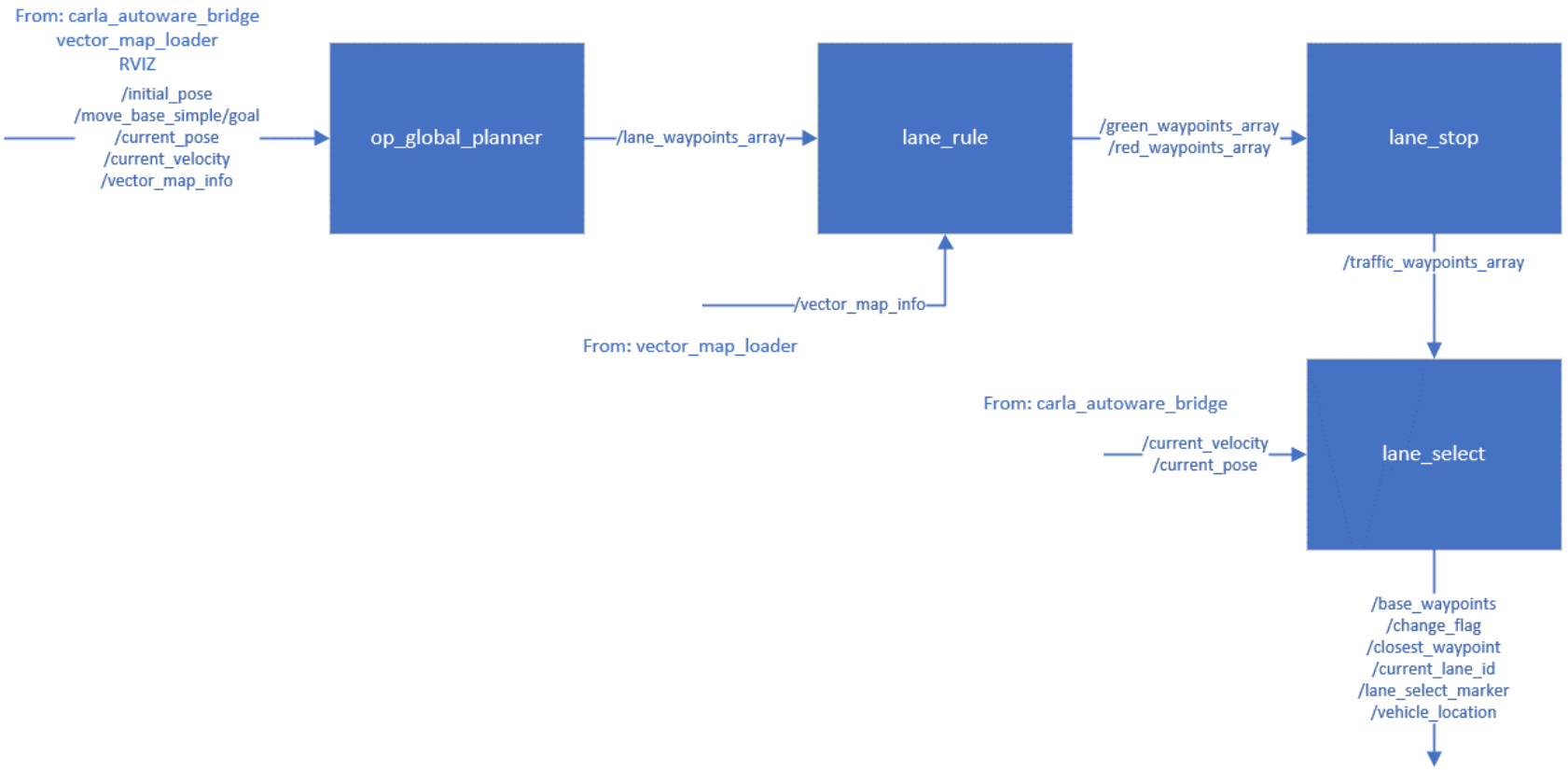

Figure 92: Mission planning Autoware.AI block diagram

4.7.1 Global Planning

The node op_global_planner visible in the mission planning block diagram generates a global path from point A to point B on a vector map. To plan this trajectory dynamic programing techniques are used whereby the only planning cost is the distance. The global planner plans the shortest path from point A to point B with the given traffic rules and the road network of the vector map. It does not consider road blocking objects since it has only the vector map as street template. [52] [53]

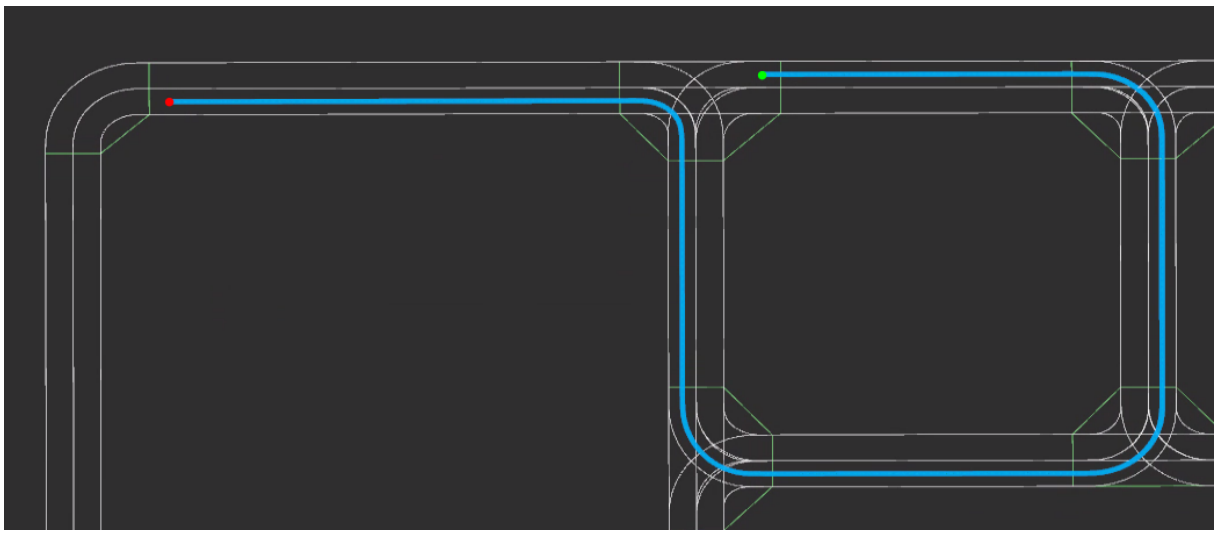

Figure 93: Global planner trajectory

The start point is marked with the red dot and the green dot marks the destination. The global planner visualizes the trajectory with the blue line, this lane is stored as waypoints.

4.7.2 Lane Planning

The Aisan Maps from CARLA do not yet have information about the positions of traffic lights included, therefore traffic light detection does not work in this example and the car will not stop at red signals. The lane_rule node detects stop lines in combination with traffic lights and publishes new waypoints according to the traffic light color. It will publish two different waypoints simultaneously, red waypoints in case the traffic light is red and green waypoints if the traffic light is green. The lane_stop node will then decide which waypoints, red or green, according to the traffic light, are taken by the car to follow. The lane_select node determines the correct lane by name, since there is no documentation from Autoware.AI this algorithm cannot be explained. [54]

4.8 Motion Planning

Motion planning as the name already implies, plans the motion for the ego-vehicle. It reacts to obstacles on the road and recomputes the trajectory if necessary. It also controls the car.

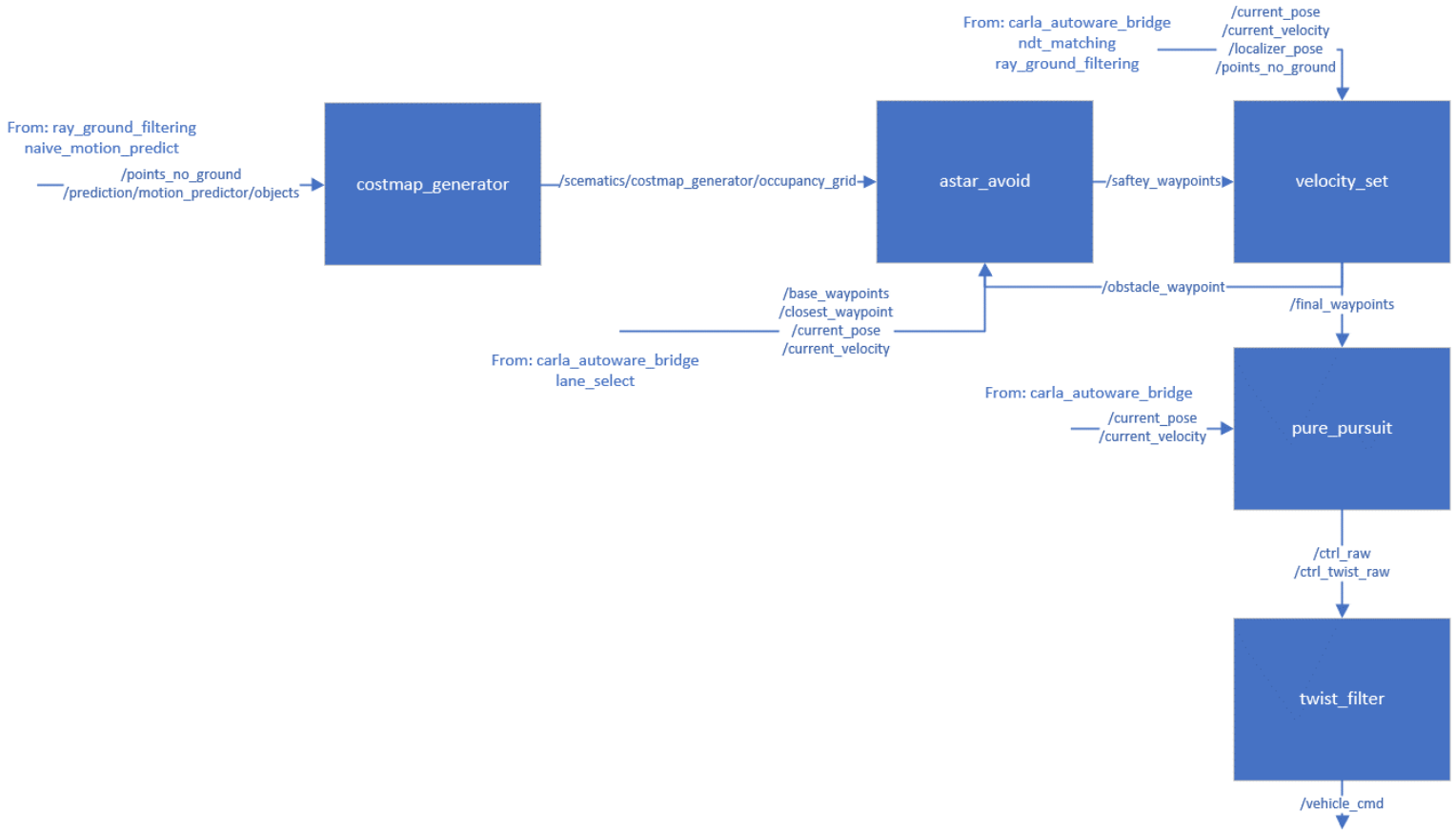

Figure 94: Motion planning block diagram

4.8.1 Costmap Generator

The costmap_generator generates an occupancy grid out of the /points_no_ground from the ray_ground_filter node in combination with the objects from the motion predictor. The output is an occupancy grid that shows which parts of the road are occupied by other road users.

4.8.2 A* Avoid

The node astar_avoid uses this occupancy grid from the costmap_generator to compute new waypoints if an obstacle is in the trajectory of the ego-vehicle. The new waypoints are then published under /safety_waypoints.

4.8.3 Velocity Set

The velocity_set node sets the velocity for the waypoints to achieve the target velocity at each point of the map using a feed-forward/back control. [55]

4.8.4 Pure Pursuit

The pure_pursuit node is a trajectory following algorithm which is widely used in autonomous robot applications. The trajectory for the ego-vehicle will be computed here to follow the waypoints. [56]

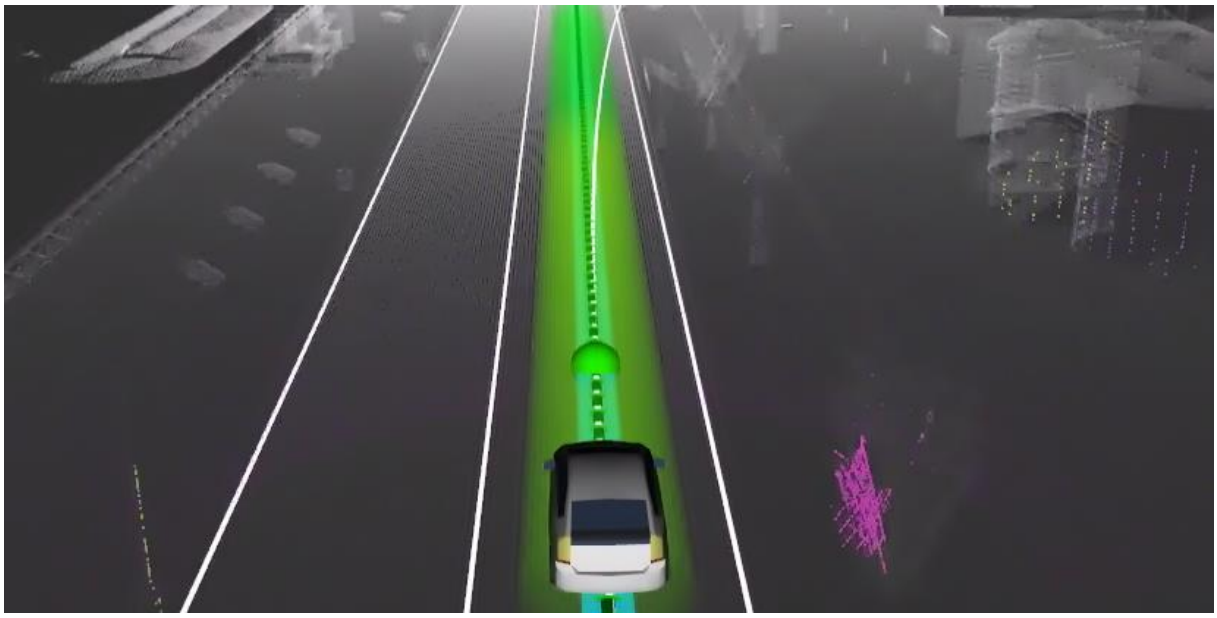

Figure 95: Computed trajectory steering angle from pure pursuit

The white line in in front of the ego-vehicle in the Figure 95 is the computed trajectory the ego-vehicle will take with the actual steering angle. This trajectory and steering angle gets refreshed with each frame.

4.8.5 Twist Filter

The twist_filter node transforms the topics /ctrl_raw and /ctrl_twist_raw commands from the Autoware format into the CARLA format. CARLA will then take these control values from ROS and the ego-vehicle in the simulation will react to these values.

4.9 Discussion of Results

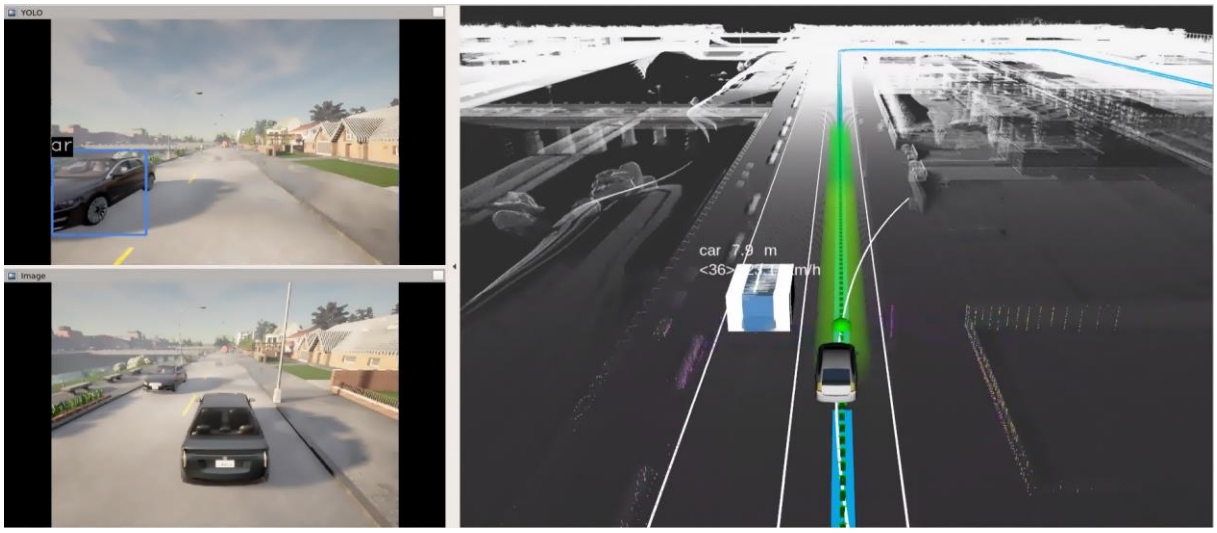

Figure 96: CARLA Autoware.AI full self-driving setup

Figure 96 shows the result of all the explained block diagrams before. On the left side the view from the CARLA simulator is shown and on the right side the representation of the environment from the Autoware.AI stack is visualized. The ego-vehicle autonomously drives from point A to point B whereby it constantly keeps the lane and the desired speed defined. It reacts to obstacles on the road and tries to prevent collisions if the objects are detected in time. The stack is able to drive autonomously but the detection does not always detect objects in time, therefore the car sometimes crashes into other vehicles. Additionally, Autoware.AI is not well documented, it seems that there was no real focus on documentation. Some nodes do not have a simple explanation of what they are supposed to do or how they do something. Also, Autoware.AI will not be developed further and will therefore not be improved anymore.

5 Conclusion

In the beginning, all the necessary basics needed to understand automated vehicle software stacks like Autoware.Auto and Autoware.AI were discussed. The basis of automated vehicles form the Advanced Driver Assistance Systems (ADAS), which are the building blocks for self-driving vehicles. The levels of self-driving have been standardized by SAE International, ranging from level 0, which only gives warnings to the driver, to level 5, where the vehicle is already fully autonomous. The software stack of a self-driving car can be divided into layers, which are basically vehicle control, sensing, processing and decision making. The Robot Operating System (ROS) is a robotics middleware from the Open Source Robotics Foundation. It provides a simple publish subscribe architecture. Autoware.Auto uses the second version of ROS therefore ROS2 and Autoware.AI is based on the first version. ROS in general provides a python and a C++ API with which the middleware can be easily accessed. With the help of a ROS Bridge data can be exchanged between the different ROS versions. RVIZ is a three-dimensional visualizer for robots. It is also used by Autoware to display LIDAR point clouds or camera data. The Vehicle to anything (V2X) technology enables communication between vehicles based on the WLAN standard 802.11.p. Different frequencies have been allocated for the V2X technology by the European Telecommunications Standards Institute (ETSI). The Autoware Foundation developed two different automated driving stacks where Autoware.AI was the first full self-driving open-source stack. The second generation, therefore the successor of Autroware.AI is Autoware.Auto where the basic use case was autonomous valet parking. CARLA is an open-source traffic simulator which provides sensors like LIDAR, radar, camera and more. During the implementation of the LIDAR perception and detection of Autoware.Auto, the basics of the LIDAR object detection were discussed. Whereby the detection always consists of several function blocks. First the data is preprocessed and unnecessary data from the LIDAR point cloud is filtered out to gain efficiency. Simple preprocessing filters are range or angle based filtering or down sample algorithms. It is very important to separate the ground points from the non-ground points before object detection, this is done by the ray ground filter already implemented in Autoware.Auto. The non-ground points from the ray ground filter will then be clustered and transformed into bounding boxes by the euclidean cluster node. During the implementation itself, it was found that ground filtering is one of the most important steps in LIDAR detection. The subsequent detection is strongly dependent on the preceding ground detection. For the extension of the perception layer with a V2X sensor from Autoware.Auto, scripts were written with the help of the ROS python API which extract vehicle information like position, rotation, and dimensions from CARLA. Since this data was in a global coordinate system from CARLA it needed to be transformed into a local coordinates system

with respect to the ego-vehicle to be able to visualize it in RVIZ. It was shown that an ideal V2X sensor detects much more objects in the environment of the ego-vehicle than the LIDAR sensor does. The Autoware.AI stack claims to be a full self-driving stack. Therefore, the whole stack was examined, its function explained and then a self-driving demo has been set up in combination with CARLA. The stack can be divided into several parts: map, sensing, localization, mission planning and motion planning. The map part publishes information about the road lines, stop lines and traffic rules onto ROS. The information about the schematics of the road is fetched from a vector map and the environment is stored as a pre-recorded point cloud. The vector maps are important for mission planning and the point cloud map is important for localization. The sensing part takes the LIDAR data and divides it into non-ground and ground via a ray ground filter. The localization part localizes the vehicle within the map using the pre-recorded point cloud from the map part and the actual LIDAR data using the Normal Distribution Matching algorithm. LIDAR and camera data are used for detection. The detected objects from the camera and the LIDAR are fused with each other at the end. This enables labeled objects in 3D space. Afterwards the future positions of the detected objects are predicted. In mission planning, a global planner is used to calculate a trajectory that will take the car from point A to point B. This trajectory is then stored as waypoints. In motion planning, the ego-vehicle in the CARLA simulation is controlled by the AD stack and follows the waypoints previously created in mission planning. Both stacks work well, and the included functionality is broad. But the lack of documentation of the open-source community makes it hard to work with. Some included functions of the stacks are only slightly or not documented at all.

Bibliography

[1] T. Denton, Automated Driving and Driver Assistance Systems, New York: Routledge, 2020.

[2] T. AD, "Conference Proceedings," in *Annual Conference on Automated Driving*, Berlin, 2015.

[3] M. H. Daniel Watzenig, Automated Driving, Switzerland: Springer, 2017.

[4] N. a. U. D. o. T. Transportation, "US Department of Transportation Announces Decisions to Move Forward with Vehicle-toVehicle Communication Technology for Light Vehicles," 2014.

[5] SAE, "Taxonomy and Definitions for Terms Related to On-Road Motor Vehicle Automated," SAE International Standard, 2014.

[6] S. International, "www.sae.org," 16 09 2021. [Online]. Available: https://www.sae.org/blog/sae-j3016-update. [Accessed 16 09 2021].

[7] O. Robotics, "ROS.org," [Online]. Available: http://wiki.ros.org/ROS/Introduction. [Accessed 17 09 2021].

[8] "Wikipedia," [Online]. Available: https://en.wikipedia.org/wiki/Willow_Garage#/media/File:PR2_Tabletop.jpg. [Accessed 21 09 2021].

[9] H. Oladepo, "Wiki ROS," Open Source Robotics, [Online]. Available: http://wiki.ros.org/Nodes. [Accessed 22 09 2021].

[10] T. Foote, "Wiki ROS," Open Source Robotics, [Online]. Available: http://wiki.ros.org/Topics. [Accessed 22 09 2021].

[11] O. Robotics, "Docs Ros," Open Robotics, [Online]. Available: https://docs.ros.org/en/foxy/Tutorials/Topics/Understanding-ROS2-Topics.html. [Accessed 19 11 2021].

[12] A. Koubaa, "Wiki ROS," Open Source Robotics, [Online]. Available: http://wiki.ros.org/Services. [Accessed 22 09 2021].

[13] O. Robotics, "Docs Ros," Open Robotics, [Online]. Available: https://docs.ros.org/en/foxy/Tutorials/Services/Understanding-ROS2-Services.html. [Accessed 19 11 2021].

[14] "Docs ROS," Open Robotics, [Online]. Available: https://docs.ros.org/en/rolling/Tutorials/Parameters/Understanding-ROS2-Parameters.html. [Accessed 22 09 2021].

[15] J. P. S. L. Geoffrey Biggs, "Design ROS2," Open Source Robotics Foundation, Inc., [Online]. Available: http://design.ros2.org/articles/actions.html. [Accessed 05 10 2021].

[16] O. Robotics, "Docs ROS," Open Robotics, [Online]. Available: https://docs.ros.org/en/foxy/Tutorials/Understanding-ROS2-Actions.html. [Accessed 05 10 2021].

[17] GvdHoorn, "WIKI ROS," Open Robotics, [Online]. Available: https://docs.ros.org/en/foxy/Tutorials/Understanding-ROS2-Actions.html. [Accessed 05 10 2021].

[18] TullyFoote, "WIKI ROS," Open Robotics, [Online]. Available: http://wiki.ros.org/roscpp. [Accessed 05 10 2021].

[19] D. Thomas, "ROS 2 Design," [Online]. Available: http://design.ros2.org/articles/ros_middleware_interface.html. [Accessed 21 09 2021].

[20] W. Woodall, "Design ROS 2," Open Source Robotics, [Online]. Available: https://design.ros2.org/articles/ros_on_dds.html. [Accessed 22 09 2021].

[21] "Github," [Online]. Available: https://github.com/ros2/ros1_bridge/blob/master/doc/index.rst. [Accessed 05 10 2021].

[22] ROS.org, "WIKI ROS," Open Robotics, [Online]. Available: http://wiki.ros.org/rviz/Tutorials. [Accessed 13 10 2021].

[23] IBM, "IBM Learn," IBM, [Online]. Available: https://www.ibm.com/in-en/cloud/learn/docker. [Accessed 13 10 2021].

[24] S. N. C. S. Hasan Syed Faraz, Intelligent Transport Systems, New York: Springer, 2013.

[25] B. B. C. K. Marc Emmelmann, Vehicular Networking: Automotive Applications and Beyond, Chichester: John Wiley & Sons Ltd, 2010.

[26] N. Rola, "Wireless Vehicular Networks for Car Collision Avoidance," Springer, New York, 2013.

[27] ETSI, "ETSI ES 202 663," ETSI, Sophia Antipolis Cedex, 2011.

[28] T. A. Foundation, "Autoware Org," The Autoware Foundation, [Online]. Available: https://www.autoware.org/. [Accessed 13 10 2021].

[29] T. A. Foundation, "Autoware.AI," The Autoware Foundation, [Online]. Available: https://www.autoware.org/autoware-ai. [Accessed 14 10 2021].

[30] C. Org, "CARLA Simulator," CARLA Team 2021, [Online]. Available: https://carla.org/. [Accessed 19 10 2021].

[31] A. D. a. G. R. a. F. C. a. A. L. a. V. Koltun, "CARLA: An Open Urban Driving Simulator," 2017.

[32] T. A. Foundation, "Github," The Autoware Foundation, [Online]. Available: https://github.com/Autoware-AI/autoware.ai/wiki/Overview. [Accessed 11 11 2021].

[33] Apex.AI, Inc, "Gitlab," 2019. [Online]. Available: https://gitlab.com/ApexAI/autowareclass2020/-/tree/master/lectures/07_Object_Perception_LIDAR. [Accessed 29 10 2021].

[34] Apex.AI, "Gitlab," Apex.AI, [Online]. Available: https://gitlab.com/ApexAI/autowareclass2020/-/blob/master/lectures/07_Object_Perception_LIDAR/04_-_Ground_Filtering.pdf. [Accessed 19 11 2021].

[35] Apex.AI, "Gitlab," Apex.AI, [Online]. Available: https://gitlab.com/ApexAI/autowareclass2020/-/blob/master/lectures/07_Object_Perception_LIDAR/03_-_LiDAR_Preprocessing.pdf. [Accessed 19 11 2021].

[36] R. B. Rusu, "Semantic 3D Object Maps for Everyday Manipulation in Human Living Environments," Computer Science department, Technische Universitaet Muenchen, Germany, 2009.

[37] S. M. LsValle, "Sampling-Based Motion Planning," Cambridge Universitiy Press, Cambridge, 2006.

[38] X. &. P. S. &. J. M. Shen, "Efficient L-shape fitting of laser scanner data for vehicle pose estimation," 2015.

[39] MathsPoetry, "Wikipedia," [Online]. Available: https://commons.wikimedia.org/wiki/File:Counterclockwise_rotation.png. [Accessed 05 11 2021].

[40] D. Malan, "Wikipedia," [Online]. Available: https://commons.wikimedia.org/wiki/File:Euler_AxisAngle.png. [Accessed 05 11 2021].

[41] A. J. Hanson, Vizualizing Quaternions, San Francisco: Elsevier, 2016.

[42] C. Community, "Github," [Online]. Available: https://github.com/carla-simulator/carla-autoware. [Accessed 25 11 2021].

[43] T. A. Foundation, "Autoware," The Autoware Foundation, [Online]. Available: https://www.autoware.org/autoware-ai. [Accessed 11 11 2021].

[44] S. Thompson, "Gitlab," [Online]. Available: https://gitlab.com/ApexAI/autowareclass2020/-/blob/master/lectures/14_HD_Maps/HD_Maps_for_Autonomous_Driving_Part_I.pdf. [Accessed 2021 11 2021].

[45] S. Thompson, "Gitlab," [Online]. Available: https://gitlab.com/ApexAI/autowareclass2020/-/blob/master/lectures/14_HD_Maps/HD_Maps_for_Autoware_Part_II.pdf. [Accessed 15 11 2021].

[46] W. S. Peter Biber, The Normal Distributions Transform: A New Approach to Laser Scan Matching, IEEE International Conference on Intelligent Robots and Systems: IEEE, 2003.

[47] Y. E. Caliskan, "Gitlab," [Online]. Available: https://gitlab.com/ApexAI/autowareclass2020/-/blob/master/lectures/10_Localization/2D_and_3D_NDT.pdf. [Accessed 16 11 2021].

[48] M. Schreier, "Bayesian environment representation, prediction, and criticality assessment for driver assistance systems," 2017.

[49] S. S. a. J. A. A. a. J. K. M. a. K. M. Krishna, "Beyond Pixels: Leveraging Geometry and Shape Cues for Online Multi-Object Tracking," arXiv, 2018.

[50] "Autoware Read The Docs," Autoware Community, 2018. [Online]. Available: https://autoware.readthedocs.io/en/feature-

documentation_rtd/DevelopersGuide/PackagesAPI/detection/vision_beyond_track.html. [Accessed 17 11 2021].

[51] A. Comunity, "Autoware Read The Docs," Autoware Community, [Online]. Available: https://autoware.readthedocs.io/en/feature-documentation_rtd/DevelopersGuide/PackagesAPI/detection/range_vision_fusion.html. [Accessed 17 11 2021].

[52] H. &. T. E. &. T. K. &. N. Y. &. S. A. &. M. Y. &. A. N. &. T. T. &. K. S. Darweesh, "Open Source Integrated Planner for Autonomous Navigation in Highly Dynamic Environments," Journal of Robotics and Mechatronics, 2017.

[53] A. Community, "Autoware Read The Docs," Autoware Community, [Online]. Available: https://autoware.readthedocs.io/en/feature-documentation_rtd/DevelopersGuide/PackagesAPI/mission/op_global_planner.html. [Accessed 18 11 2021].

[54] A. Community, "Github," Autoware Organization, [Online]. Available: https://github.com/Autoware-AI/core_planning/tree/master/lane_planner. [Accessed 18 11 2021].

[55] A. Foundation, "AutowareFoundation Gitlab," Autoware Foundation, [Online]. Available: https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/longitudinal-controller-design.html. [Accessed 18 11 2021].

[56] A. Foundation, "Autoware Foundation Gitlab," Autoware Foundation, [Online]. Available: https://autowarefoundation.gitlab.io/autoware.auto/AutowareAuto/pure-pursuit.html. [Accessed 18 11 2021].

[57] O. S. R. Foundation, "ROS2.org," [Online]. Available: http://design.ros2.org/articles/why_ros2.html. [Accessed 17 09 2021].

[58] T. A. Foundation, "Autoware.Auto," The Autoware Foundation, [Online]. Available: https://www.autoware.org/autoware-auto. [Accessed 14 10 2021].

[59] G. a. S. B. H. a. T. H. a. L. Q. a. L. Rong, LGSVL Simulator: A High Fidelity Simulator for Autonomous Driving, arXiv preprint arXiv:2005.03778, 2020.

Appendix

Source Code V2X Sensor

```
import rclpy
import time
import math
from rclpy.node import Node


from rclpy.time import Time
from std_msgs.msg import String
from carla_msgs.msg import CarlaActorList
from derived_object_msgs.msg import ObjectArray
from visualization_msgs.msg import Marker, MarkerArray


class MinimalPublisher(Node):

    marker_array = MarkerArray()

    def __init__(self):
        super().__init__('minimal_publisher')
        self.publisher_ = self.create_publisher(MarkerArray, 'RVIZ_MARKER_ARRAY', 10)
        timer_period = 0.5  # seconds
        self.timer = self.create_timer(timer_period, self.timer_callback)
        self.i = 0

    def makeMarker(self, pos_x, pos_y, pos_z, id):

        marker = Marker()
        marker.header.frame_id = "ego_vehicle"
        marker.header.stamp = self.get_clock().now().to_msg()
        marker.ns = "RVIZ_MARKER_ARRAY"
        marker.id = id
        marker.type = marker.CUBE
        marker.action = marker.ADD
        marker.scale.x = 3.0
        marker.scale.y = 1.0
        marker.scale.z = 1.0
        marker.color.r = 1.0
        marker.color.g = 0.0
        marker.color.b = 0.0
        marker.color.a = 1.0
        marker.pose.orientation.w = 0.0
```

```
            marker.pose.position.x = pos_y
            marker.pose.position.y = pos_x
            marker.pose.position.z = pos_z

            self.marker_array.markers.append(marker)
            #print(self.marker_array)

            return self.marker_array

        def timer_callback(self):
            self.publisher_.publish(self.marker_array)
            #print(self.marker_array)


    class MinimalSubscriber(Node):

        v2xList = []
        v2xIDList = []

        marker_array = MarkerArray()

        egoVehicleId = 0

        def __init__(self):
            super().__init__('minimal_subscriber')
            self.subscription = self.create_subscription(
                CarlaActorList,
                '/carla/actor_list',
                self.listener_callback,
                10)
            self.subscription  # prevent unused variable warning

            self.subscription1 = self.create_subscription(
                ObjectArray,
                '/carla/objects',
                self.listener_callback1,
                10)
            self.subscription1  # prevent unused variable warning


            self.publisher_ = self.create_publisher(MarkerArray, 'RVIZ_MARKER_ARRAY', 10)
            timer_period = 0.2  # seconds
            self.timer = self.create_timer(timer_period, self.timer_callback)
```

```
    def timer_callback(self):
        #self.marker_array.markers.clear()
        #self.makeMarker(0.0,0.0,0.0,100,0.0,0.0,0.0,0.0,1.0,1.0,1.0)
        self.publisher_.publish(self.marker_array)
        #print(self.marker_array)

    def listener_callback1(self, msg):
        #print(msg)
        #print(msg.objects[0].shape.dimensions[1])
        #
        print(msg.objects[0].pose.orientation.x)
        print(msg.objects[0].pose.orientation.y)
        print(msg.objects[0].pose.orientation.z)
        print(msg.objects[0].pose.orientation.w)
        #print()
        #print(msg.objects[0].pose.position.x)
        #print(msg.objects[0].pose.position.y)
        #print(msg.objects[0].pose.position.z)
        #print()
        #msg_str = str(msg)
        #self.v2xList.clear()
        self.marker_array.markers.clear()
        #print(msg.objects)
        for i in range(0, len(msg.objects)):
            if(msg.objects[i].id == self.egoVehicleId):
                egoVehicle = V2xCars(msg.objects[i].id, msg.objects[i].pose.position.x,
msg.objects[i].pose.position.y, msg.objects[i].pose.position.z, msg.objects[i].accel.linear.x,
msg.objects[i].accel.linear.y, msg.objects[i].accel.linear.z, "none", msg.objects[i].pose.orientation.x,
msg.objects[i].pose.orientation.y, msg.objects[i].pose.orientation.z, 0.0)
            for j in range(0,len(self.v2xIDList)):
                if(self.v2xIDList[j] == self.egoVehicleId and msg.objects[i].id == self.egoVehicleId):
                    #print()
                    self.makeMarker(egoVehicle.pos_x - msg.objects[i].pose.position.x, egoVehicle.pos_y -
msg.objects[i].pose.position.y, egoVehicle.pos_z - msg.objects[i].pose.position.z +
msg.objects[i].shape.dimensions[2]/2, msg.objects[i].id, 0.0, 0.0, 0.0, 0.0, msg.objects[i].shape.dimensions[1],
msg.objects[0].shape.dimensions[i], msg.objects[i].shape.dimensions[2])
                if(self.v2xIDList[j] == msg.objects[i].id and self.v2xIDList[j] != self.egoVehicleId):
                    rel_x = msg.objects[i].pose.position.x - egoVehicle.pos_x
                    rel_y = msg.objects[i].pose.position.y  - egoVehicle.pos_y
                    rel_z = msg.objects[i].pose.position.z  - egoVehicle.pos_z

                    if msg.objects[0].pose.orientation.z > 0:
                        alpha_ego = -math.acos(msg.objects[0].pose.orientation.w)*2 ## quaternion angle recovering
                    else:
                        alpha_ego = math.acos(msg.objects[0].pose.orientation.w)*2 ## quaternion angle recovering
```

```
x_ = rel_x*math.cos(alpha_ego) - rel_y*math.sin(alpha_ego)
y_ = rel_x*math.sin(alpha_ego) + rel_y*math.cos(alpha_ego)


if msg.objects[i].pose.orientation.z > 0:
    alpha_target = -math.acos(msg.objects[i].pose.orientation.w)*2 ## quaternion angle recovering
else:
    alpha_target = math.acos(msg.objects[i].pose.orientation.w)*2 ## quaternion angle recovering




print()
print((msg.objects[0].pose.orientation.z))
print((msg.objects[0].pose.orientation.w))
print("EGO Z: " + str(math.sin(alpha_ego/2)))
print("EGO W: " + str(math.cos(alpha_ego/2)))
print("ALPHA EGO: " + str(alpha_ego))
print()

alpha_desired = alpha_target - alpha_ego

if alpha_desired < -math.pi:
    alpha_desired = alpha_desired+math.pi*2

if alpha_desired > math.pi:
    alpha_desired = alpha_desired-math.pi*2




alpha_orientation_w = math.cos(alpha_desired/2)


'''
if(alpha_orientation_w < 0):
    alpha_orientation_w = alpha_orientation_w
    z = math.sqrt(1-(alpha_orientation_w)**2)
else:
    z = math.sqrt(1-(alpha_orientation_w)**2)'''

z = -math.sin(alpha_desired/2)
##print("Alpha Target: " + str(alpha_target))
```

```
                    ##print("Alpha EGO: " + str(alpha_ego))
                    ##print("Alpha Desired: " + str(alpha_desired))


                    ##print("ZZZ: " + str(z))
                    ##print("WWWWW: " + str(alpha_orientation_w))

                    self.makeMarker(x_, y_, rel_z + msg.objects[i].shape.dimensions[2]/2, msg.objects[i].id,
msg.objects[i].pose.orientation.y, msg.objects[i].pose.orientation.x, z, alpha_orientation_w,
                    msg.objects[i].shape.dimensions[1], msg.objects[i].shape.dimensions[0],
msg.objects[i].shape.dimensions[2])


        #print(msg.actors[1].id
            #print(msg.objects[i].pose.position.x, msg.objects[i].pose.position.y)
            #print(msg.objects[i].accel.linear.x)

        #    print(msg.objects[i].pose.orientation.x, msg.objects[i].pose.orientation.y)
        #    self.v2xList.append(V2xCars(msg.objects[i].id, msg.objects[i].pose.position.x,
msg.objects[i].pose.position.y, msg.objects[i].pose.position.z, msg.objects[i].accel.linear.x,
msg.objects[i].accel.linear.y, msg.objects[i].accel.linear.z, "none"))
        #print(msg.actors[1].id)



        #print(dir(msg.objects[1].id))
        #print(msg.objects[1])



    def listener_callback(self, msg):
        self.v2xIDList.clear()
        #print(msg.actors)
        for i in range(0, len(msg.actors)):
            if(msg.actors[i].rolename == "ego_vehicle"):
                self.egoVehicleId = msg.actors[i].id
            if (msg.actors[i].rolename == "ego_vehicle"  or msg.actors[i].rolename == "autopilot"):
                self.v2xIDList.append(msg.actors[i].id)

        #print(len(self.v2xIDList))

    aua = 0
    def makeMarker(self, pos_x, pos_y, pos_z, id, orientation_x, orientation_y, orientation_z, orientation_w,
scale_x, scale_y, scale_z):
```

```
        marker = Marker()
        marker.header.frame_id = "ego_vehicle"
        marker.header.stamp = self.get_clock().now().to_msg()
        marker.ns = "RVIZ_MARKER_ARRAY"
        marker.id = id
        marker.type = marker.CUBE
        marker.action = marker.ADD
        marker.scale.x = scale_y
        marker.scale.y = scale_x
        marker.scale.z = scale_z
        marker.color.r = 1.0
        marker.color.g = 0.0
        marker.color.b = 0.0
        marker.color.a = 1.0
        marker.pose.orientation.x = 0.0#orientation_x
        marker.pose.orientation.y = 0.0 #orientation_y
        marker.pose.orientation.z = orientation_z
        marker.pose.orientation.w = orientation_w
        marker.pose.position.x = pos_x
        marker.pose.position.y = pos_y
        marker.pose.position.z = pos_z

        self.marker_array.markers.append(marker)
        #print("WWWWWW: " + str(0.0 + math.cos(self.aua/2)))

        return self.marker_array


class V2xCars:
    role = "none"
    id = 0
    pos_x = 0
    pos_y = 0
    pos_z = 0
    acc_x = 0
    acc_y = 0
    acc_z = 0

    ori_x = 0
    ori_y = 0
    ori_z = 0
    ori_w = 0
```

```
    def __init__(self, id, pos_x, pos_y, pos_z, acc_x, acc_y, acc_z, role, ori_x, ori_y, ori_z, ori_w):
        self.id = id
        self.pos_x = pos_x
        self.pos_y = pos_y
        self.pos_z = pos_z
        self.acc_x = acc_x
        self.acc_y = acc_y
        self.acc_z = acc_z
        self.role = role
        self.ori_x = ori_x
        self.ori_y = ori_y
        self.ori_z = ori_z
        self.ori_w = ori_w




def main(args=None):
    rclpy.init(args=args)

    minimal_subscriber = MinimalSubscriber()

    #minimal_publisher = MinimalPublisher()

    rclpy.spin(minimal_subscriber)
    #rclpy.spin(minimal_publisher)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)

    # Destroy the node explicitly
    # (optional - otherwise it will be done automatically
    # when the garbage collector destroys the node object)
    #minimal_publisher.destroy_node()
    minimal_subscriber.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```